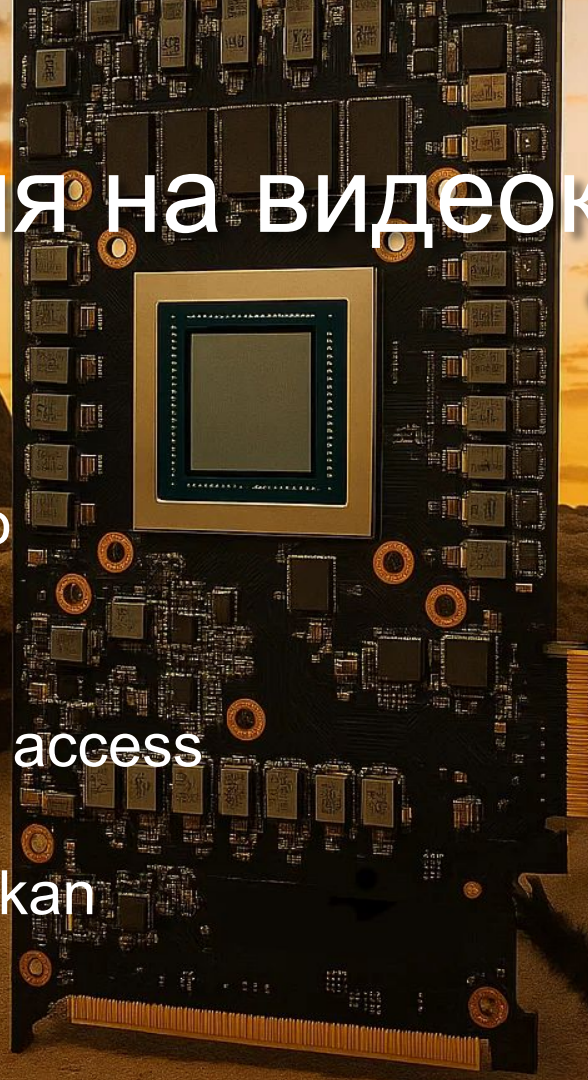


Вычисления на видеокартах

Лекция 2

- Архитектура GPU
- Модель массового параллелизма
- code divergence
- coalesced memory access
- Код кернала:
OpenCL CUDA Vulkan



Vulkan



nVIDIA
CUDA



OpenCL™



План лекции

Большая часть из лекции “CS Space: Видеокарты: что они могут?”

- 1) **Вычисления:**
shared instruction pointer, code divergence
- 2) **Работа с памятью:**
hyper-threading, latency hiding, occupancy, registers pressure/spilling, coalesced memory access pattern, cache lines, local/shared memory
- 3) **Общая картина ЭВМ-архитектуры:** CPU - RAM - PCI-E - VRAM - GPU
- 4) **Модель вычислений массового параллелизма**
- 5) **Профилирование и оптимизация:** NVIDIA Nsight, SoL анализ

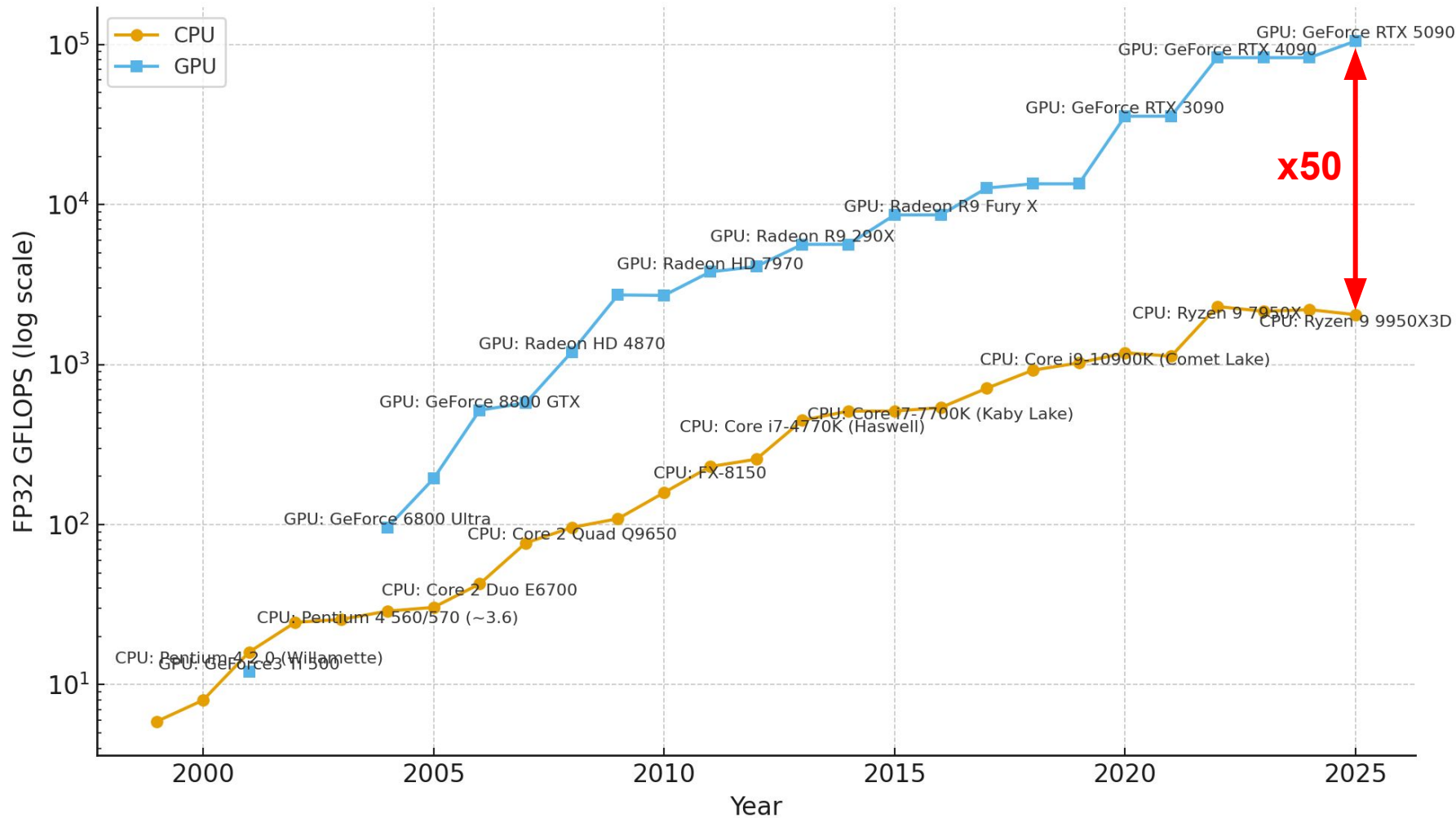
И обсудим как выглядит код на видеокарте:

- 6) **Пример кода:** A+B на C++ (OpenMP), OpenCL, CUDA, Vulkan (GLSL)

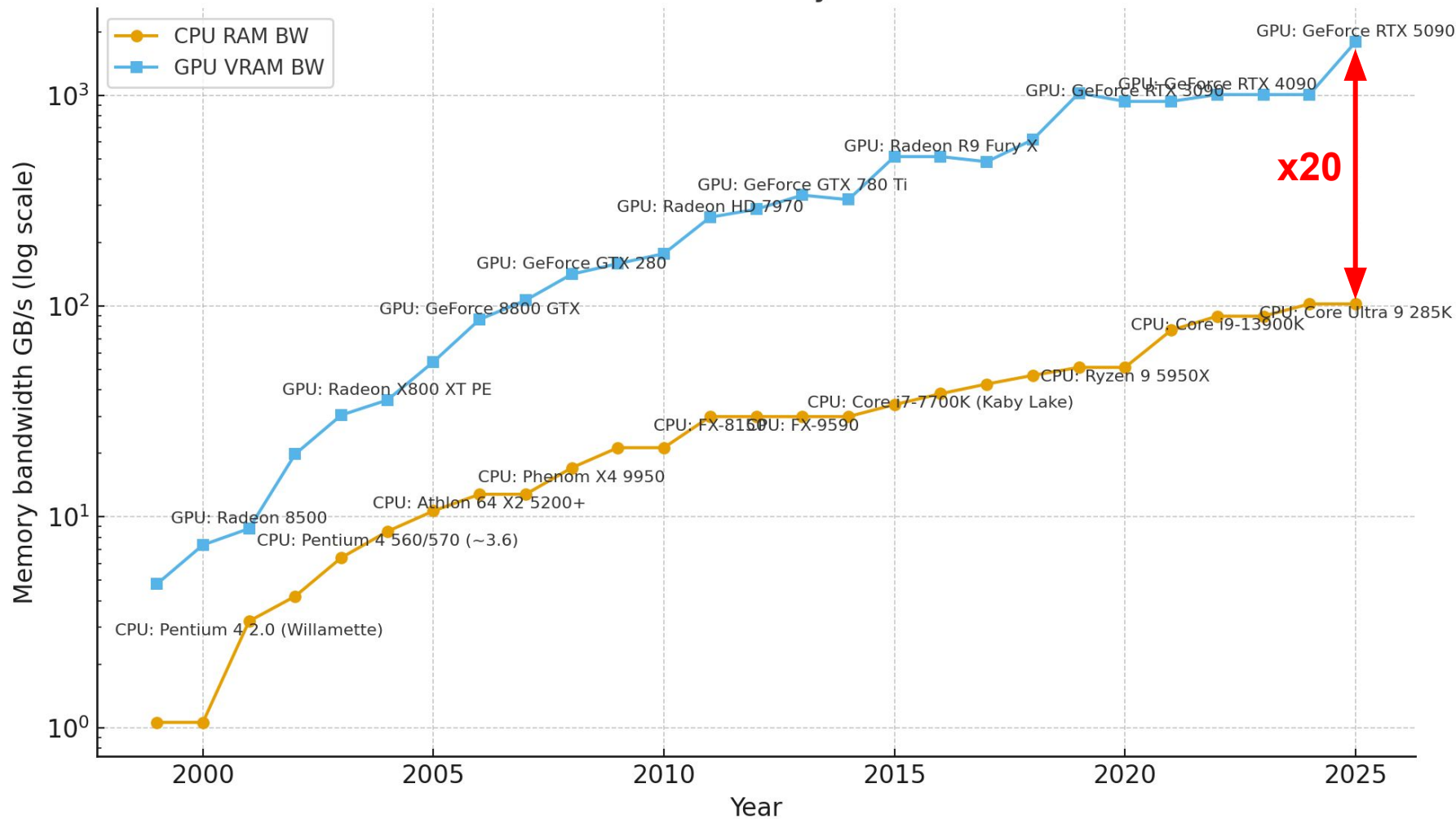
Глава 1: Вычисления

shared instruction pointer, code divergence

CPU vs GPU FP32 Peak

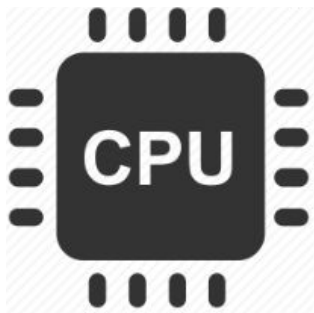


CPU vs GPU Memory Bandwidth



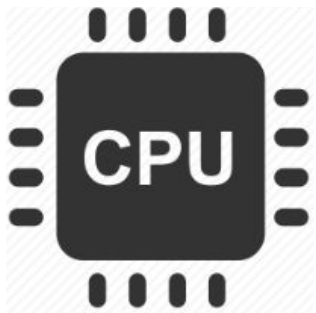
Архитектура

$100 \cdot 10^9$ FLOPS (**F**loating-point operations **p**er **s**econd)

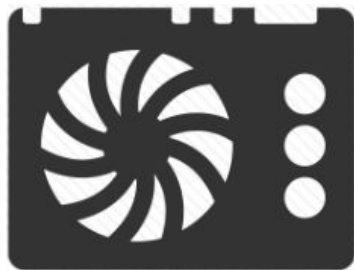


Архитектура

$100 \cdot 10^9$ FLOPS (**F**loating-point operations **p**er **s**econd)



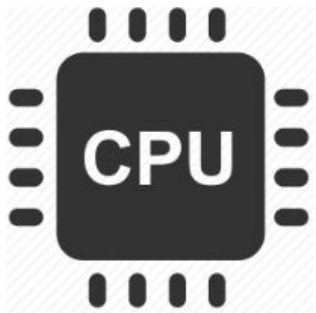
$100 \cdot 10^{12}$ FLOPS (**х1000 раз больше**)



GPU

Архитектура

$100 \cdot 10^9$ FLOPS



40 GB/s memory bandwidth

$100 \cdot 10^{12}$ FLOPS (**х1000 раз больше**)

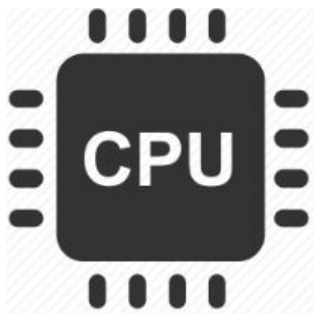


1000 GB/s (**х25 раз больше**)

GPU

Архитектура

$100 \cdot 10^9$ FLOPS



Мало ядер, но они **МОЩНЫЕ**

6 GHz



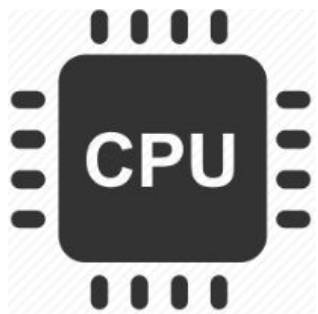
$100 \cdot 10^{12}$ FLOPS



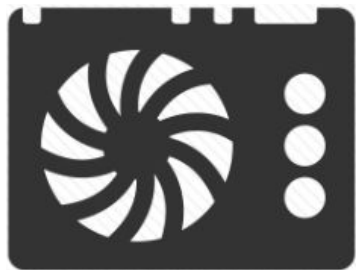
GPU

Архитектура

$100 \cdot 10^9$ FLOPS



$100 \cdot 10^{12}$ FLOPS



GPU

Мало ядер, но они **МОЩНЫЕ**

6 GHz

ТЫСЯЧИ слабых ядер

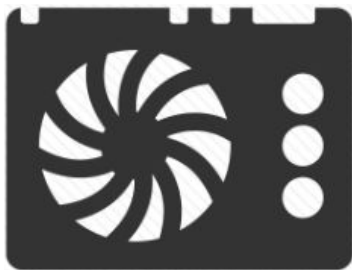
1 GHz



Архитектура

Как уместить ТЫСЯЧИ ядер?
На CPU ведь это почему-то невозможно!

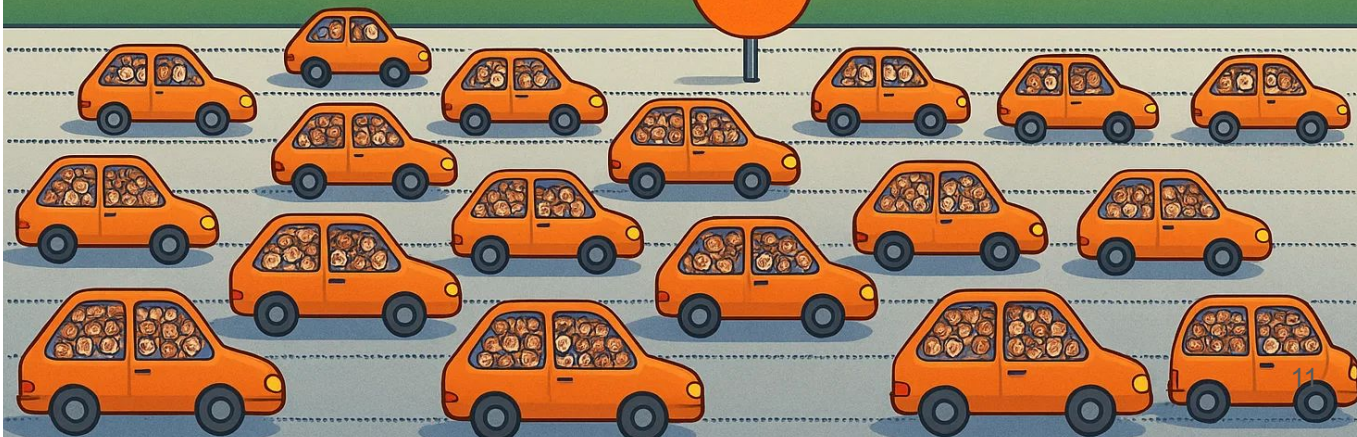
$100 \cdot 10^{12}$ FLOPS



GPU

ТЫСЯЧИ слабых ядер

1 GHz



Архитектура

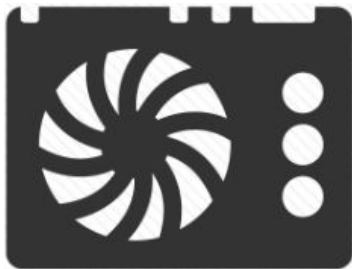


warp

32 слабых медленных
CUDA ядер

Как уместить ТЫСЯЧИ ядер?
На CPU ведь это почему-то невозможно!

$100 \cdot 10^{12}$ FLOPS



GPU



ТЫСЯЧИ слабых ядер

1 GHz

SM - Streaming Multiprocessor



Архитектура

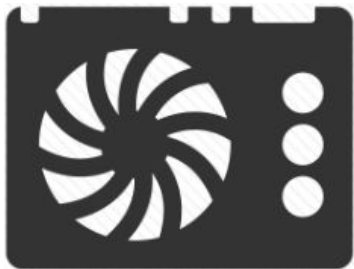


warp

32 слабых медленных
CUDA ядер

Как уместить ТЫСЯЧИ ядер?
На CPU ведь это почему-то невозможно!

$100 \cdot 10^{12}$ FLOPS



GPU



SM - Streaming Multiprocessor



RTX 3090: **10496 CUDA cores** = 82 SM · 4 warps · 32 ALUs



SM - Streaming Multiprocessor

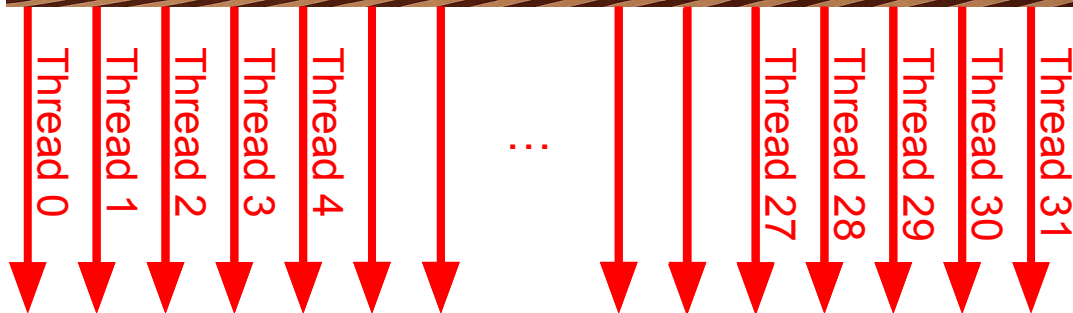


<https://www.techpowerup.com/review/nvidia-geforce-ampere-architecture-board-design-gaming-tech-software/3.html>

CPU ядро



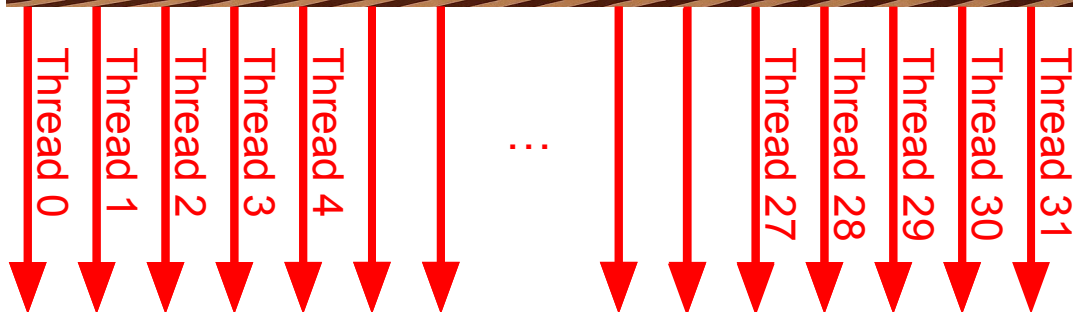
GPU warp



Как уместить ТЫСЯЧИ ядер?
На CPU ведь это почему-то невозможно!

Архитектура

GPU warp

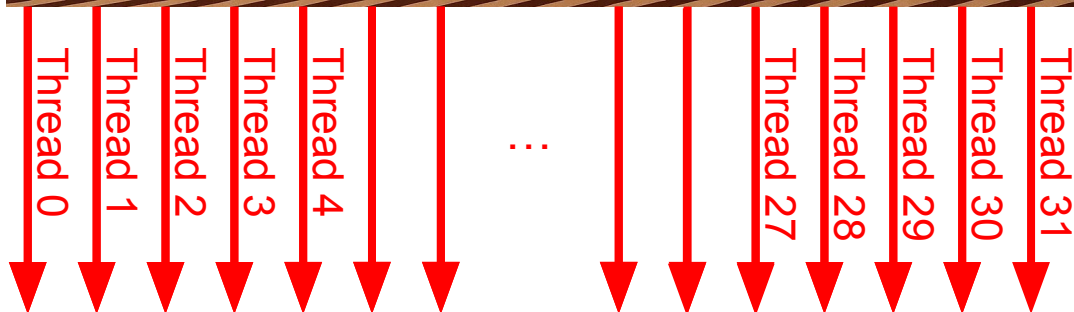


Один Instruction pointer
на все потоки warp-a!

```
239 → int warpThreadID = threadIdx.x % 32;  
240 int result = 0;  
241 if (warpThreadID < 16) {  
242     result = dataA[warpThreadID];  
243 } else {  
244     result = dataB[warpThreadID];  
245 }
```

Архитектура

GPU warp

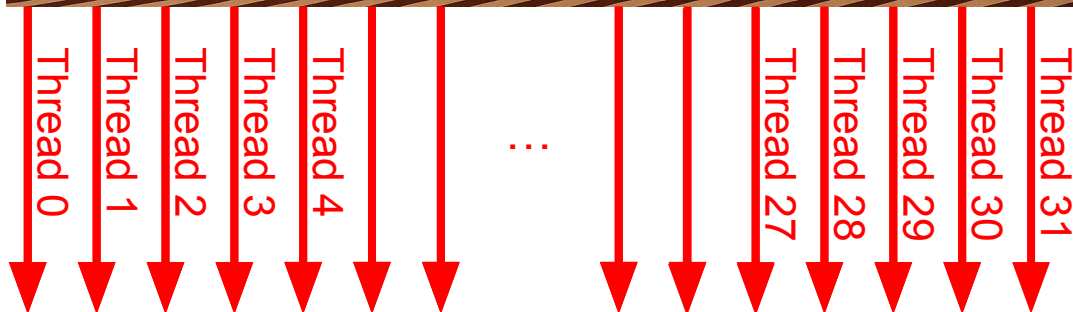


Один Instruction pointer
на все потоки warp-a!

```
239 int warpThreadID = threadIdx.x % 32;  
240 → int result = 0;  
241 if (warpThreadID < 16) {  
242     result = dataA[warpThreadID];  
243 } else {  
244     result = dataB[warpThreadID];  
245 }
```

Архитектура

GPU warp

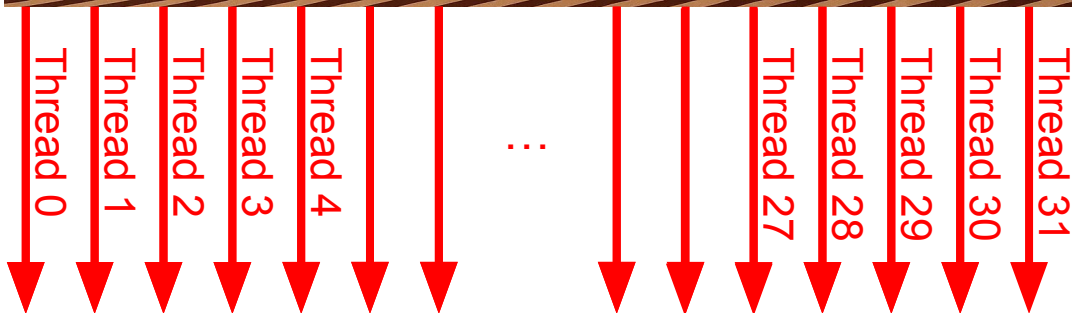


```
239 int warpThreadID = threadIdx.x % 32;  
240 int result = 0;  
241 → if (warpThreadID < 16) {  
242     result = dataA[warpThreadID];  
243 } else {  
244     result = dataB[warpThreadID];  
245 }
```

Один Instruction pointer
на все потоки warp-a!

Архитектура

GPU warp

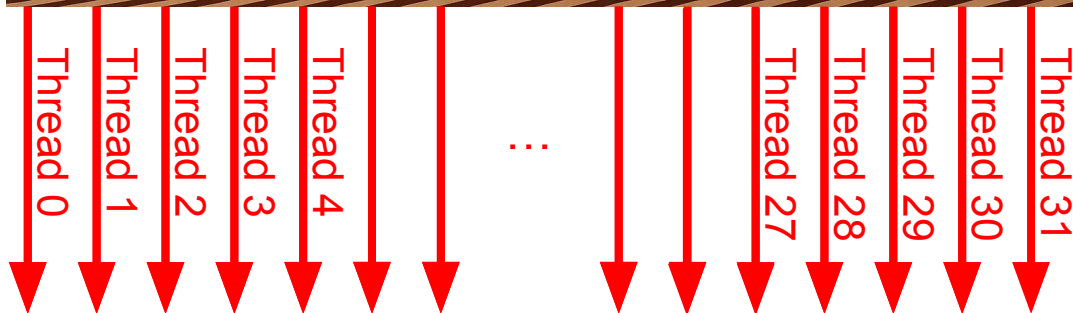


Один Instruction pointer
на все потоки warp-a!

```
239 int warpThreadID = threadIdx.x % 32;  
240 int result = 0;  
241 if (warpThreadID < 16) {  
242     result = dataA[warpThreadID];  
243 } else {  
244     result = dataB[warpThreadID];  
245 }
```

Архитектура

GPU warp

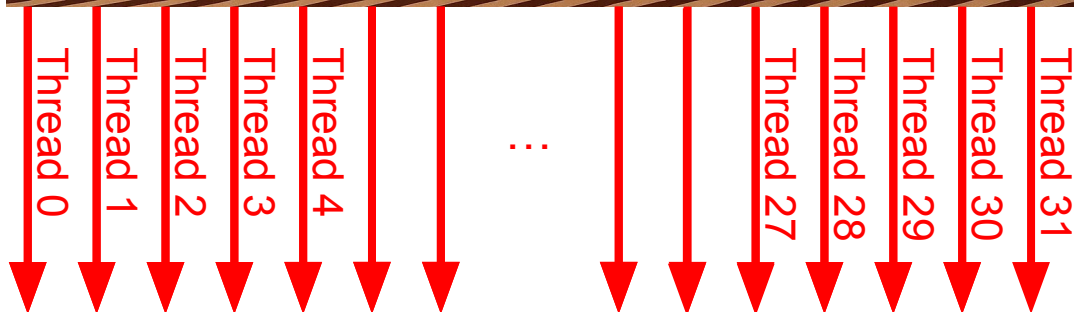


Один Instruction pointer
на все потоки warp-a!

```
239 int warpThreadID = threadIdx.x % 32;  
240 int result = 0;  
241 if (warpThreadID < 16) {  
242     result = dataA[warpThreadID];  
243 } else {  
244     result = dataB[warpThreadID];  
245 }
```

Архитектура

GPU warp

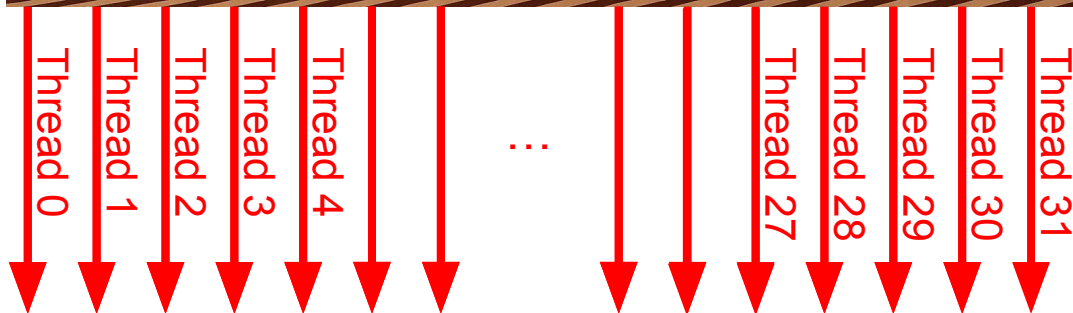


Один Instruction pointer
на все потоки warp-a!

```
239 int warpThreadID = threadIdx.x % 32;  
240 int result = 0;  
241 if (warpThreadID < 16) {  
242     result = dataA[warpThreadID];  
243 } else {  
244 → result = dataB[warpThreadID];  
245 }
```

Архитектура

GPU warp



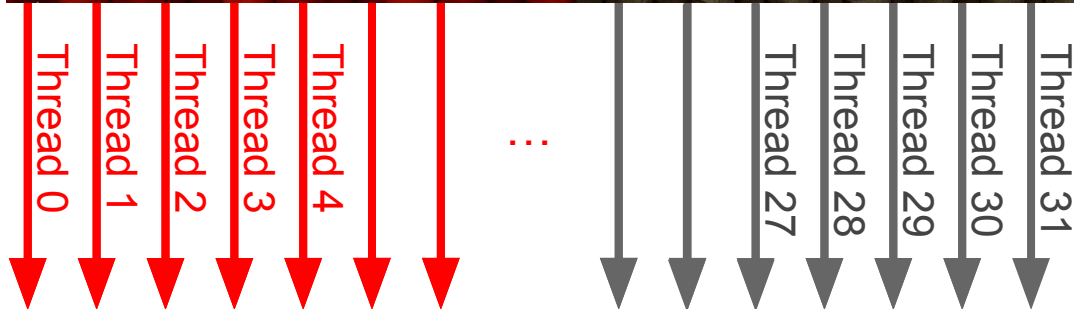
Один Instruction pointer
на все потоки warp-a!

```
239 int warpThreadID = threadIdx.x % 32;  
240 int result = 0;  
241 if (warpThreadID < 16) {  
242     result = dataA[warpThreadID];  
243 } else {  
244     result = dataB[warpThreadID];  
245 }
```

Выходит зайдём в обе ветки if-а?

Архитектура

GPU warp

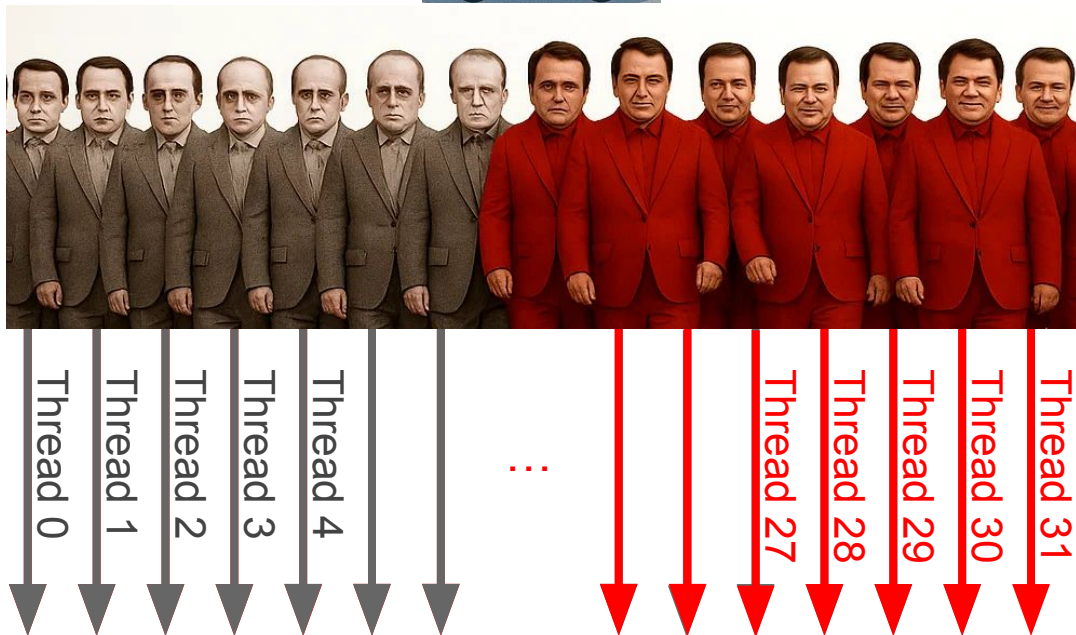
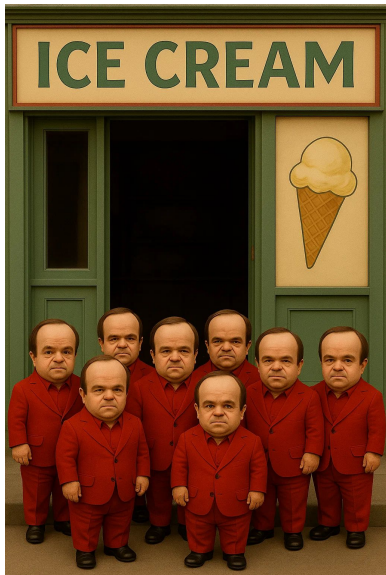


```
239 int warpThreadID = threadIdx.x % 32;
240 int result = 0;
241 if (warpThreadID < 16) {
242     result = dataA[warpThreadID]; // exec mask: [++++ ++++++ ++++++ ++++++ ---- - - - -]
243 } else {
244     result = dataB[warpThreadID]; // exec mask: [- - - - - - - - - - ++++++ ++++++ ++++++ ++++++]
245 }
```

Один Instruction pointer
на все потоки warp-a!

Архитектура

GPU warp

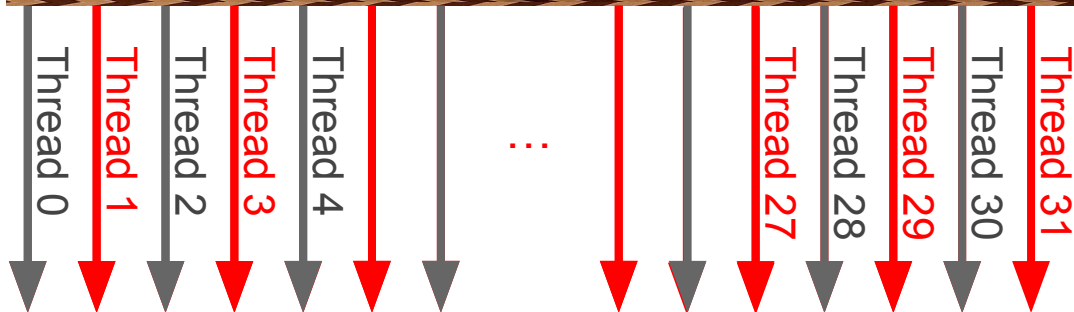


Один Instruction pointer
на все потоки warp-a!

```
239 int warpThreadID = threadIdx.x % 32;  
240 int result = 0;  
241 if (warpThreadID < 16) {  
242     result = dataA[warpThreadID]; // exec mask: [++++ ++++++ ++++++ ++++++ ---- ---- ---- ----]  
243 } else {  
244     result = dataB[warpThreadID]; // exec mask: [---- ---- ---- ---- ++++++ ++++++ ++++++ ++++++]  
245 }
```

Архитектура

GPU warp

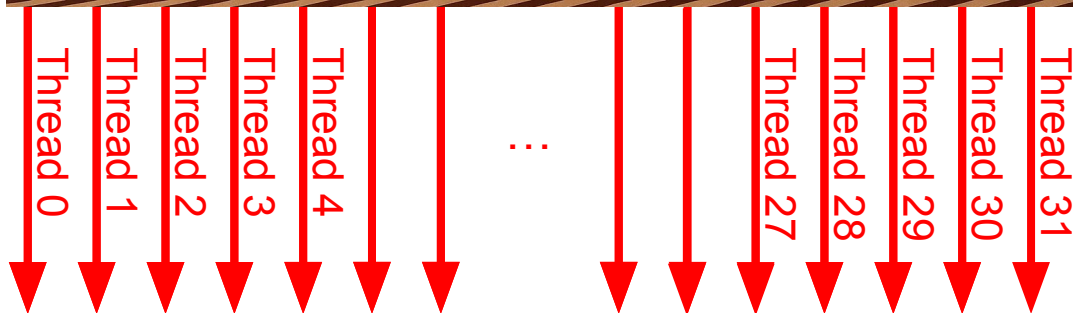


Один Instruction pointer
на все потоки warp-a!

```
239 int warpThreadID = threadIdx.x % 32;  
240 int result = 0;  
241 if (warpThreadID % 2 == 0) {  
242     result = dataA[warpThreadID]; // exec mask: [+--+ +--+ +--+ +--+ +--+ +--+ +--+]  
243 } else {  
244     result = dataB[warpThreadID]; // exec mask: [-+++ -+++ -+++ -+++ -+++ -+++ -+++ -+++]  
245 }
```

Архитектура

GPU warp



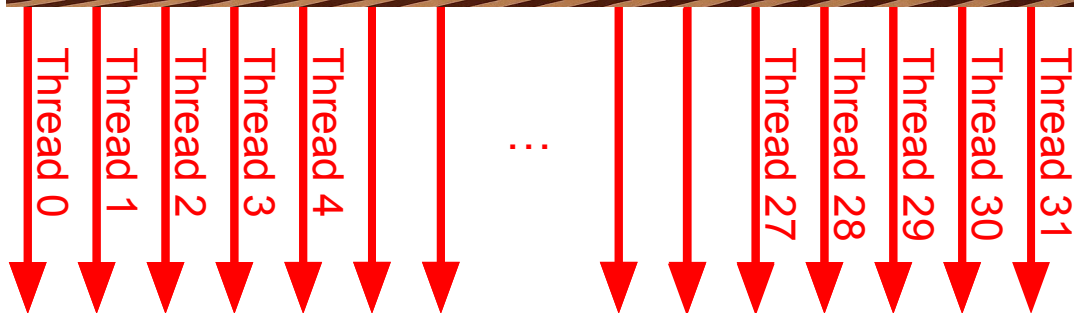
Один Instruction pointer
на все потоки warp-a!

Сколько “времени” будет работать warp?

```
241 if (predicate1) {  
242     if (predicate2) {  
243         if (predicate3) {  
244             // A1  
245         } else {  
246             // A2  
247         }  
248     } elif (predicate4) {  
249         // A3  
250     }  
251 } else {  
252     // A4  
253 }
```

Архитектура

GPU warp



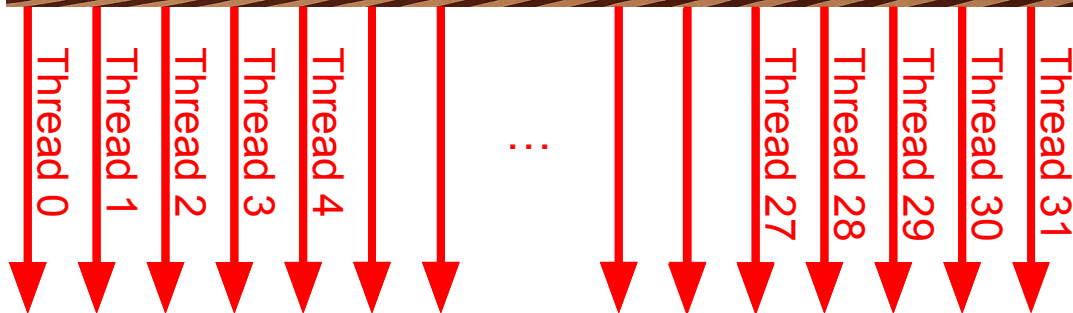
Один Instruction pointer
на все потоки warp-a!

```
241 if (predicate1) {  
242     if (predicate2) {  
243         if (predicate3) {  
244             // A1  
245         } else {  
246             // A2  
247         }  
248     } elif (predicate4) {  
249         // A3  
250     }  
251 } else {  
252     // A4  
253 }
```

Сколько “времени” будет работать warp?
 $A1 + A2 + A3 + A4$ при **code divergence**

Архитектура

GPU warp

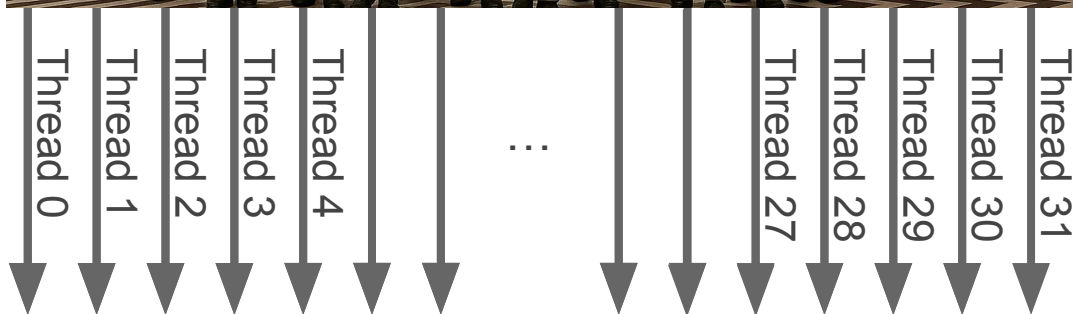
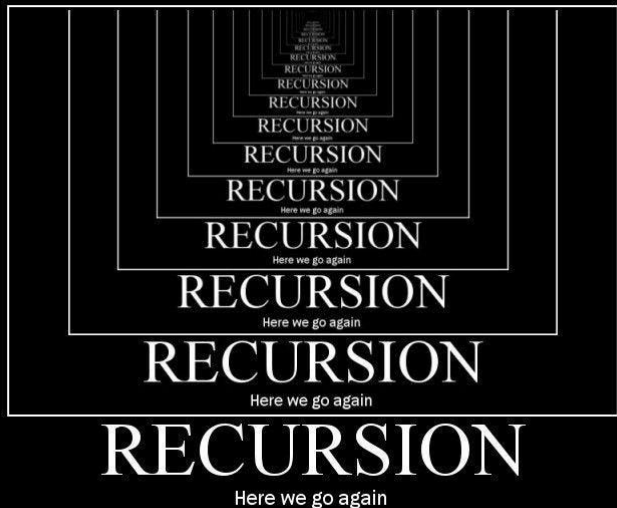


Один Instruction pointer
на все потоки warp-a!

Сколько “времени” будет работать достаточно глубокая рекурсия?

Архитектура

GPU warp

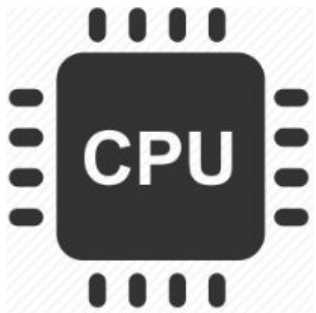


Один Instruction pointer
на все потоки warp-a!

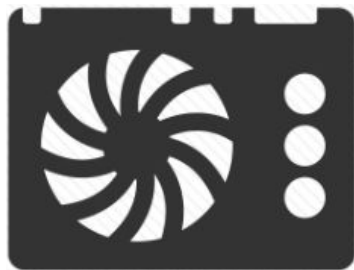
Глава 2: Работа с памятью

hyper-threading, latency hiding, occupancy, registers pressure/spilling,
coalesced memory access pattern, cache lines, local/shared memory, bank conflicts

Архитектура



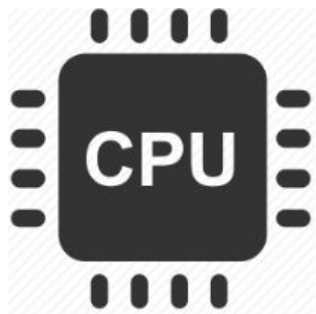
40
GB/s



1000
GB/s

GPU

Архитектура



40
GB/s

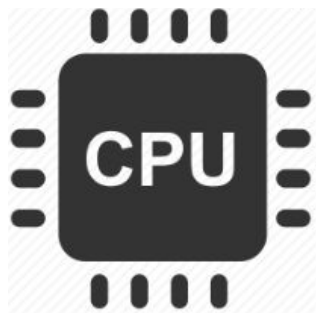
Память - малая пропускная способность
НИЗКАЯ LATENCY



1000
GB/s

GPU

Архитектура



40
GB/s



GPU

1000
GB/s

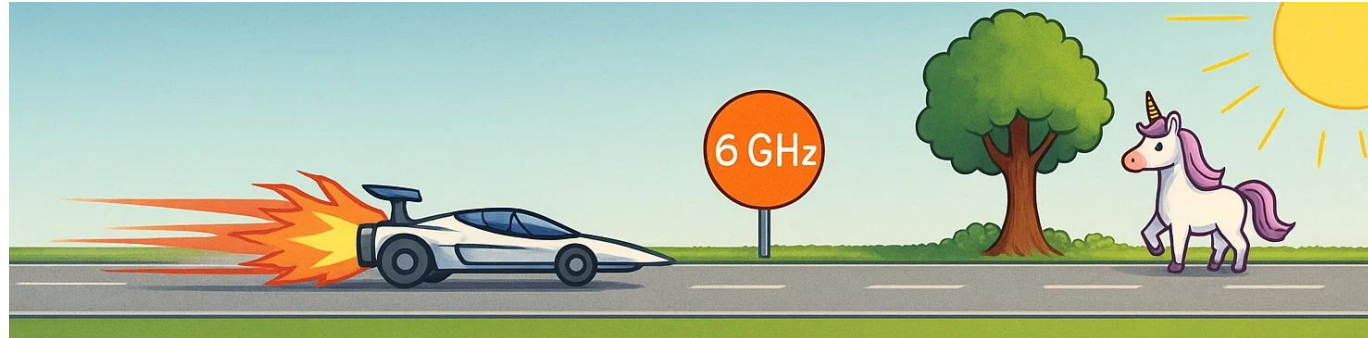
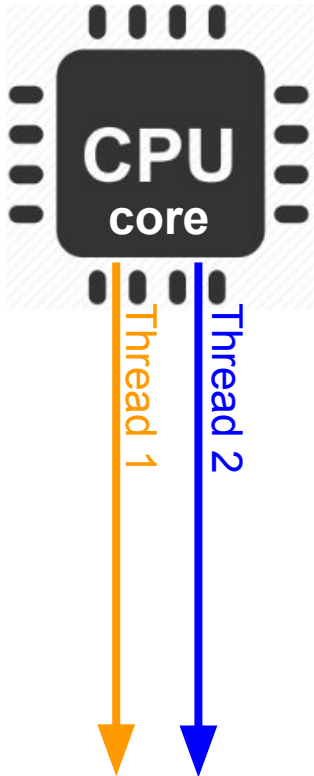
Память - малая пропускная способность
НИЗКАЯ LATENCY



Память - **ОГРОМНАЯ** пропускная способность
но большая latency



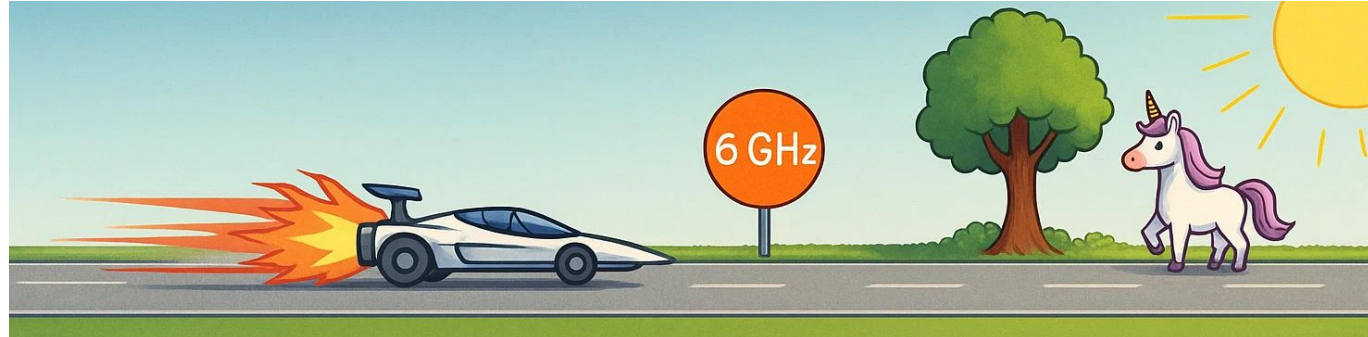
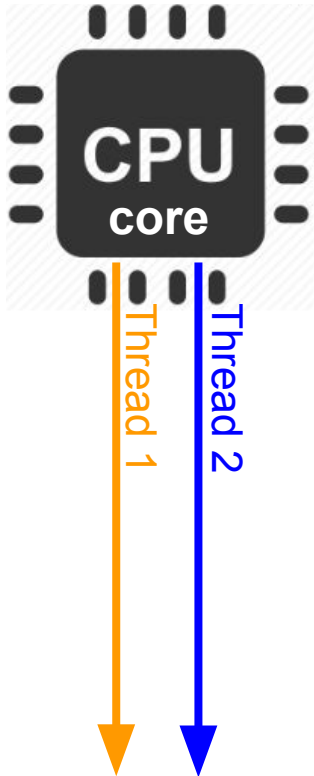
Архитектура CPU: многопоточность



Многопоточность благодаря переключению контекста!

Что нужно перещелкнуть в состоянии ЦПУ ядра для другого потока?

Архитектура CPU: многопоточность

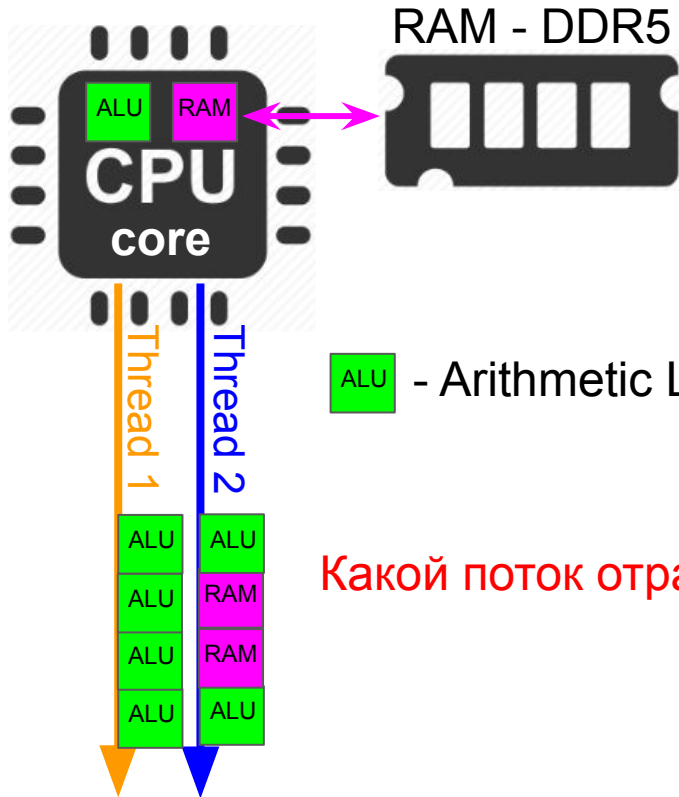


Многопоточность благодаря переключению контекста!

Context switch:

- Обновляет Instruction pointer (указатель на строку кода)
- Подгружает значения регистров процессора

Архитектура CPU: Hyper-Threading, SMT

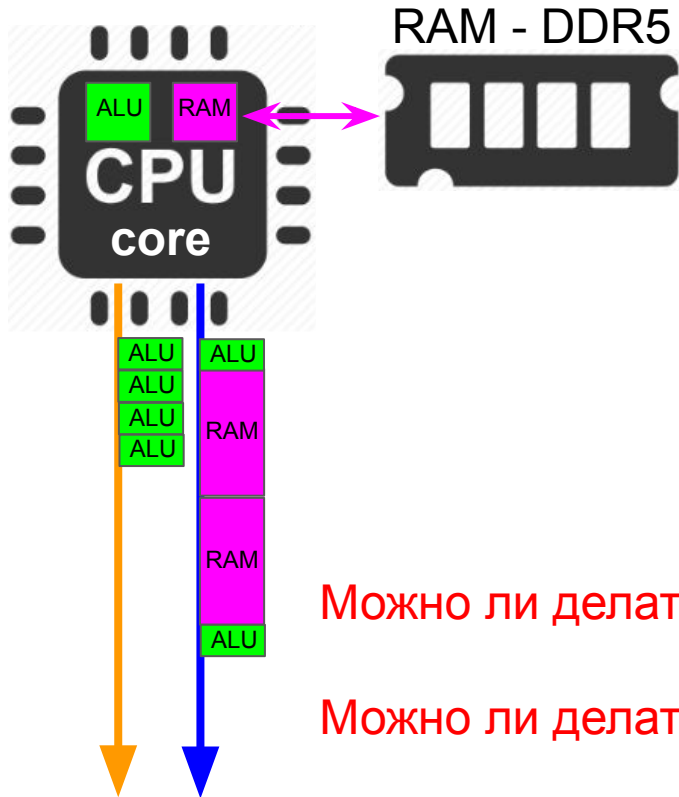


 - Arithmetic Logical Unit

Какой поток отработает быстрее?



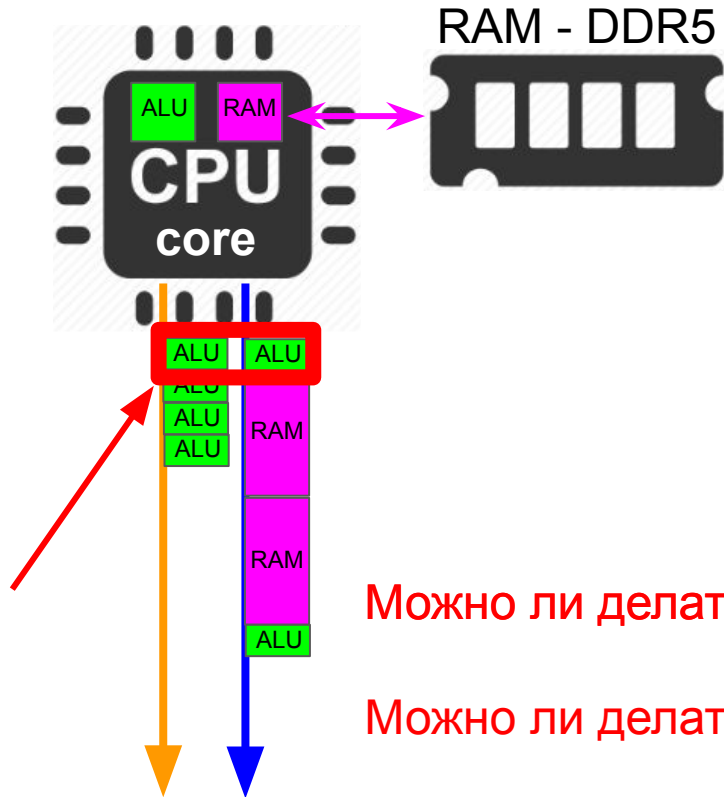
Архитектура CPU: Hyper-Threading, SMT



Можно ли делать **ALU** пока ждем **RAM** ?

Можно ли делать **ALU** пока ждем **ALU** ?

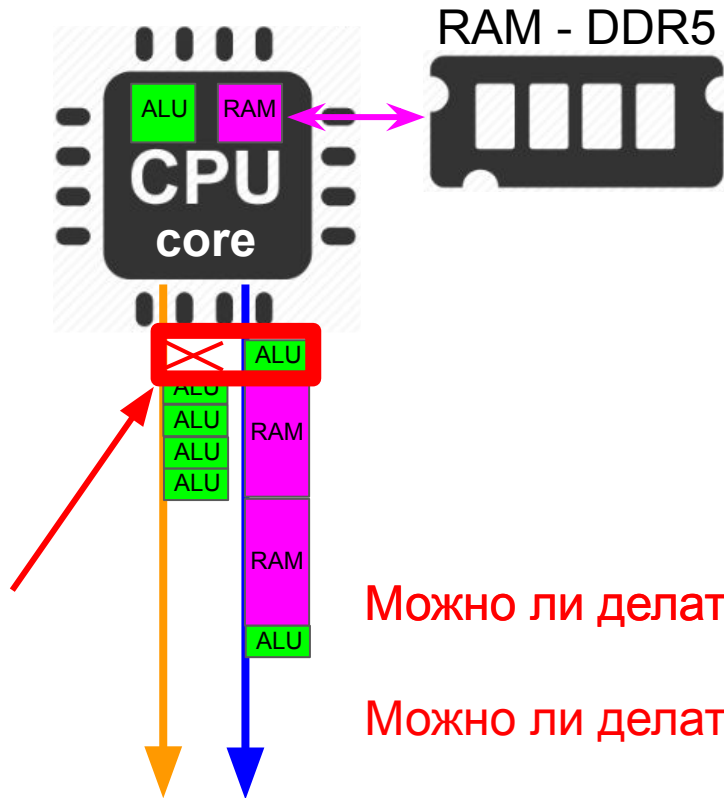
Архитектура CPU: Hyper-Threading, SMT



Можно ли делать **ALU** пока ждем **RAM** ?

Можно ли делать **ALU** пока ждем **ALU** ?

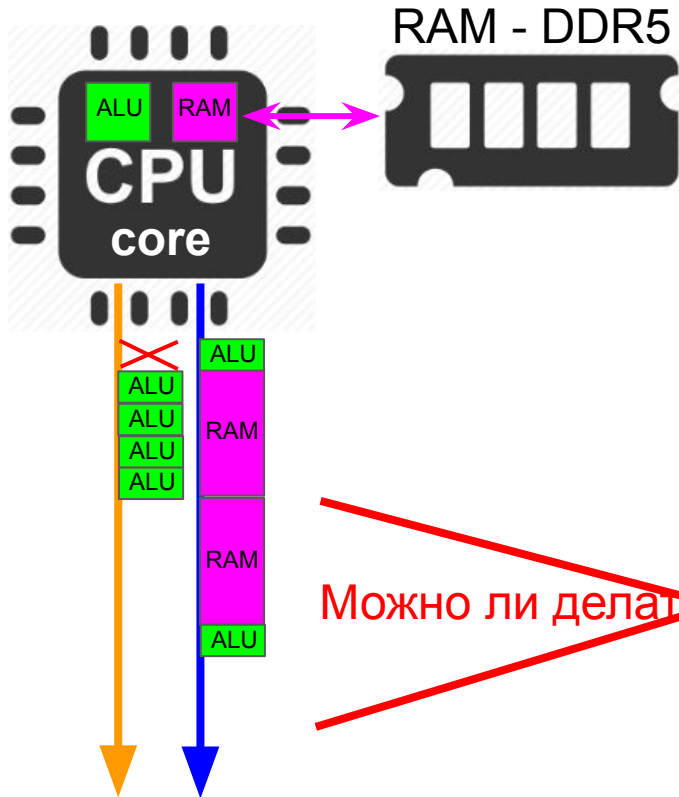
Архитектура CPU: Hyper-Threading, SMT



Можно ли делать **ALU** пока ждем **RAM** ?

Можно ли делать **ALU** пока ждем **ALU** ?

Архитектура CPU: Hyper-Threading, SMT

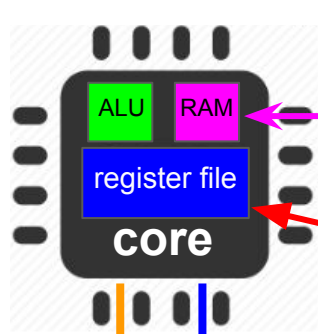


Можно ли делать ALU пока ждем RAM ?

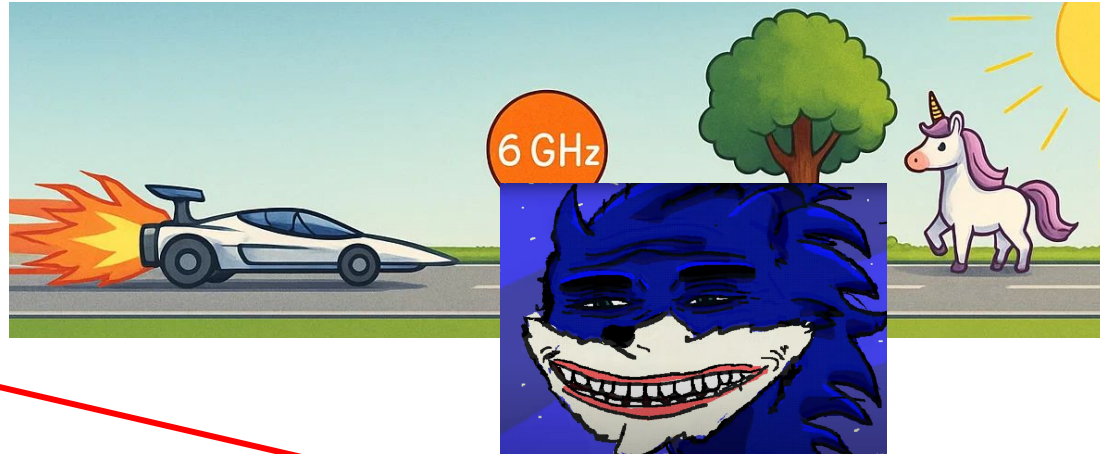
Context switch - дорого!
Долго подгружает регистры!



Архитектура CPU: Hyper-Threading, SMT



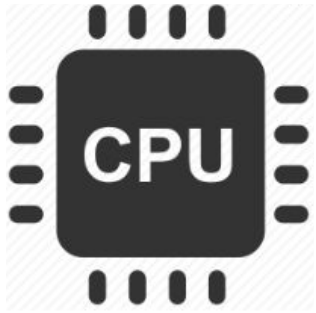
RAM - DDR5



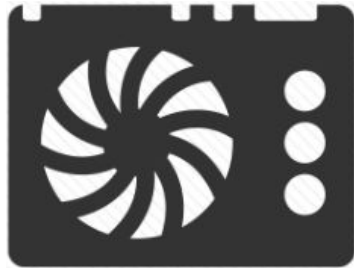
Давайте держать два множества регистров!

Можно делать **ALU** пока ждем **RAM** ! Это и есть SMT и HT!

Архитектура GPU



40
GB/s



1000
GB/s

Память - малая пропускная способность
НИЗКАЯ LATENCY



Память - **ОГРОМНАЯ** пропускная способность
но большая latency



Архитектура GPU

1 x Streaming Multiprocessor

SM - Streaming Multiprocessor

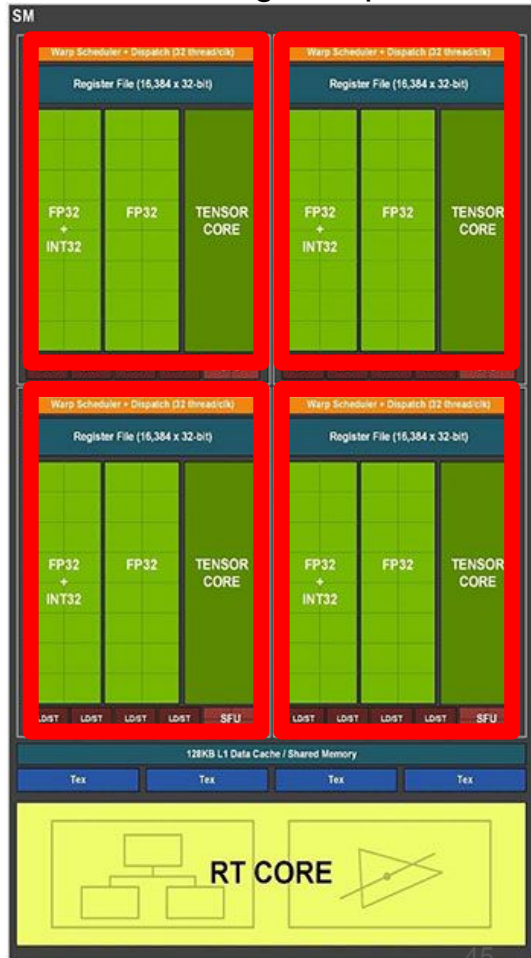


Архитектура GPU

1 x Streaming Multiprocessor

4 x Processing Blocks

SM - Streaming Multiprocessor



Архитектура GPU

1 x Streaming Multiprocessor

4 x Processing Blocks

32 x слабых ALU / FP32

SM - Streaming Multiprocessor



Архитектура GPU



warp

32 слабых медленных
виртуальных потоков

SM - Streaming Multiprocessor



Архитектура GPU



wavefront
64 threads



nvidia
warp
32 threads



warp

32 слабых медленных
виртуальных потоков

SM - Streaming Multiprocessor



Архитектура GPU



wavefront

64 threads

**Compute Units
(CU)**



NVIDIA

warp

32 threads

**Streaming Multiprocessor
(SM)**



warp

32 слабых медленных
виртуальных потоков

SM - Streaming Multiprocessor



Архитектура GPU



warp

32 слабых медленных виртуальных потоков

32 слабых медленных
CUDA Cores (**ALU/FP32**)

В чем разница?

SM - Streaming Multiprocessor



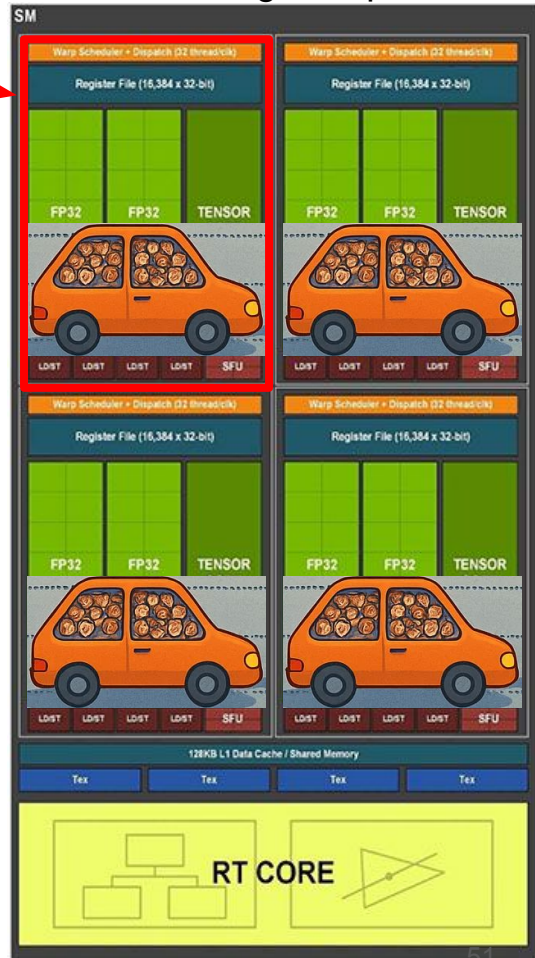
Архитектура GPU



warp

32 слабых медленных
виртуальных потоков

SM - Streaming Multiprocessor



Архитектура GPU



warp

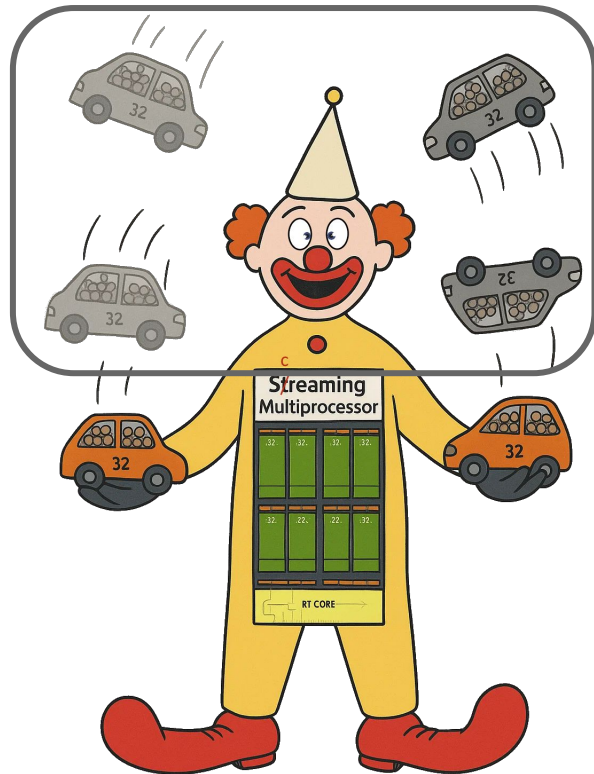
32 слабых медленных
виртуальных потоков

Register File для быстрого
жонглирования warp-ами!
(быстрый "context switch")

SM - Streaming Multiprocessor



Архитектура GPU



warp

32 слабых медленных виртуальных потоков

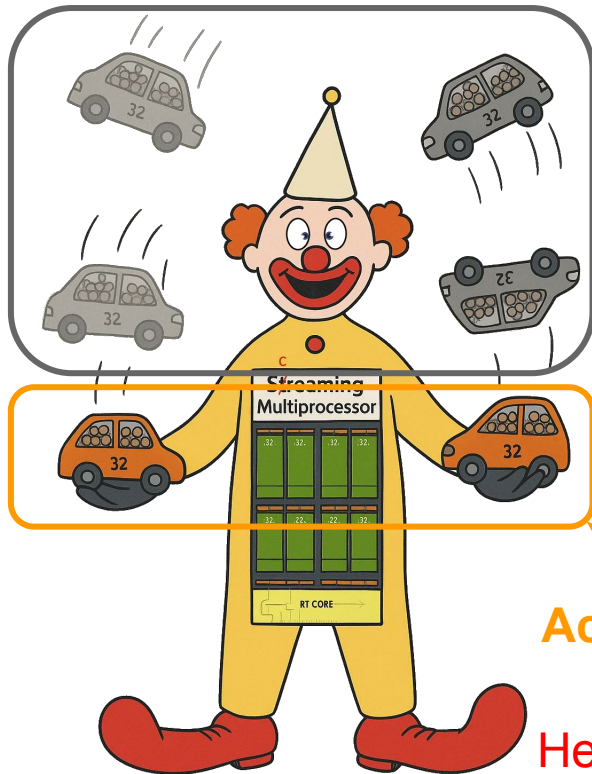
Register File для быстрого жонглирования warp-ами! (быстрый "context switch")

Inactive warps: ждут когда придут данные из VRAM

SM - Streaming Multiprocessor



Архитектура GPU



warp

32 слабых медленных виртуальных потоков

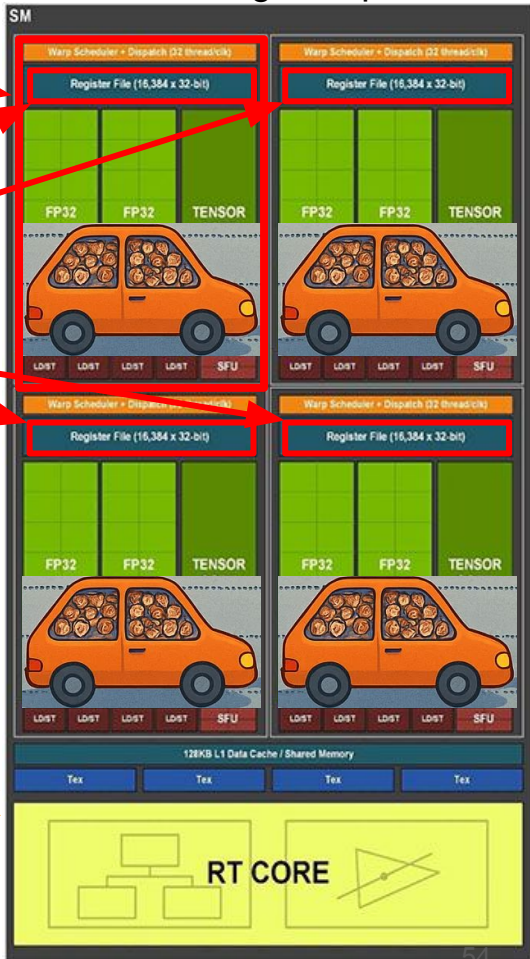
Register File для быстрого жонглирования warp-ами! (быстрый "context switch")

Inactive warps: ждут когда придут данные из VRAM

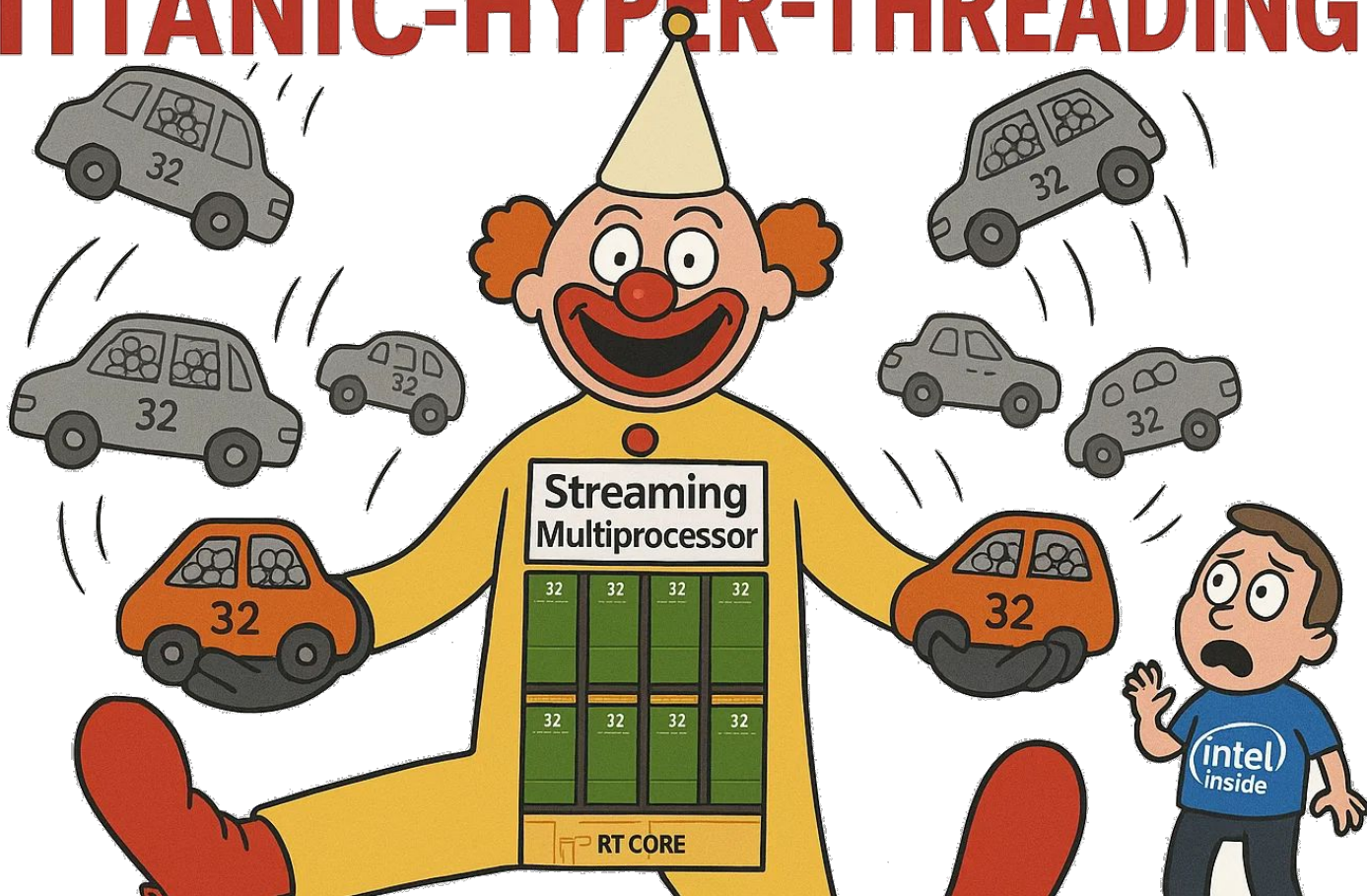
Active warps: данные в регистрах
Работаем-работаем! 🧐

Не напоминает ли вам это что-то?

SM - Streaming Multiprocessor



ULTRA SUPER MEGA GIGA TITANIC-HYPER-THREADING



Архитектура GPU



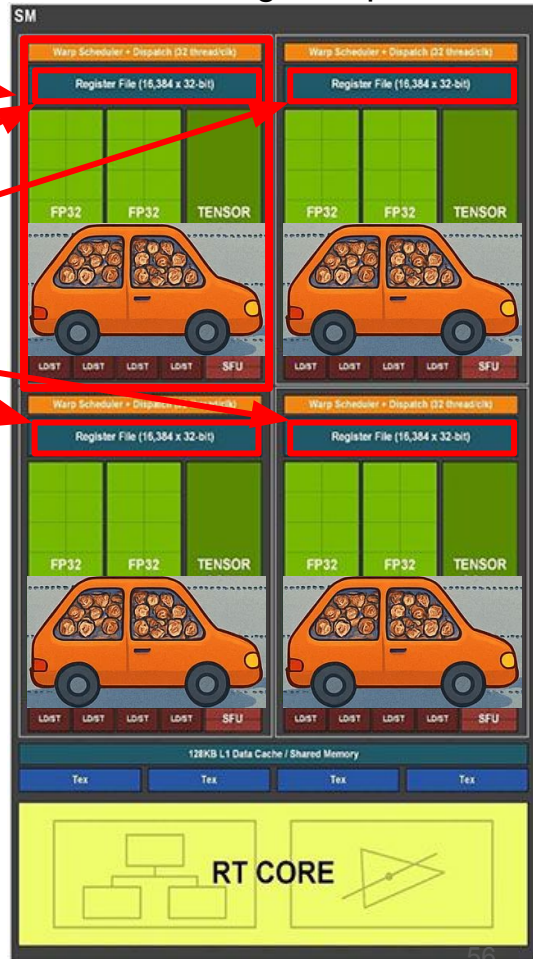
warp

32 слабых медленных
виртуальных потоков

Register File для быстрого
жонглирования warp-ами!
(быстрый "context switch")

Occupancy = насколько
много доступно warp-ов для
жонглирования.

SM - Streaming Multiprocessor



Архитектура GPU



warp

32 слабых медленных
виртуальных потоков

Register File для быстрого
жонглирования warp-ами!
(быстрый "context switch")

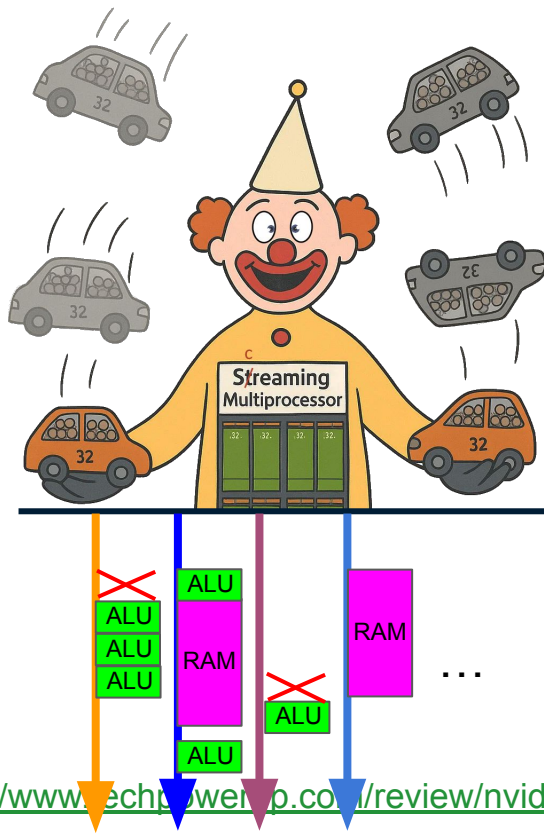
Осцирансу = насколько
много доступно warp-ов для
жонглирования.

**Если осцирансу высокая,
то что?**

SM - Streaming Multiprocessor



Архитектура GPU



warp

32 слабых медленных виртуальных потоков

Register File для быстрого жонглирования warp-ами! (быстрый "context switch")

Осцирансу = насколько много доступно warp-ов для жонглирования.

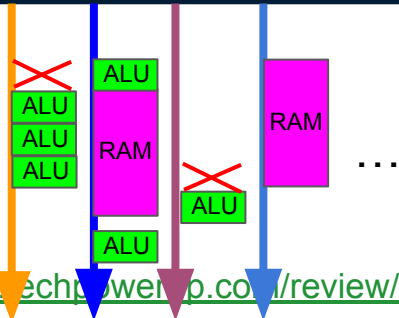
Если осцирансу высокая, то хорошо скрывается latency доступа к памяти т.к. всегда найдется готовый warp!

SM - Streaming Multiprocessor



Архитектура GPU

А почему у SM может быть разное число warp-ов?



warp

32 слабых медленных виртуальных потоков

Register File для быстрого жонглирования warp-ами! (быстрый “context switch”)

Осцирансу = насколько много доступно warp-ов для жонглирования.

Если осцирансу высокая, то хорошо скрывается latency доступа к памяти т.к. всегда найдется готовый warp!

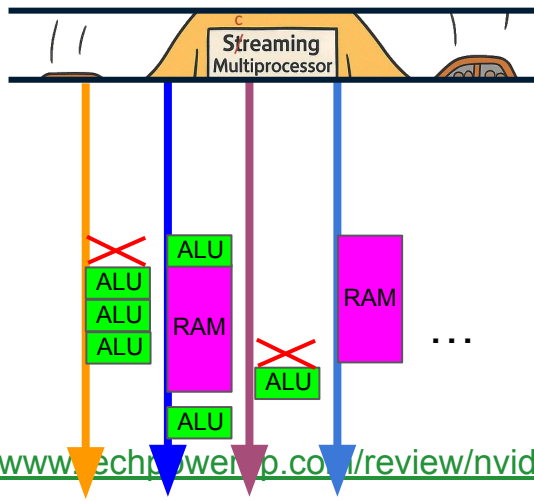
SM - Streaming Multiprocessor



Архитектура GPU

А почему у SM может быть
разное число warp-ов?

Registers Pressure!



32 слабых медленных
виртуальных потоков

Register File для быстрого
жонглирования warp-ами!
(быстрый “context switch”)

Осцирансу = насколько
много доступно warp-ов для
жонглирования.

Если осцирансу высокая, то
хорошо скрывается latency
доступа к памяти т.к. всегда
найдется готовый warp!



warp

SM - Streaming Multiprocessor

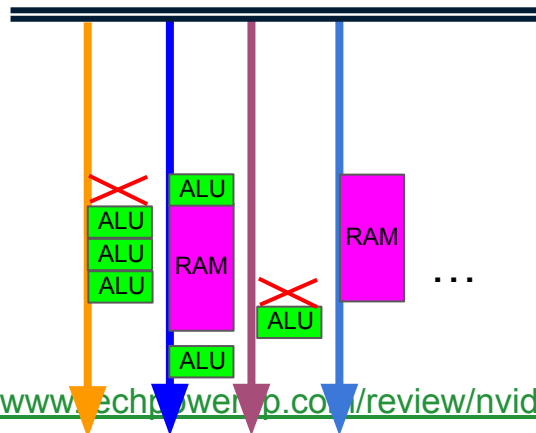


Архитектура GPU

А почему у SM может быть
разное число warp-ов?

Registers Pressure!

Вплоть до **Registers Spilling!**



Register File для быстрого
жонглирования warp-ами!
(быстрый “context switch”)

Occupancy = насколько
много доступно warp-ов для
жонглирования.

Если occupancy высокая, то
хорошо скрывается latency
доступа к памяти т.к. всегда
найдется готовый warp!



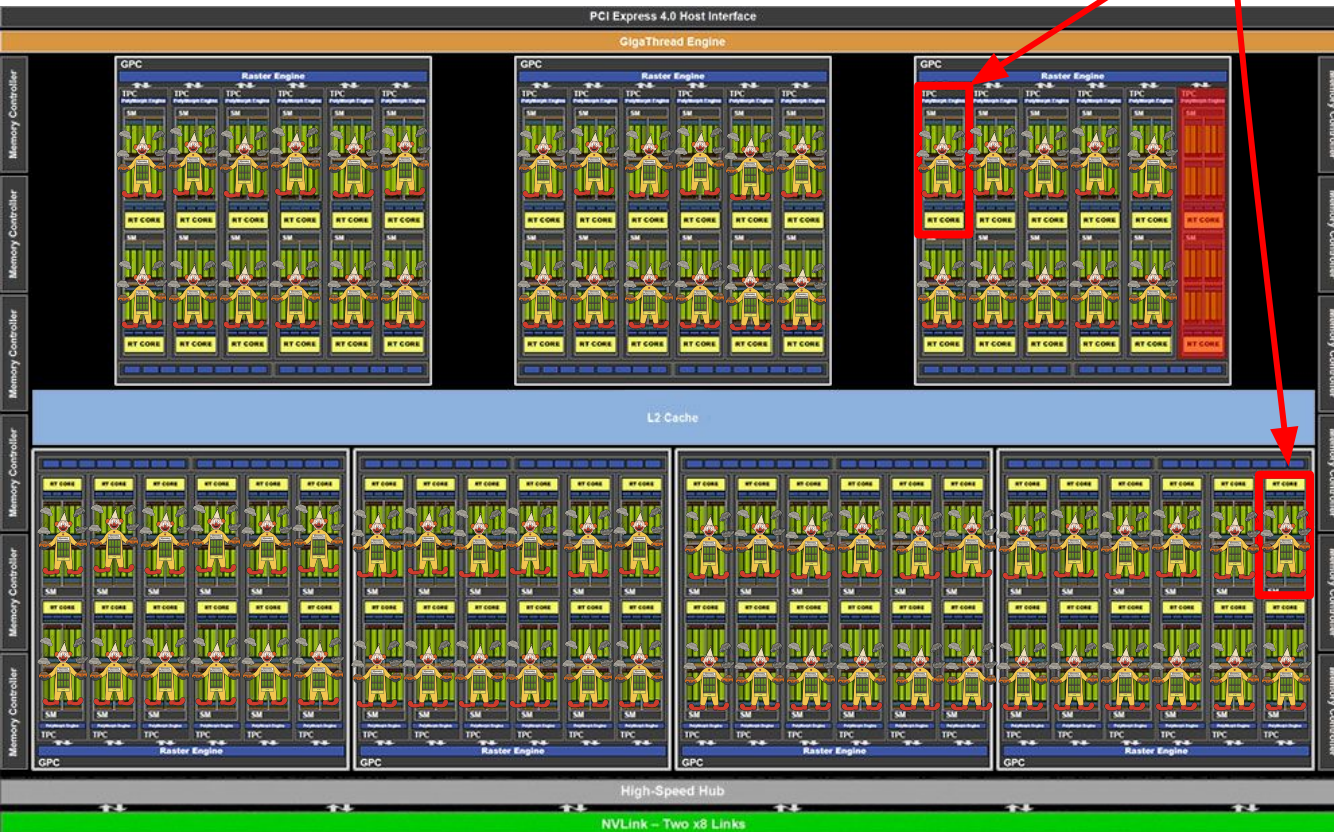
warp

32 слабых медленных
виртуальных потоков

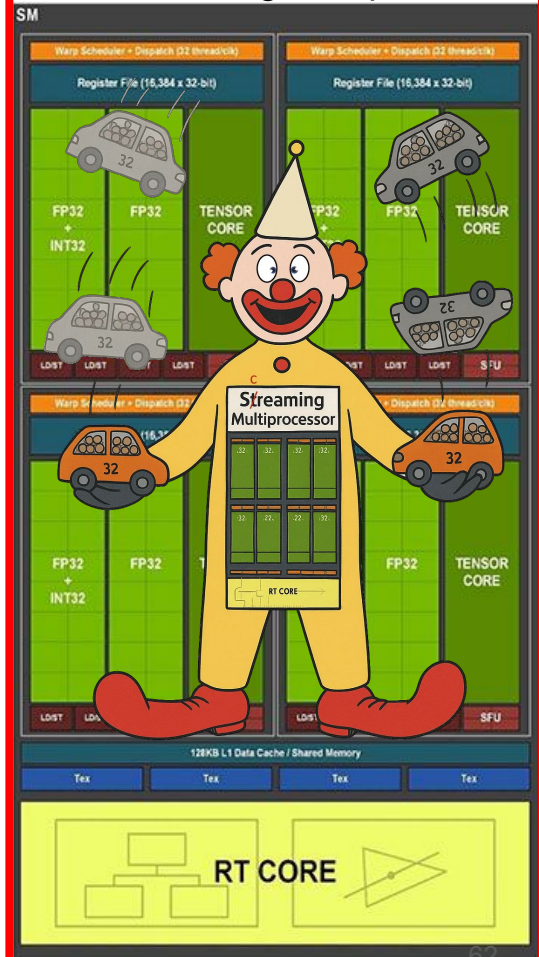
SM - Streaming Multiprocessor



RTX 3090: 10496 CUDA cores = 82 SM · 4 warps · 32 ALUs



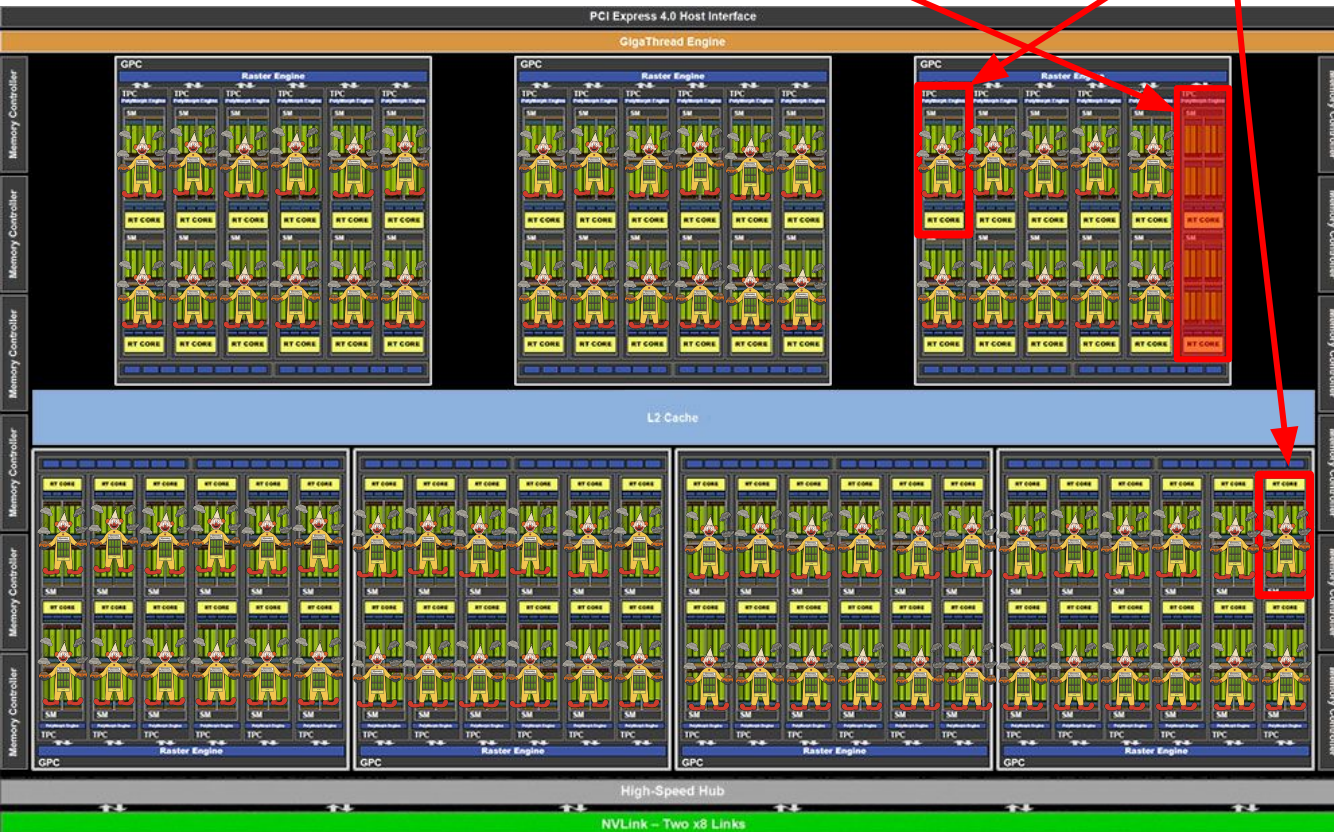
SM - Streaming Multiprocessor



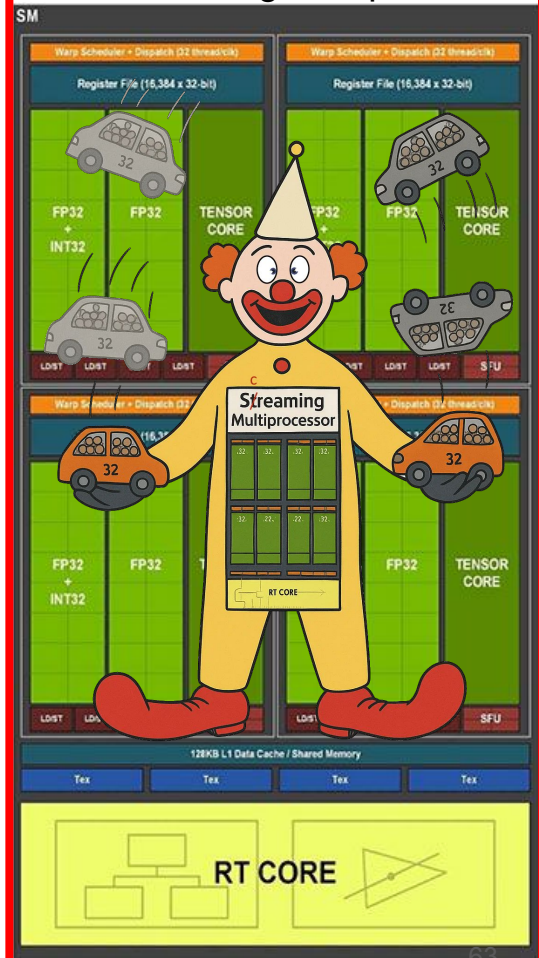
<https://www.techpowerup.com/review/nvidia-geforce-ampere-architecture-board-design-gaming-tech-software/3.html>

RTX 3090: 10496 CUDA cores = 82 SM · 4 warps · 32 ALUs

Куда сбежали два клоуна?



SM - Streaming Multiprocessor



<https://www.techpowerup.com/review/nvidia-geforce-ampere-architecture-board-design-gaming-tech-software/3.html>

Архитектура VRAM (**coalesced** memory access pattern)



```
float value = data[index];
```

Архитектура VRAM (**coalesced** memory access pattern)

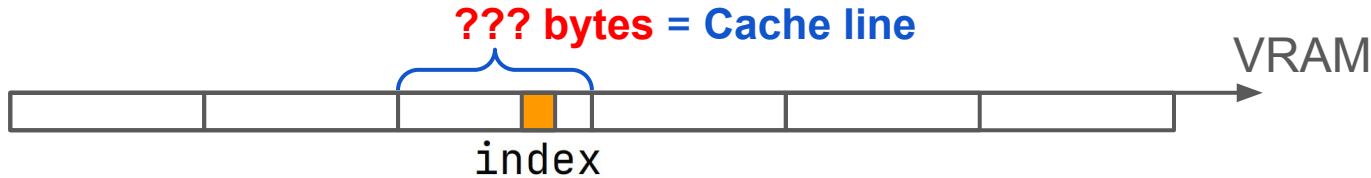
??? bytes = Cache line



```
float value = data[index];
```

Какого размера вы бы решили
сделать **Cache line**?

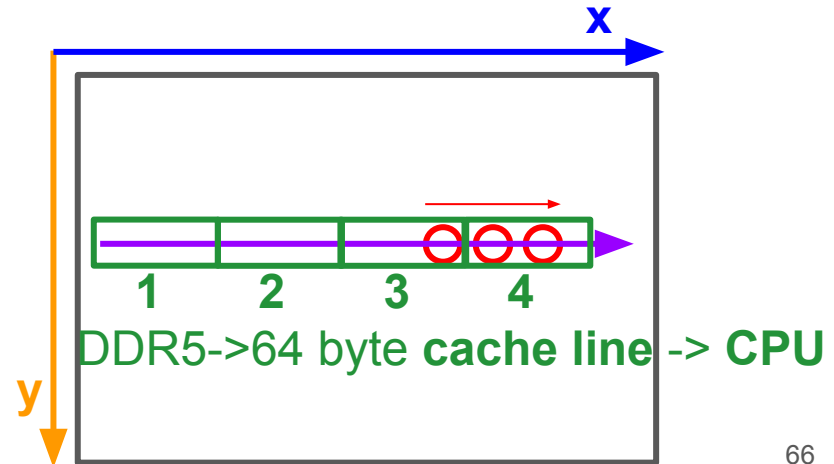
Архитектура VRAM (**coalesced** memory access pattern)



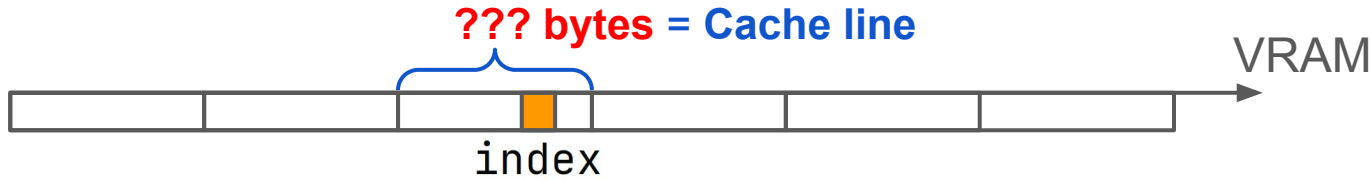
```
float value = data[index];
```

Какого размера вы бы решили сделать **Cache line**?

Напоминание про CPU:



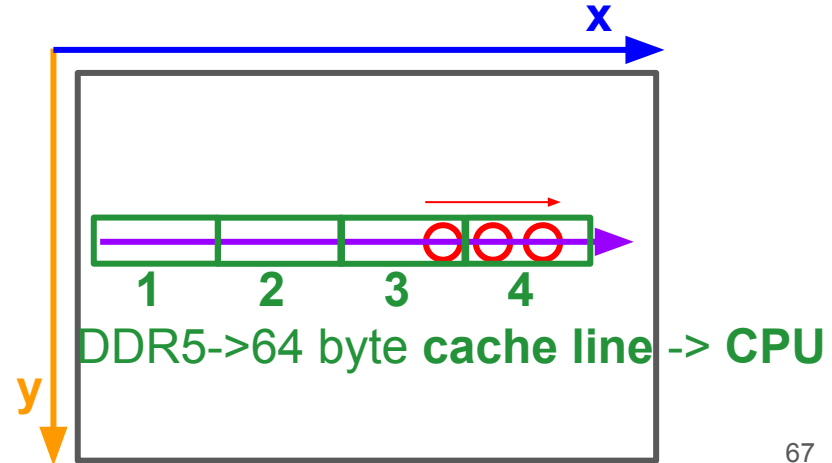
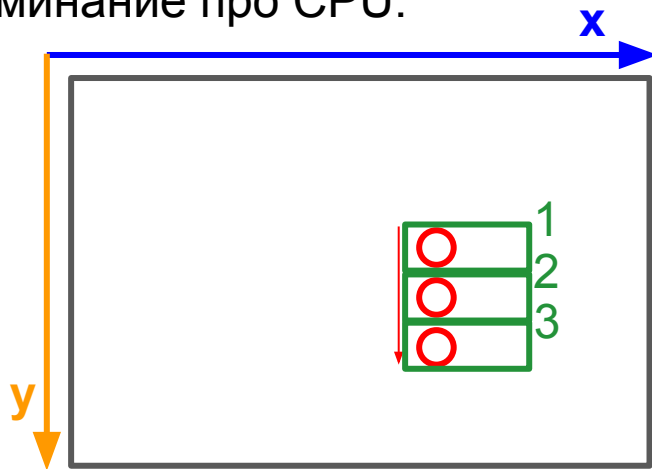
Архитектура VRAM (**coalesced** memory access pattern)



```
float value = data[index];
```

Какого размера вы бы решили сделать **Cache line**?

Напоминание про CPU:



Архитектура VRAM (**coalesced** memory access pattern)

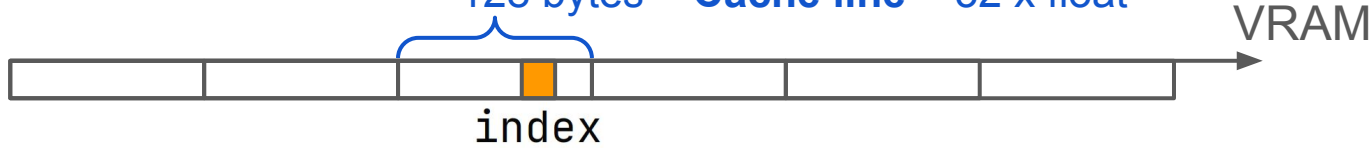
128 bytes = **Cache line** = 32 x float



```
float value = data[index];
```

Архитектура VRAM (**coalesced** memory access pattern)

128 bytes = **Cache line** = 32 x float



```
float value = data[index];
```



Архитектура VRAM (**coalesced** memory access pattern)

128 bytes = **Cache line** = 32 x float



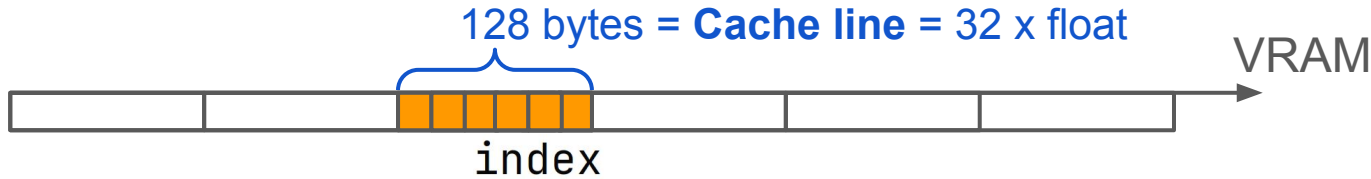
float value = data[index]; index | cache lines count

[1024+0; 1024+32) ???

Сколько **cache line**-ов чтобы покрыть заказ?
Сколько потребуется **транзакций** из VRAM?



Архитектура VRAM (**coalesced** memory access pattern)



<code>float value = data[index];</code>	index	cache lines count
---	-------	-------------------

	<code>[1024+0; 1024+32)</code>	1 транзакция (coalesced)
--	--------------------------------	--------------------------------------



Архитектура VRAM (**coalesced** memory access pattern)

128 bytes = **Cache line** = 32 x float

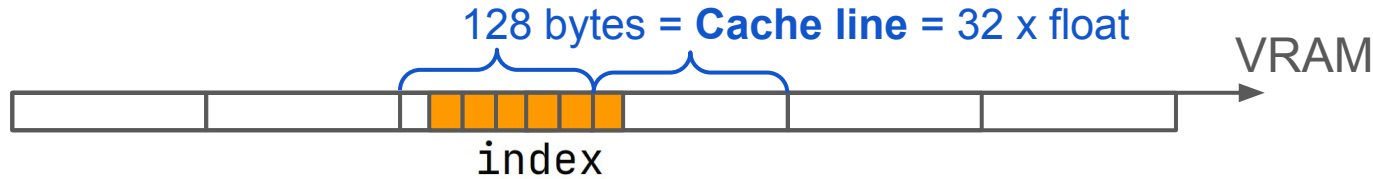


`float value = data[index];` index | cache lines count

[1024+0; 1024+32)	1 транзакция (coalesced)
[1024+1; 1024+33)	???

Сколько **cache line**-ов чтобы покрыть заказ?
Сколько потребуется **транзакций** из VRAM?

Архитектура VRAM (**coalesced** memory access pattern)



`float value = data[index];` index | cache lines count

[1024+0; 1024+32)

1 транзакция
(**coalesced**)

[1024+1; 1024+33)

2 транзакции
(**coalesced**)

Насколько просела достигнутая
полезная пропускная способность VRAM?

Архитектура VRAM (**coalesced** memory access pattern)

128 bytes = **Cache line** = 32 x float



float value = data[index]; index | cache lines count

[1024+0; 1024+32)

1 транзакция
(**coalesced**)

[1024+1; 1024+33)

2 транзакции
(**coalesced**)

{1024 + i*32}

???

Сколько **cache line**-ов чтобы покрыть заказ?
Сколько потребуется **транзакций** из VRAM?



Архитектура VRAM (**coalesced** memory access pattern)

128 bytes = **Cache line** = 32 x float



float value = data[index]; index | cache lines count

[1024+0; 1024+32)

1 транзакция
(**coalesced**)

[1024+1; 1024+33)

2 транзакции
(**coalesced**)

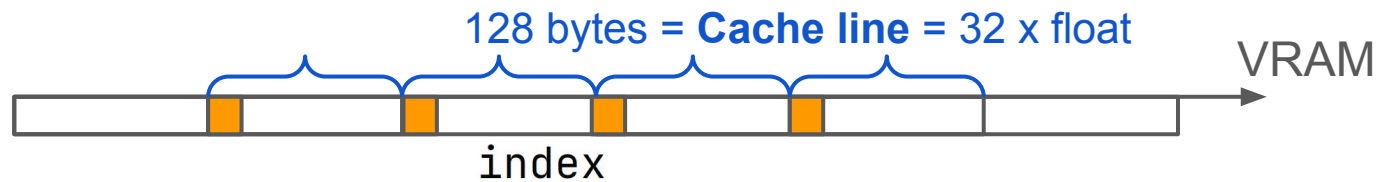
{1024 + i*32}

???

Сколько **cache line**-ов чтобы покрыть заказ?
Сколько потребуется **транзакций** из VRAM?



Архитектура VRAM (**coalesced** memory access pattern)



`float value = data[index];` index cache lines count

[1024+0; 1024+32)

1 транзакция
(**coalesced**)

[1024+1; 1024+33)

2 транзакции
(**coalesced**)

{1024 + i*32}

32 транзакции
(**uncoalesced**)



Насколько просела достигнутая
полезная пропускная способность VRAM?⁷⁶

Архитектура VRAM (**coalesced** memory access pattern)

128 bytes = **Cache line** = 32 x float



Почему именно 128 байт?

`float value = data[index];` index cache lines count

[1024+0; 1024+32)

1 транзакция
(**coalesced**)

[1024+1; 1024+33)

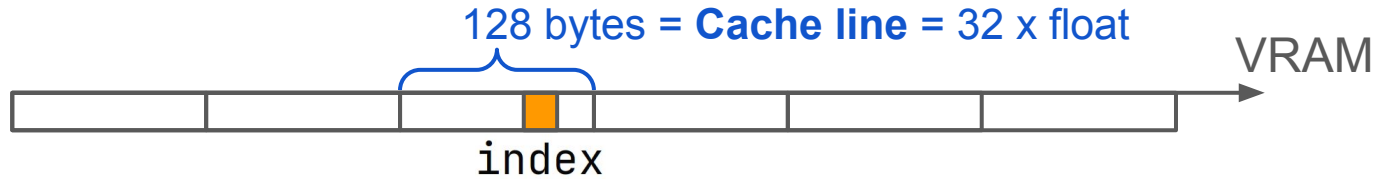
2 транзакции
(**coalesced**)

{1024 + i*32}

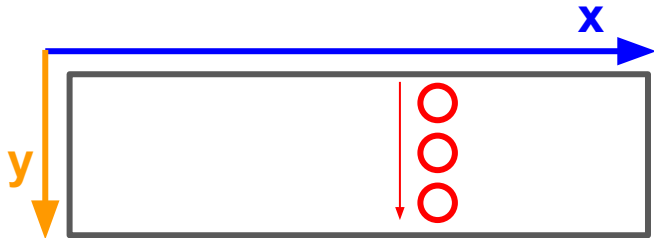
32 транзакции
(**uncoalesced**)



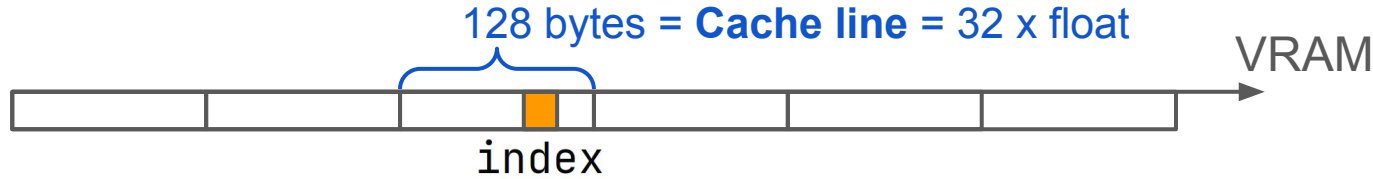
Архитектура VRAM (**coalesced** memory access pattern)



```
sum = 0;
for (int x = 0; x < width; ++x) {
    for (int y = 0; y < height; ++y) {
        sum += image[y * width + x];
    }
}
```

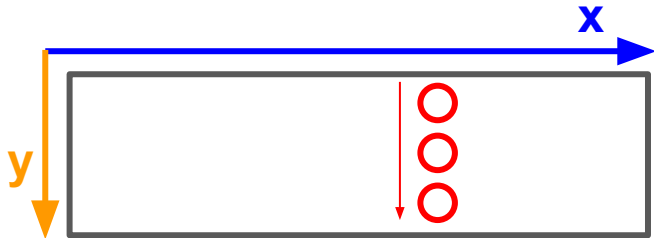


Архитектура VRAM (**coalesced** memory access pattern)

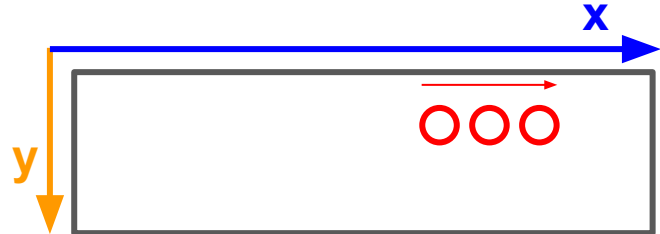


Но это однопоточный код, как в него добавить массовый параллелизм?

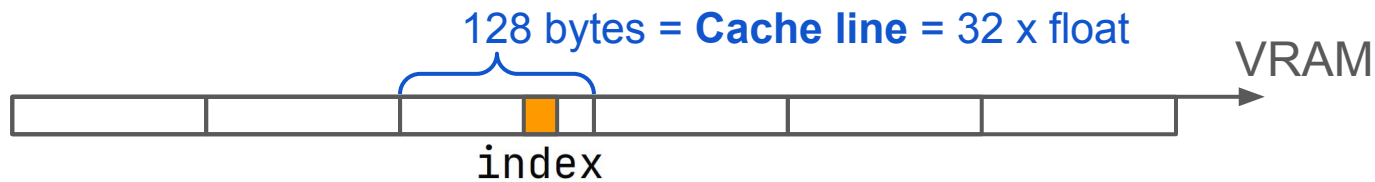
```
sum = 0;
for (int x = 0; x < width; ++x) {
    for (int y = 0; y < height; ++y) {
        sum += image[y * width + x];
    }
}
```



```
sum = 0;
for (int y = 0; y < height; ++y) {
    for (int x = 0; x < width; ++x) {
        sum += image[y * width + x];
    }
}
```



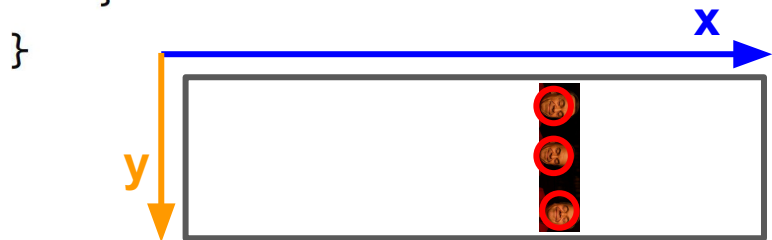
Архитектура VRAM (**coalesced** memory access pattern)



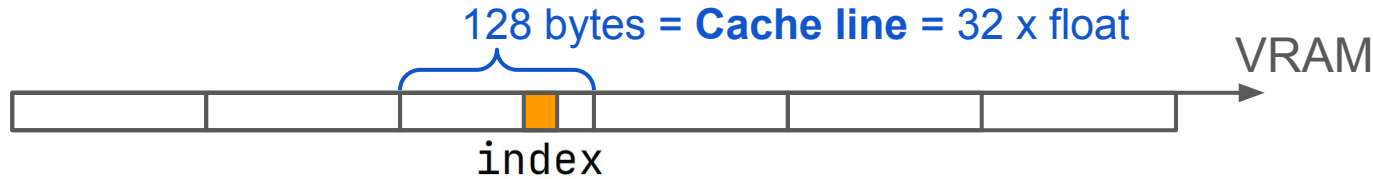
Но это однопоточный код, как в него добавить массовый параллелизм?

```
sum = 0;  
for (int x = 0; x < width; ++x) {  
    y =  
    sum += image[y * width + x];  
}
```

Номер потока в *warp*



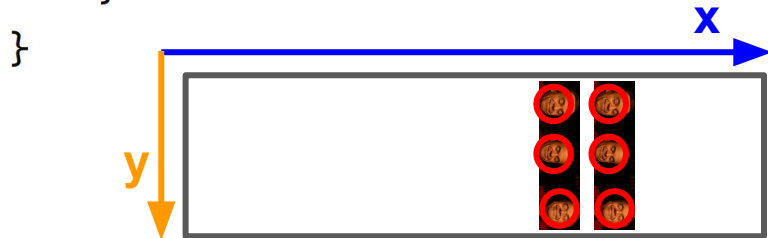
Архитектура VRAM (**coalesced** memory access pattern)



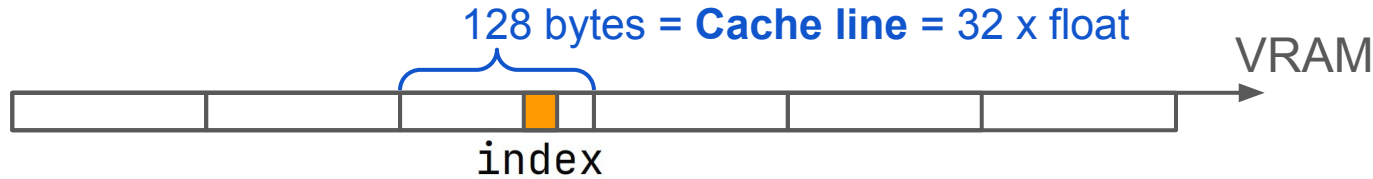
Но это однопоточный код, как в него добавить массовый параллелизм?

```
sum = 0;  
for (int x = 0; x < width; ++x) {  
    y =  
    sum += image[y * width + x];  
}
```

← Номер потока в *warp*



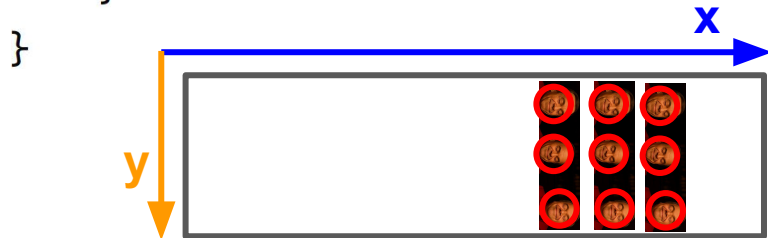
Архитектура VRAM (**coalesced** memory access pattern)



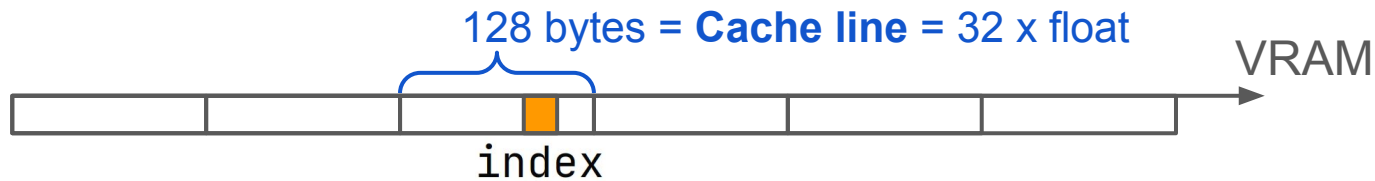
Но это однопоточный код, как в него добавить массовый параллелизм?

```
sum = 0;  
for (int x = 0; x < width; ++x) {  
    y =  
    sum += image[y * width + x];  
}
```

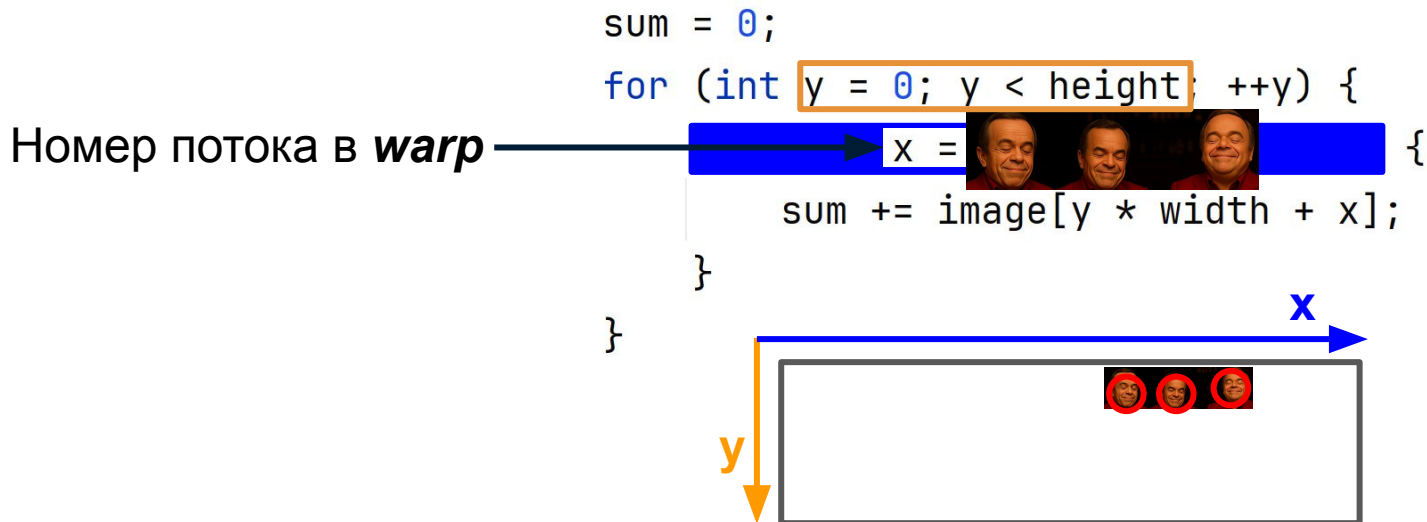
Номер потока в *warp*



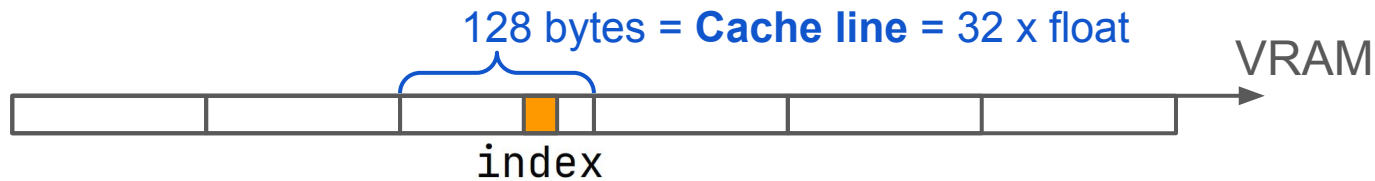
Архитектура VRAM (**coalesced** memory access pattern)



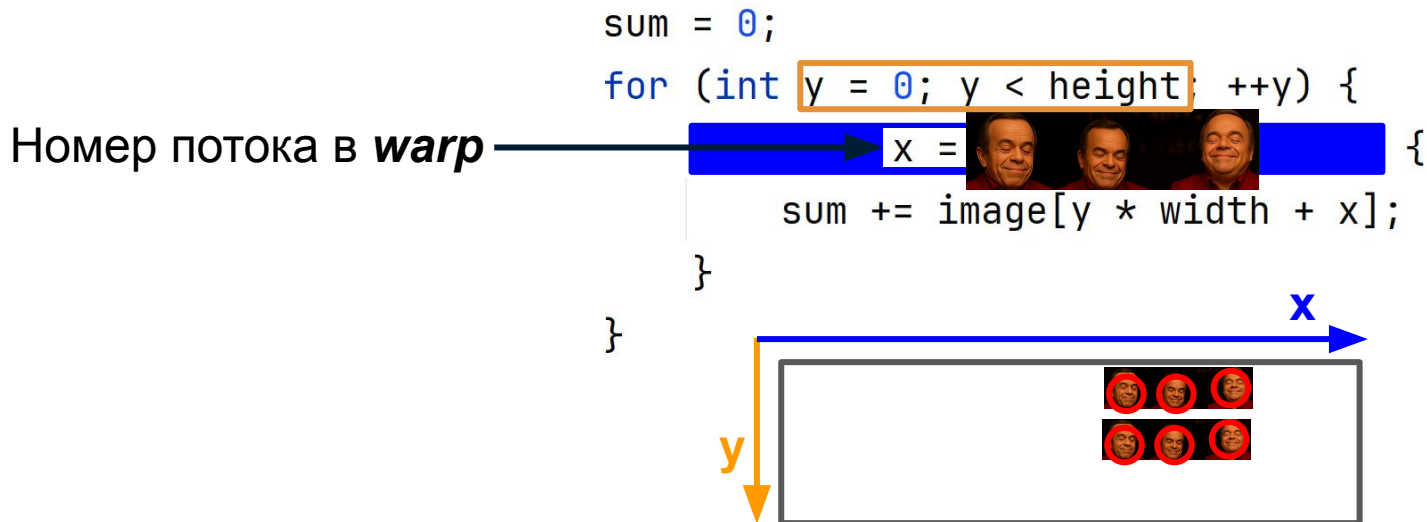
Но это однопоточный код, как в него добавить массовый параллелизм?



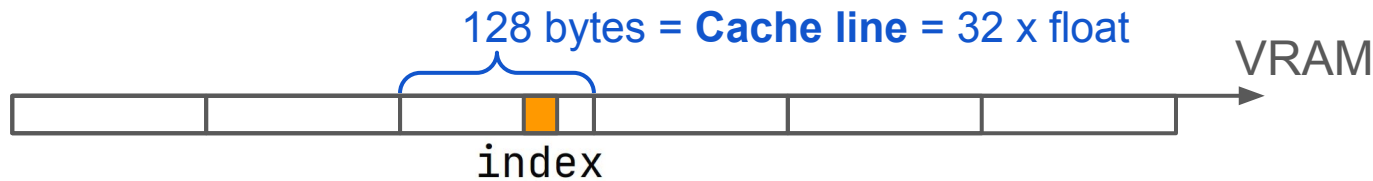
Архитектура VRAM (**coalesced** memory access pattern)



Но это однопоточный код, как в него добавить массовый параллелизм?

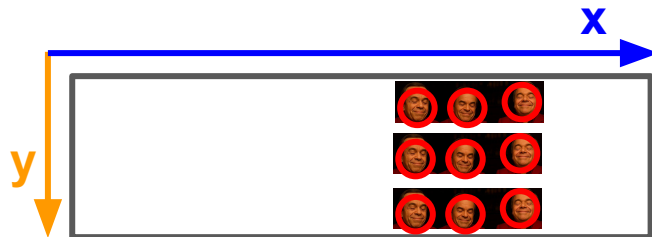


Архитектура VRAM (**coalesced** memory access pattern)

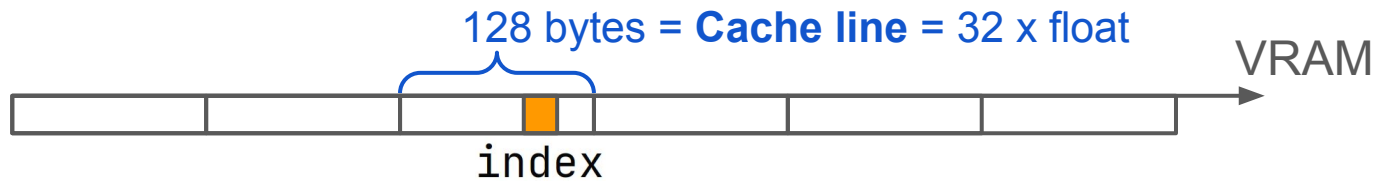


Но это однопоточный код, как в него добавить массовый параллелизм?

```
sum = 0;  
for (int y = 0; y < height; ++y) {  
    Номер потока в warp → x =  {  
        sum += image[y * width + x];  
    }  
}
```

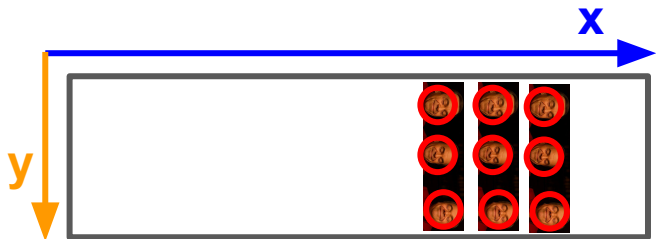


Архитектура VRAM (**coalesced** memory access pattern)

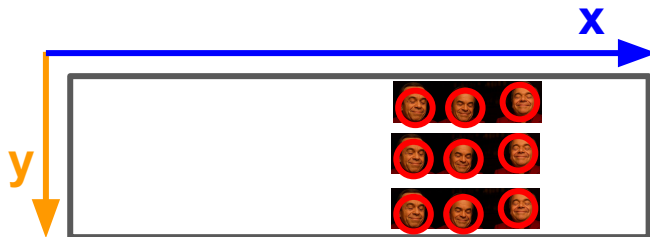


Какой код выбрали бы вы?

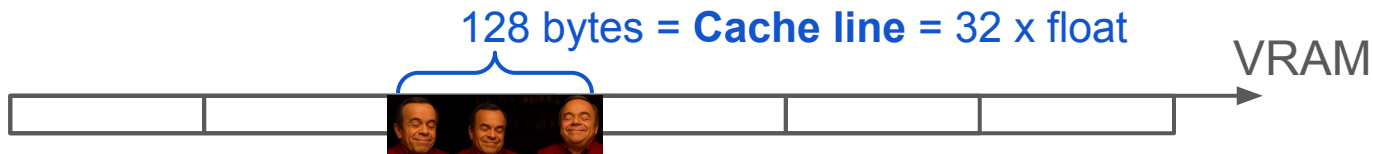
```
sum = 0;
for (int x = 0; x < width; ++x) {
    y =
    sum += image[y * width + x];
}
```



```
sum = 0;
for (int y = 0; y < height; ++y) {
    x =
    sum += image[y * width + x];
}
```



Архитектура VRAM (**coalesced** memory access pattern)

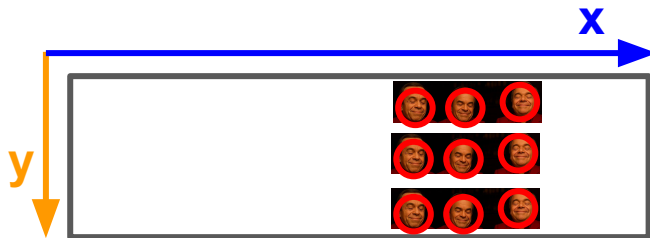


Какой код выбрали бы вы?

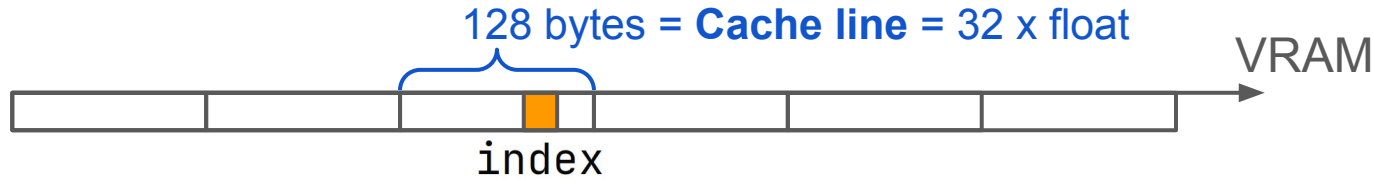
Они загрузят cache line
и распределят в рамках *warp*-а
байты, произойдет **broadcast**

```
sum = 0;  
for (int y = 0; y < height; ++y) {  
    x =  {  
        sum += image[y * width + x];  
    }  
}
```

The code snippet shows a loop over the y-axis of an image. Inside the loop, the x-coordinate is assigned a value from a set of three small images of a man's face. This represents a broadcast operation where the same data is shared across multiple threads in a warp.

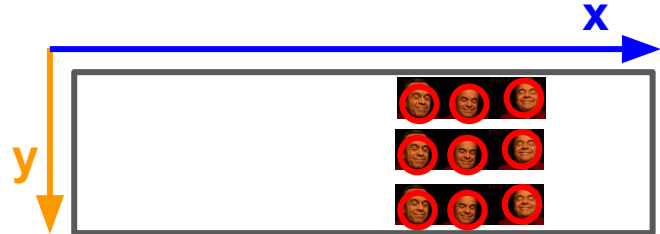


Архитектура VRAM (**coalesced** memory access pattern)

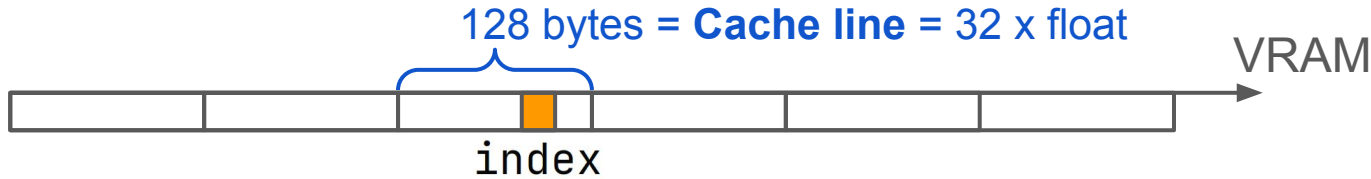


SIMT (thread)

```
sum = 0;  
for (int y = 0; y < height; ++y) {  
    x =  {  
        sum += image[y * width + x];  
    }  
}
```

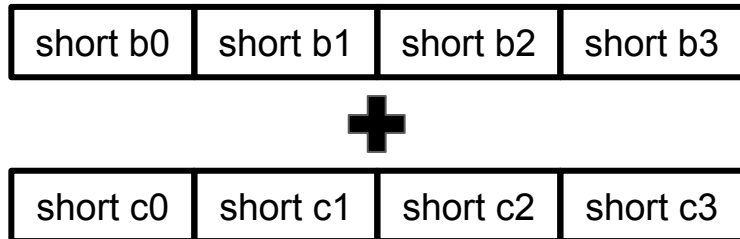


Архитектура VRAM (coalesced memory access pattern)



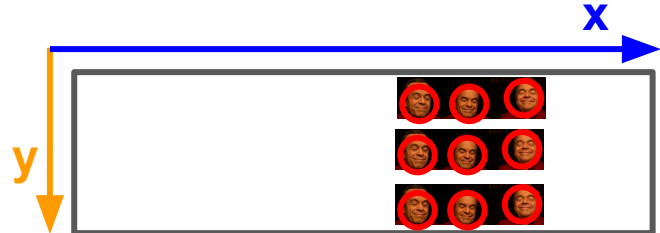
SIMD (data)

```
__m256i maskAll = _mm256_setzero_si256();
__m256i iters = _mm256_setzero_si256();
__m256 xs = _mm256_set1_ps(0.0f);
__m256 ys = _mm256_set1_ps(0.0f);
for (iteration = 0; iteration < MAX_ITERAT
    __m256 xsn = _mm256_add_ps(_mm256_sub
    __m256 ysn = _mm256_add_ps(_mm256_mul
    xs = _mm256_add_ps(_mm256_andnot_ps((
    ys = _mm256_add_ps(_mm256_andnot_ps((
maskAll = (__m256i) _mm256_or_ps(_mm256_cmp_ps(_mm256_add_ps(_mm256_mul_ps(xs, xs), _mm256_mul_ps(ys,
iters = _mm256_add_epi16(iters, _mm256_andnot_si256(maskAll, _mm256_set1_epi16(1)));
int mask = _mm256_movemask_epi8(maskAll);
if (mask == (int) 0xffffffff) {
    break;
}
```



SIMT (thread)

```
sum = 0;
for (int y = 0; y < height; ++y) {
    X = {
        sum += image[y * width + x];
    }
}
```

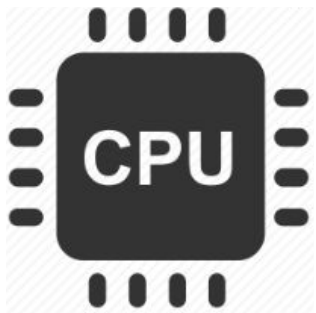


Глава 3: Общая картина ЭВМ-архитектуры

CPU - RAM - PCI-E - VRAM - GPU

Архитектура

$100 \cdot 10^9$ FLOPS



40 GB/s
low latency

RAM - DDR5

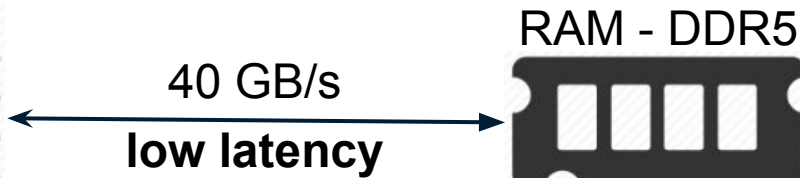
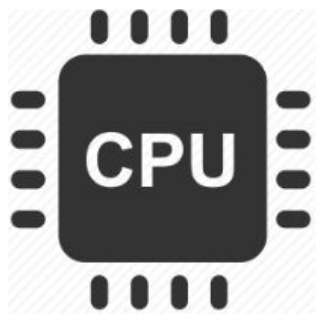


$100 \cdot 10^{12}$ FLOPS



Архитектура

$100 \cdot 10^9$ FLOPS



RAM - DDR5



$100 \cdot 10^{12}$ FLOPS



1000 GB/s
(x25 раз больше)

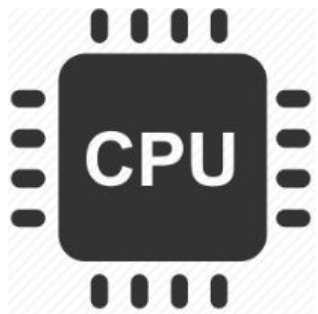


VRAM - GDDR6 или HBM (high bandwidth)

Архитектура

Где лучше исполнять OS?
(например реагировать на клики пользователя)
Какие есть метрики качества?

$100 \cdot 10^9$ FLOPS



40 GB/s
low latency

RAM - DDR5



$100 \cdot 10^{12}$ FLOPS



1000 GB/s
(x25 раз больше)

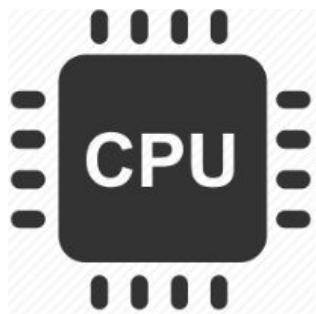


VRAM - GDDR6 или HBM (high bandwidth)

Архитектура

Где быстрее сложить два массива чисел?

$100 \cdot 10^9$ FLOPS



40 GB/s

RAM - DDR5



$100 \cdot 10^{12}$ FLOPS



1000 GB/s

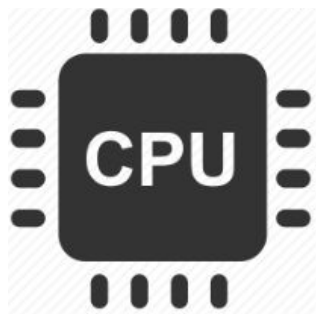


VRAM - GDDR6 или HBM (**high bandwidth**)

Архитектура

Где быстрее сложить два массива чисел?
И во что мы упираемся - в память или в ALUs?
(ALUs = Arithmetic Logic Units)

$100 \cdot 10^9$ FLOPS



40 GB/s

RAM - DDR5



$100 \cdot 10^{12}$ FLOPS



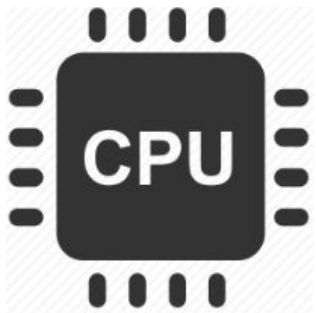
1000 GB/s



VRAM - GDDR6 или HBM (**high bandwidth**)

Архитектура

$100 \cdot 10^9$ FLOPS



40 GB/s

RAM - DDR5



$100 \cdot 10^{12}$ FLOPS



1000 GB/s



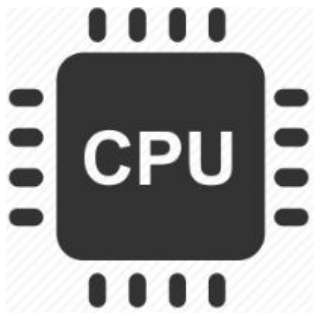
VRAM - GDDR6

PCI-E 5.0 x16
16 GB/s

Где быстрее сложить два массива чисел?
И во что мы упираемся - в память или в ALUs?
А что если данные находятся в **RAM**?

Архитектура

$100 \cdot 10^9$ FLOPS



40 GB/s



RAM - DDR5

$100 \cdot 10^{12}$ FLOPS



1000 GB/s

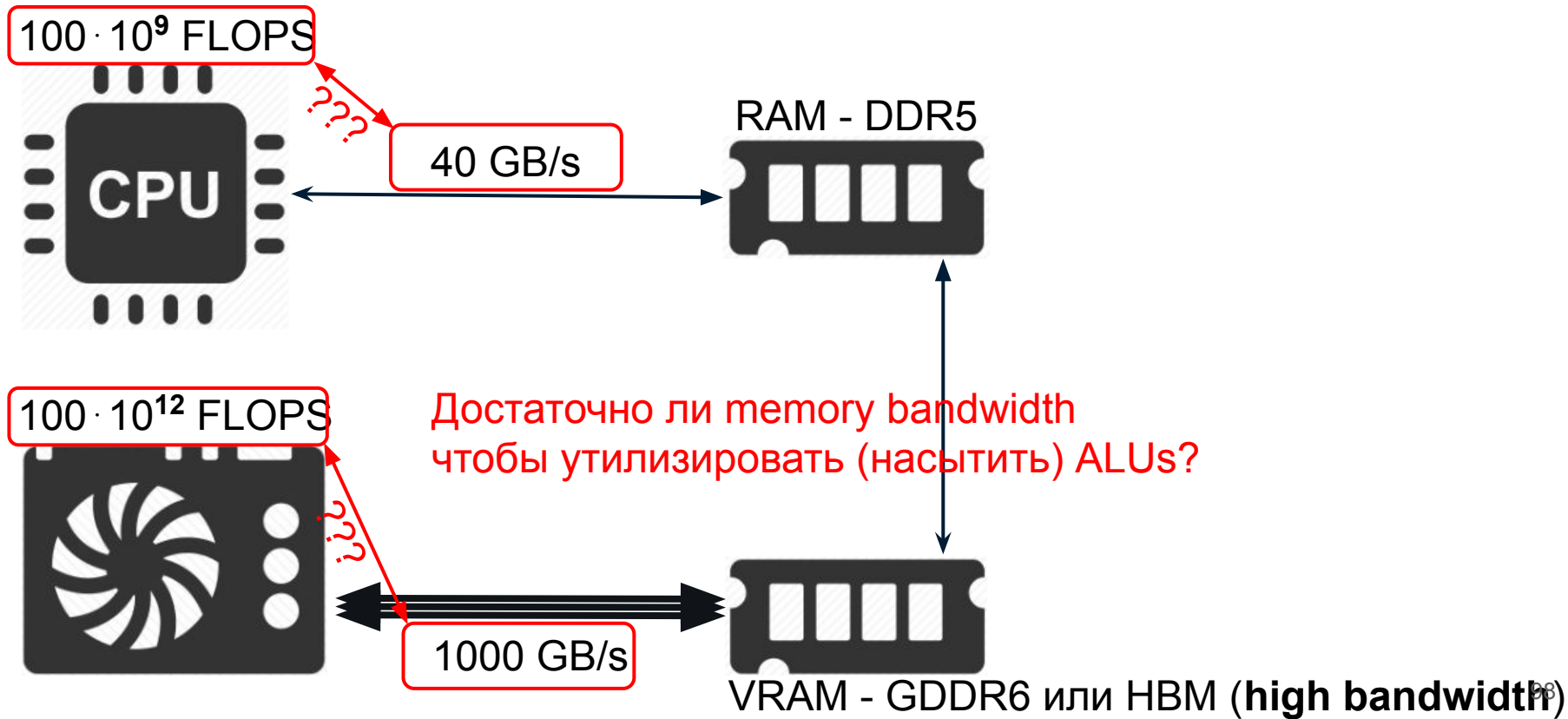


VRAM - GDDR6

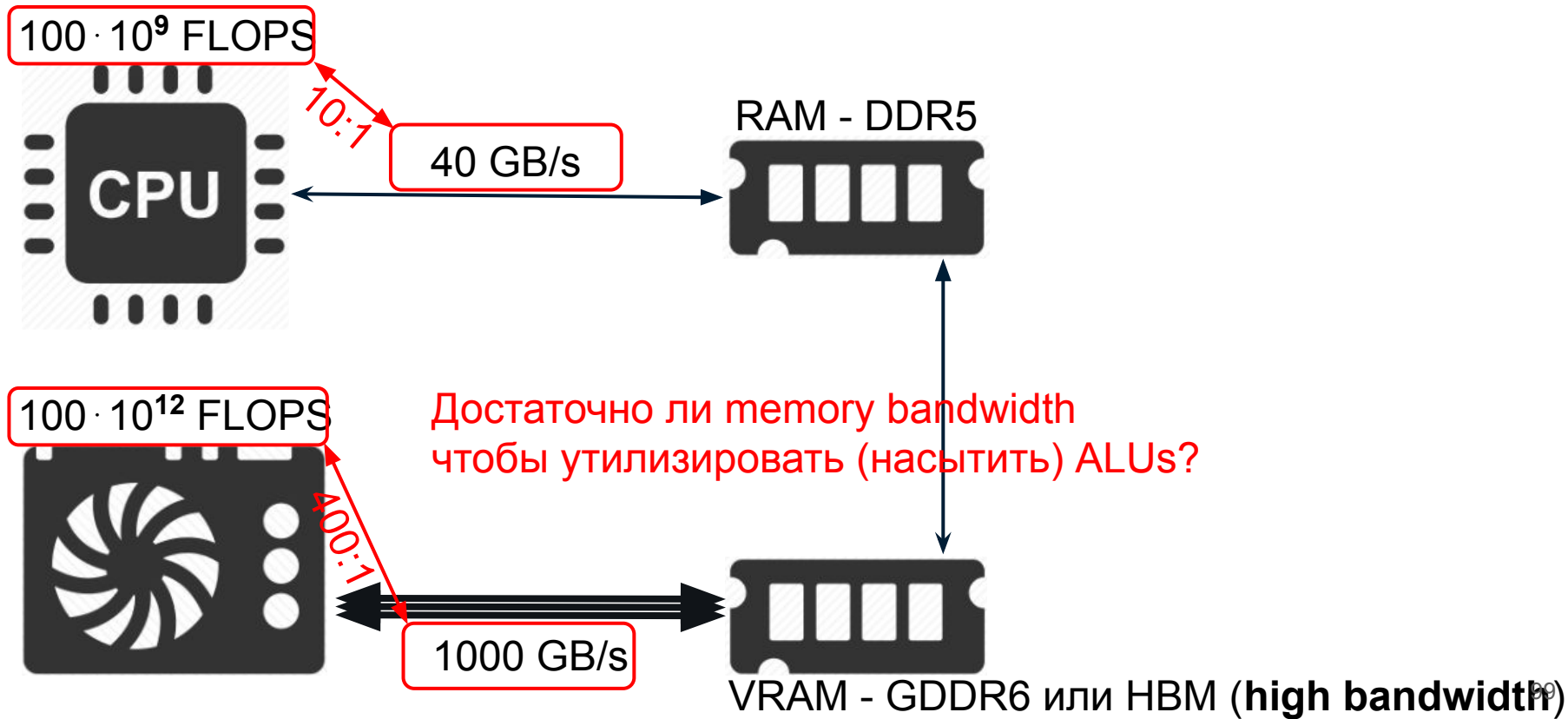
PCI-E 5.0 x16
16 GB/s

Где быстрее сложить два массива чисел?
И во что мы упираемся - в память или в ALUs?
А что если данные находятся в **RAM**?

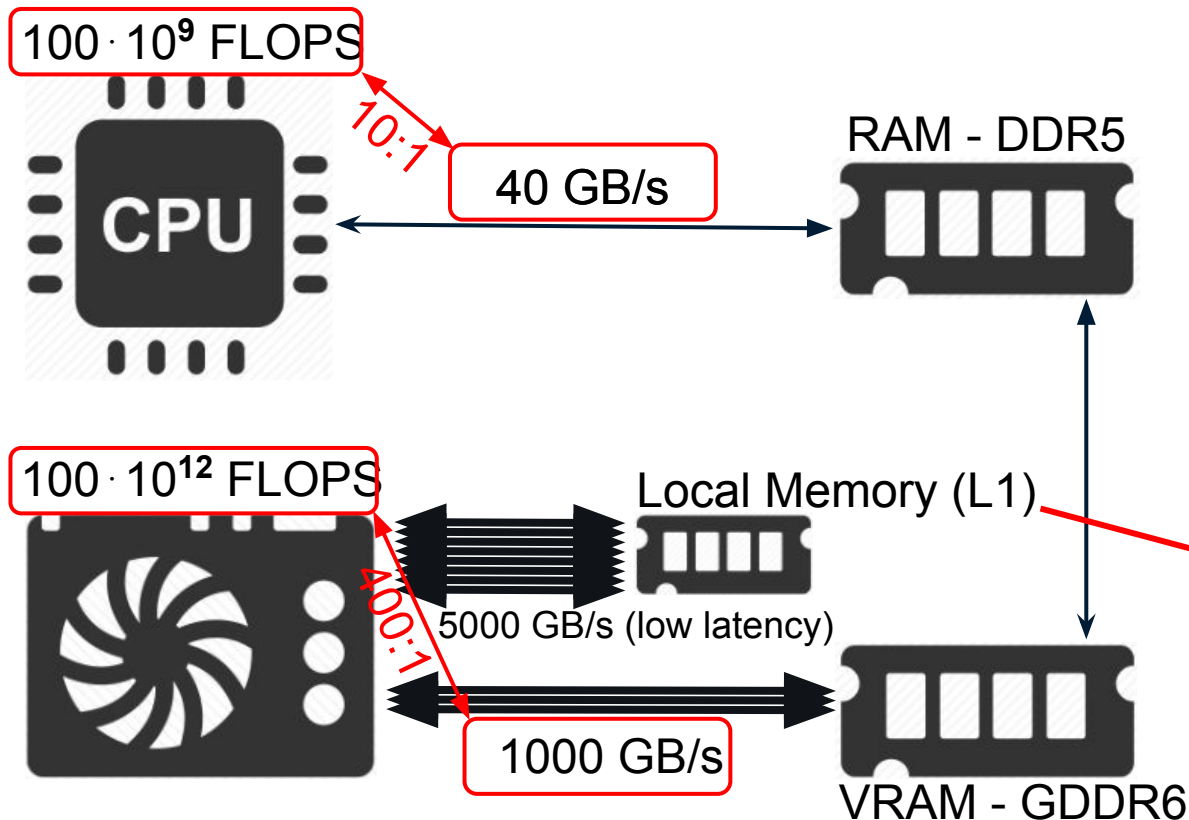
Архитектура



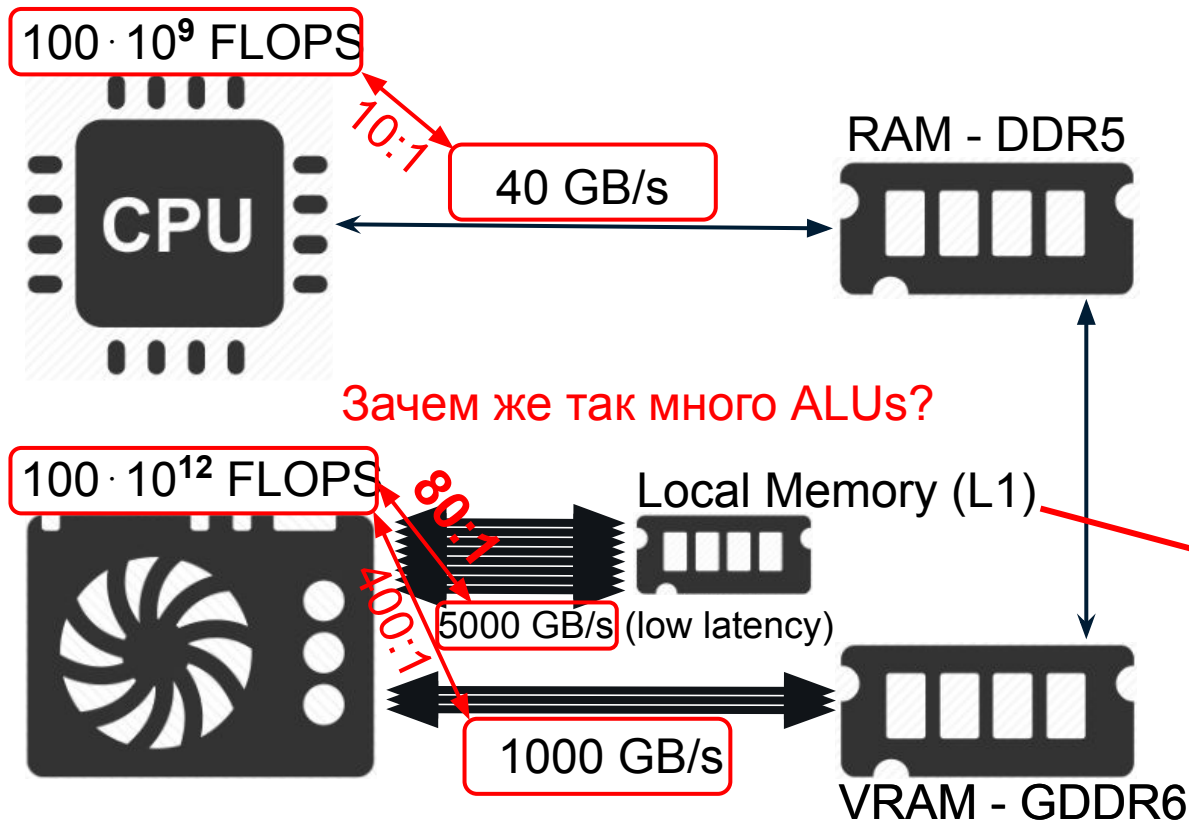
Архитектура



Архитектура

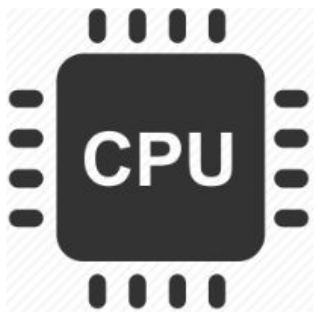


Архитектура



Архитектура

$100 \cdot 10^9$ FLOPS



40 GB/s

RAM - DDR5



Можно ли ее использовать
так же как VRAM?

$100 \cdot 10^{12}$ FLOPS



5000 GB/s (low latency)

Local Memory (L1)



1000 GB/s

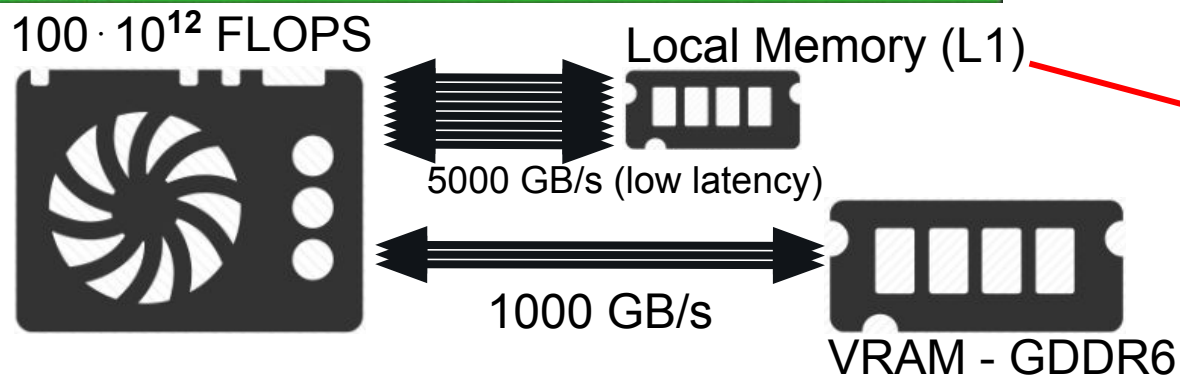


VRAM - GDDR6



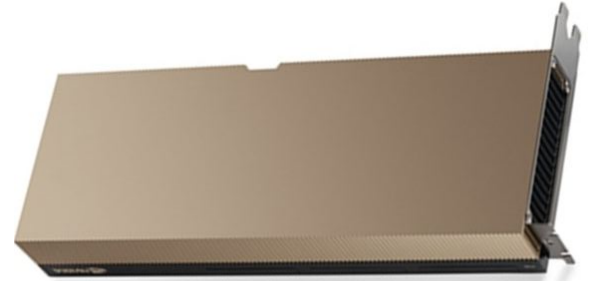


SM - Streaming Multiprocessor



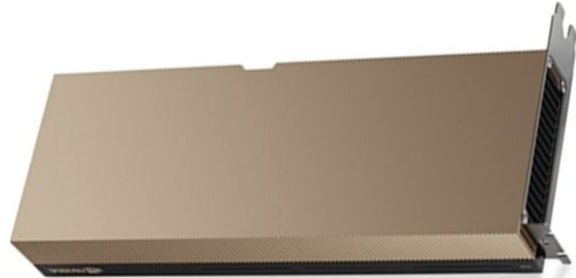
Архитектура GPU

- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)



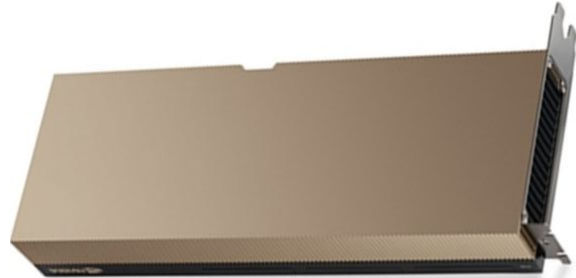
Архитектура GPU

- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра

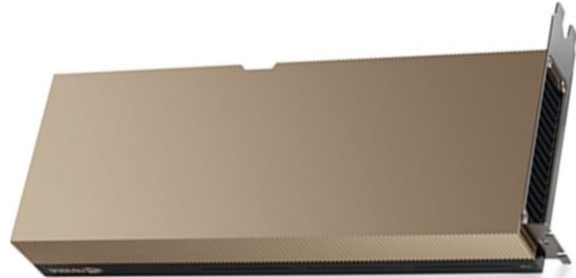


Архитектура GPU

- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)

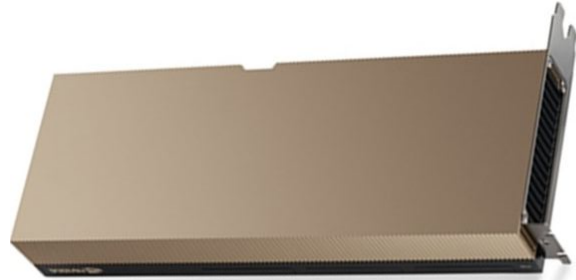


Архитектура GPU



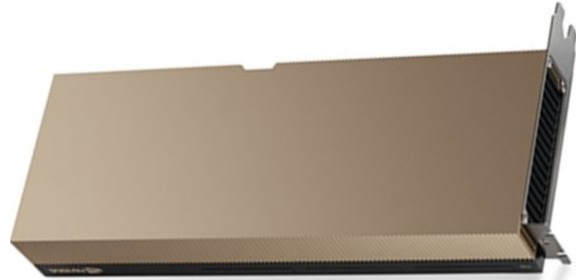
- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)

Архитектура GPU



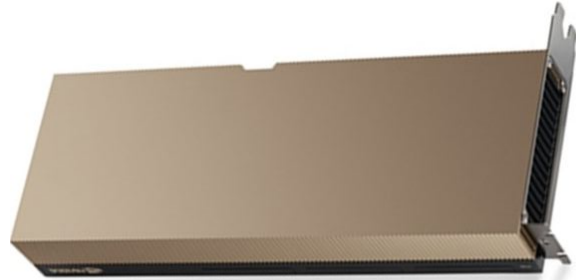
- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:

Архитектура GPU



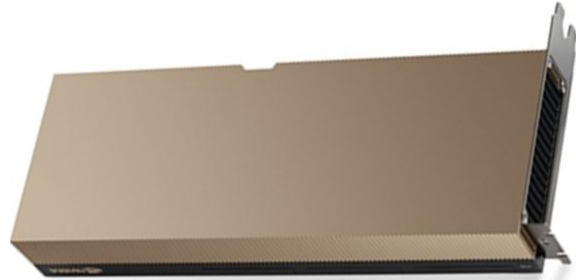
- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:
 - но большая задержка (**latency**)

Архитектура GPU



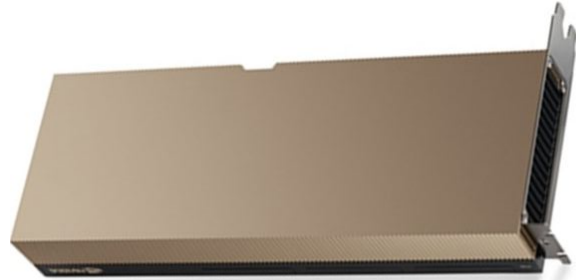
- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:
 - но большая задержка (**latency**)
 - скрывается за счет **сверх-SMT**/Hyper-Threading (при высокой **occupancy**)

Архитектура GPU



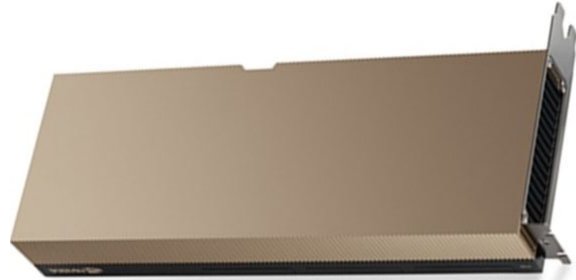
- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:
 - но большая задержка (**latency**)
 - скрывается за счет **сверх-SMT**/Hyper-Threading (при высокой **occupancy**)
 - транзакции разбиты на 128-байтные **cache line**

Архитектура GPU



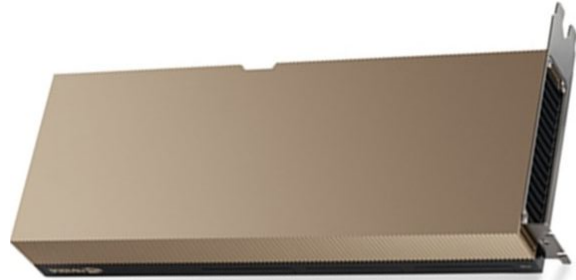
- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:
 - но большая задержка (**latency**)
 - скрывается за счет **сверх-SMT**/Hyper-Threading (при высокой **occupancy**)
 - транзакции разбиты на 128-байтные **cache line**
 - но при **un-coalesced** memory access pattern - малая проп. способность

Архитектура GPU



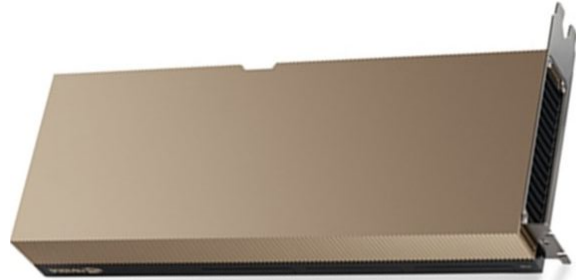
- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:
 - но большая задержка (**latency**)
 - скрывается за счет **сверх-SMT**/Hyper-Threading (при высокой **occupancy**)
 - транзакции разбиты на 128-байтные **cache line**
 - но при **un-coalesced** memory access pattern - малая проп. способность
- 3) Локальная память (**local/shared memory**):

Архитектура GPU



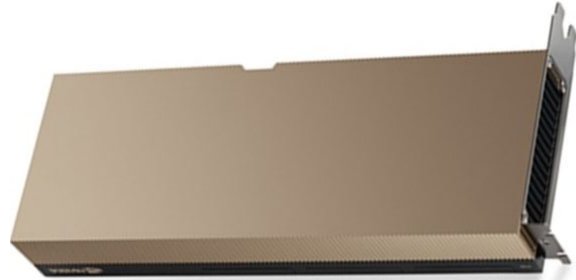
- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:
 - но большая задержка (**latency**)
 - скрывается за счет **сверх-SMT**/Hyper-Threading (при высокой **occupancy**)
 - транзакции разбиты на 128-байтные **cache line**
 - но при **un-coalesced** memory access pattern - малая проп. способность
- 3) Локальная память (**local/shared memory**):
 - **титаническая** пропускная способность + низкая задержка

Архитектура GPU



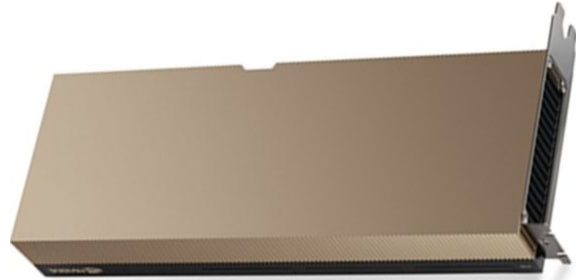
- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:
 - но большая задержка (**latency**)
 - скрывается за счет **сверх-SMT**/Hyper-Threading (при высокой **occupancy**)
 - транзакции разбиты на 128-байтные **cache line**
 - но при **un-coalesced** memory access pattern - малая проп. способность
- 3) Локальная память (**local/shared memory**):
 - **титаническая** пропускная способность + низкая задержка
 - нет проблемы с coalesced-паттерном

Архитектура GPU



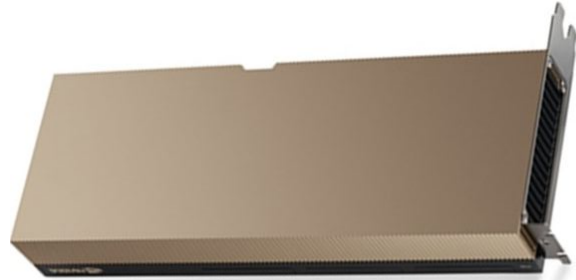
- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:
 - но большая задержка (**latency**)
 - скрывается за счет **сверх-SMT**/Hyper-Threading (при высокой **occupancy**)
 - транзакции разбиты на 128-байтные **cache line**
 - но при **un-coalesced** memory access pattern - малая проп. способность
- 3) Локальная память (**local/shared memory**):
 - **титаническая** пропускная способность + низкая задержка
 - нет проблемы с coalesced-паттерном
 - **но?**

Архитектура GPU

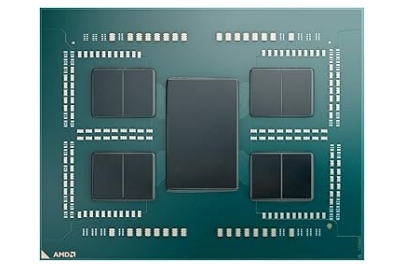


- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:
 - но большая задержка (**latency**)
 - скрывается за счет **сверх-SMT**/Hyper-Threading (при высокой **occupancy**)
 - транзакции разбиты на 128-байтные **cache line**
 - но при **un-coalesced** memory access pattern - малая проп. способность
- 3) Локальная память (**local/shared memory**):
 - **титаническая** пропускная способность + низкая задержка
 - нет проблемы с coalesced-паттерном
 - но **небольшая** + **локальная** для каждого **warp WorkGroup=Block**

Архитектура GPU



- 1) Десятки тысяч ядер (много **FLOPs**):
 - сгруппированы по 32 в **warp**-ы (по 64 в **wavefront**-ы у AMD)
 - но слабые ядра
 - но у warp-а общий **instruction pointer** (опасность **code divergence**)
 - активен **warp** который не ждет данных из **VRAM** (при высокой **occupancy**)
- 2) Огромная пропускная способность **VRAM**:
 - но большая задержка (**latency**)
 - скрывается за счет **сверх-SMT/Hyper-Threading** (при высокой **occupancy**)
 - транзакции разбиты на 128-байтные **cache line**
 - но при **un-coalesced** memory access pattern - малая проп. способность
- 3) Локальная память (**local/shared memory**):
 - **титаническая** пропускная способность + низкая задержка
 - нет проблемы с coalesced-паттерном Позже мы обсудим **bank conflicts**
 - но **небольшая** + **локальная** для каждого **warp WorkGroup=Block**
И в этом секрет успеха! Рецепт к **масштабируемости!**



AMD Ryzen Threadripper 7980X Core-to-Core Latency



64 ядра = 8 x Чипетов по 8 ядер

AMD Ryzen Threadripper 7980X Core-to-Core Latency

	0-100		100-200		200-300		300-400		400-500		500-600		600-700		700-800		800-900		900-1000		1000-1100		1100-1200		1200-1300		1300-1400		1400-1500		1500-1600		1600-1700		1700-1800		1800-1900		1900-2000		2000-2100		2100-2200		2200-2300		2300-2400		2400-2500		2500-2600		2600-2700		2700-2800		2800-2900		2900-3000		3000-3100		3100-3200		3200-3300		3300-3400		3400-3500		3500-3600		3600-3700		3700-3800		3800-3900		3900-4000		4000-4100		4100-4200		4200-4300		4300-4400		4400-4500		4500-4600		4600-4700		4700-4800		4800-4900		4900-5000		5000-5100		5100-5200		5200-5300		5300-5400		5400-5500		5500-5600		5600-5700		5700-5800		5800-5900		5900-6000		6000-6100		6100-6200		6200-6300		6300-6400		6400-6500		6500-6600		6600-6700		6700-6800		6800-6900		6900-7000		7000-7100		7100-7200		7200-7300		7300-7400		7400-7500		7500-7600		7600-7700		7700-7800		7800-7900		7900-8000		8000-8100		8100-8200		8200-8300		8300-8400		8400-8500		8500-8600		8600-8700		8700-8800		8800-8900		8900-9000		9000-9100		9100-9200		9200-9300		9300-9400		9400-9500		9500-9600		9600-9700		9700-9800		9800-9900		9900-10000		10000-10100		10100-10200		10200-10300		10300-10400		10400-10500		10500-10600		10600-10700		10700-10800		10800-10900		10900-11000		11000-11100		11100-11200		11200-11300		11300-11400		11400-11500		11500-11600		11600-11700		11700-11800		11800-11900		11900-12000		12000-12100		12100-12200		12200-12300		12300-12400		12400-12500		12500-12600		12600-12700		12700-12800		12800-12900		12900-13000		13000-13100		13100-13200		13200-13300		13300-13400		13400-13500		13500-13600		13600-13700		13700-13800		13800-13900		13900-14000		14000-14100		14100-14200		14200-14300		14300-14400		14400-14500		14500-14600		14600-14700		14700-14800		14800-14900		14900-15000		15000-15100		15100-15200		15200-15300		15300-15400		15400-15500		15500-15600		15600-15700		15700-15800		15800-15900		15900-16000		16000-16100		16100-16200		16200-16300		16300-16400		16400-16500		16500-16600		16600-16700		16700-16800		16800-16900		16900-17000		17000-17100		17100-17200		17200-17300		17300-17400		17400-17500		17500-17600		17600-17700		17700-17800		17800-17900		17900-18000		18000-18100		18100-18200		18200-18300		18300-18400		18400-18500		18500-18600		18600-18700		18700-18800		18800-18900		18900-19000		19000-19100		19100-19200		19200-19300		19300-19400		19400-19500		19500-19600		19600-19700		19700-19800		19800-19900		19900-20000		20000-20100		20100-20200		20200-20300		20300-20400		20400-20500		20500-20600		20600-20700		20700-20800		20800-20900		20900-21000		21000-21100		21100-21200		21200-21300		21300-21400		21400-21500		21500-21600		21600-21700		21700-21800		21800-21900		21900-22000		22000-22100		22100-22200		22200-22300		22300-22400		22400-22500		22500-22600		22600-22700		22700-22800		22800-22900		22900-23000		23000-23100		23100-23200		23200-23300		23300-23400		23400-23500		23500-23600		23600-23700		23700-23800		23800-23900		23900-24000		24000-24100		24100-24200		24200-24300		24300-24400		24400-24500		24500-24600		24600-24700		24700-24800		24800-24900		24900-25000		25000-25100		25100-25200		25200-25300		25300-25400		25400-25500		25500-25600		25600-25700		25700-25800		25800-25900		25900-26000		26000-26100		26100-26200		26200-26300		26300-26400		26400-26500		26500-26600		26600-26700		26700-26800		26800-26900		26900-27000		27000-27100		27100-27200		27200-27300		27300-27400		27400-27500		27500-27600		27600-27700		27700-27800		27800-27900		27900-28000		28000-28100		28100-28200		28200-28300		28300-28400		28400-28500		28500-28600		28600-28700		28700-28800		28800-28900		28900-29000		29000-29100		29100-29200		29200-29300		29300-29400		29400-29500		29500-29600		29600-29700		29700-29800		29800-29900		29900-30000		30000-30100		30100-30200		30200-30300		30300-30400		30400-30500		30500-30600		30600-30700		30700-30800		30800-30900		30900-31000		31000-31100		31100-31200		31200-31300		31300-31400		31400-31500		31500-31600		31600-31700		31700-31800		31800-31900		31900-32000		32000-32100		32100-32200		32200-32300		32300-32400		32400-32500		32500-32600		32600-32700		32700-32800		32800-32900		32900-33000		33000-33100		33100-33200		33200-33300		33300-33400		33400-33500		33500-33600		33600-33700		33700-33800		33800-33900		33900-34000		34000-34100		34100-34200		34200-34300		34300-34400		34400-34500		34500-34600		34600-34700		34700-34800		34800-34900		34900-35000		35000-35100		35100-35200		35200-35300		35300-35400		35400-35500		35500-35600		35600-35700		35700-35800		35800-35900		35900-36000		36000-36100		36100-36200		36200-36300		36300-36400		36400-36500		36500-36600		36600-36700		36700-36800		36800-36900		36900-37000		37000-37100		37100-37200		37200-37300		37300-37400		37400-37500		37500-37600		37600-37700		37700-37800		37800-37900		37900-38000		38000-38100		38100-38200		38200-38300		38300-38400		38400-38500		38500-38600		38600-38700		38700-38800		38800-38900		38900-39000		39000-39100		39100-39200		39200-39300		39300-39400		39400-39500		39500-39600		39600-39700		39700-39800		39800-39900		39900-40000		40000-40100		40100-40200		40200-40300		40300-40400		40400-40500		40500-40600		40600-40700		40700-40800		40800-40900		40900-41000		41000-41100		41100-41200		41200-41300		41300-41400		41400-41500		41500-41600		41600-41700		41700-41800		41800-41900		41900-42000		42000-42100		42100-42200		42200-42300		42300-42400		42400-42500		42500-42600		42600-42700		42700-42800		42800-42900		42900-43000		43000-43100		43100-43200		43200-43300		43300-43400		43400-43500		43500-43600		43600-43700		43700-43800		43800-43900		43900-44000		44000-44100		44100-44200		44200-44300		44300-44400		44400-44500		44500-44600		44600-44700		44700-44800		44800-44900		44900-45000		45000-45100		45100-45200		45200-45300		45300-45400		45400-45500		45500-45600		45600-45700		45700-45800		45800-45900		45900-46000		46000-46100		46100-46200		46200-46300		46300-46400		46400-46500		46500-46600		46600-46700		46700-46800		46800-46900		46900-47000		47000-47100		47100-47200		47200-47300		47300-47400		47400-47500		47500-47600		47600-47700		47700-47800		47800-47900		47900-48000		48000-48100		48100-48200		48200-48300		48300-48400		48400-48500		48500-48600		48600-48700		48700-48800		48800-48900		48900-49000		49000-49100		49100-49200		49200-49300		49300-49400		49400-49500		49500-49600		49600-49700		49700-49800		49800-49900		49900-50000		50000-50100		50100-50200		50200-50300		50300-50400		50400-50500		50500-50600		50600-50700		50700-50800		50800-50900		50900-51000		51000-51100		51100-51200		51200-51300		51300-51400		51400-51500		51500-51600		51600-51700		51700-51800		51800-51900		51900-52000		52000-52100		52100-52200		52200-52300		52300-52400		52400-52500		52500-52600		52600-52700		52700-52800		52800-52900		52900-53000		53000-53100		53100-53200		53200-53300		53300-53400		53400-53500		53500-53600		53600-53700		53700-53800		53800-53900		53900-54000		54000-54100		54100-54200		54200-54300		54300-54400		54400-54500		54500-54600		54600-54700		54700-54800		54800-54900		54900-55000		55000-55100		55100-55200		55200-55300		55300-55400		55400-55500		55500-55600		55600-55700		55700-55800		55800-55900		55900-56000		56000-56100		56100-56200		56200-56300		56300-56400		56400-56500		56500-56600		56600-56700		56700-56800		56800-56900		56900-57000		57000-57100		57100-57200		57200-57300		57300-57400		57400-57500		57500-57600		57600-57700		57700-57800		57800-57900		57900-58000		58000-58100		58100-58200		58200-58300		58300-58400		58400-58500		58500-58600		58600-58700		58700-58800		58800-58900		58900-59000		59000-59100		59100-59200		59200-59300		59300-59400		59400-59500		59500-59600		59600-59700		59700-59800		59800-59900		59900-60000		60000-60100		60100-60200		60200-60300		60300-60400		60400-60500		60500-60600		60600-60700		60700-60800		60800-60900		60900-61000		61000-61100		61100-61200		61200-61300		61300-61400		61400-61500		61500-61600		61600-61700		61700-61800		61800-61900		61900-62000		62000-62100		62100-62200		62200-62300		62300-62400		62400-62500		62500-62600		62600-62700		62700-62800		62800-62900		62900-63000		63000-63100		63100-63200		63200-63300		63300-63400		63400-63500		63500-63600		63600-63700		63700-63800		63800-63900		63900-64000		64000-64100		64100-64200		64200-64300		64300-64400		64400-64500		64500-64600		64600-64700		64700-64800		64800-64900		64900-65000		65000-65100		65100-65200		65200-65300		65300-65400		65400-65500		65500-65600		65600-65700		65700-65800		65800-65900		65900-66000		66000-66100		66100-66200		66200-66300	
--	-------	--	---------	--	---------	--	---------	--	---------	--	---------	--	---------	--	---------	--	---------	--	----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	-----------	--	------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--	-------------	--

64 ядра = 8 x Чиплетов (по 8 ядер в чиплете)

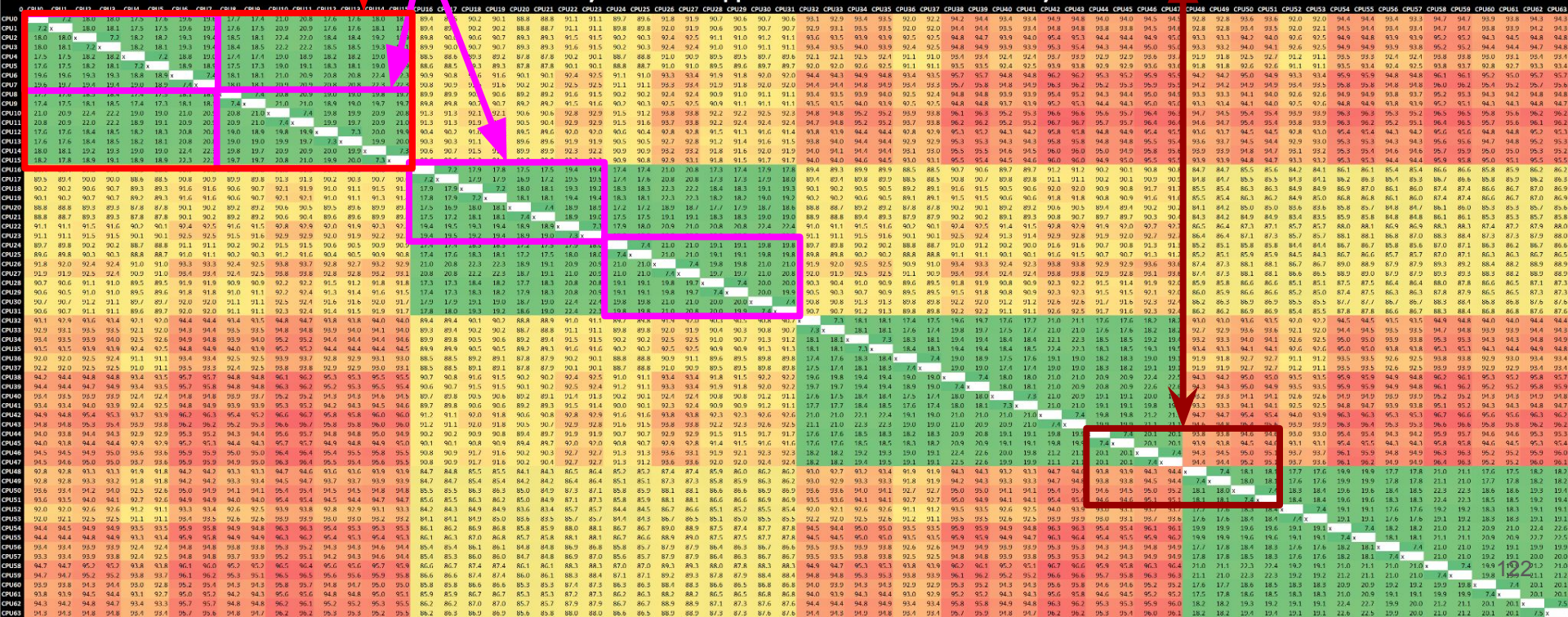
CPU19	CPU20	CPU21	CPU22	CPU23	CPU24	CPU25	CPU26	CPU27	CPU28	CPU29	CPU30	CPU31	CPU32	CPU33	CPU34	CPU35	CPU36	CPU37	CPU38	CPU39	CPU40	CPU41	CPU42
90.1	88.8	88.8	91.1	91.1	89.7	89.6	91.8	91.9	90.6	90.7	90.6	90.6	93.1	92.9	93.4	93.5	92.0	92.2	94.2	94.4	93.4	93.4	93.4
90.2	88.8	88.7	91.1	91.1	89.8	89.8	92.0	91.9	90.6	90.5	90.7	90.7	92.9	93.1	93.5	93.5	92.0	92.2	94.4	94.4	93.5	93.4	93.4

[illegible]

Но почему задержка маленькими группами по 16 ядер?

64 ядра = 8 x чипов по 8 ядер

AMD Ryzen Threadripper 7980X Core-to-Core Latency

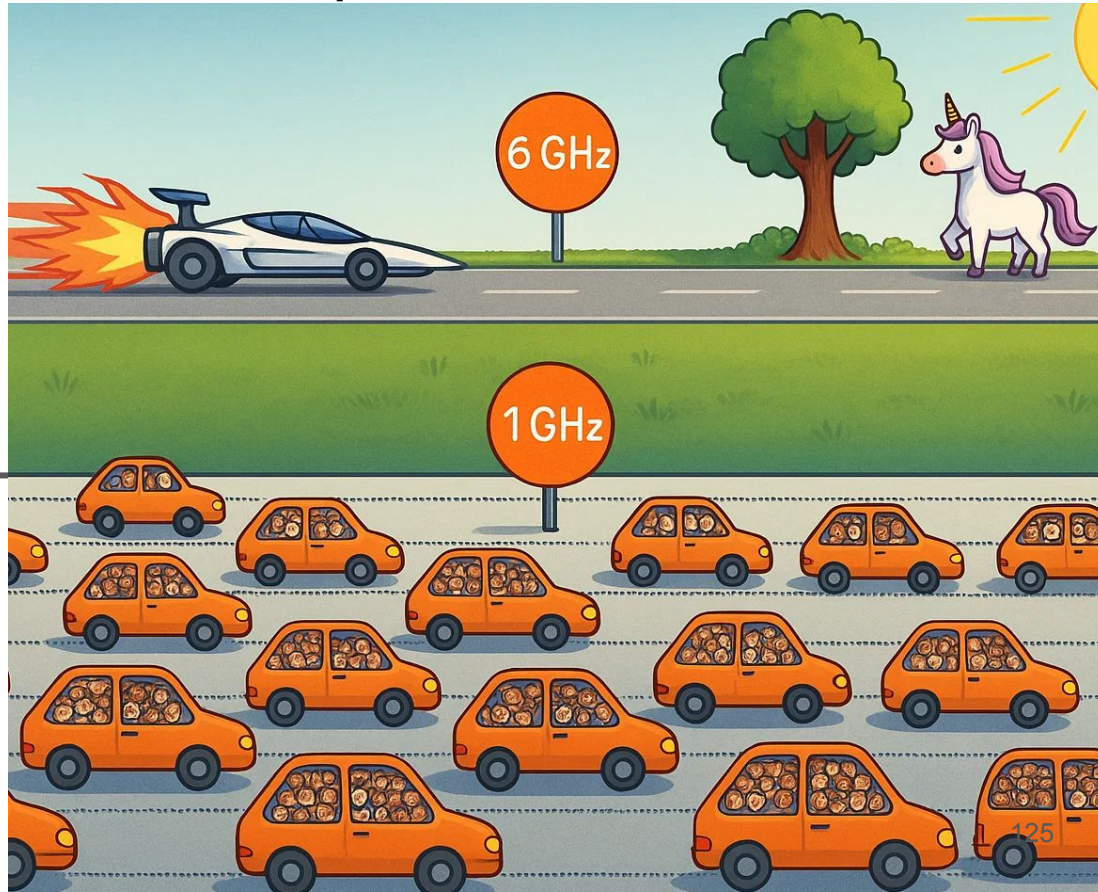
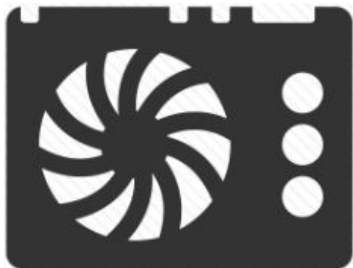
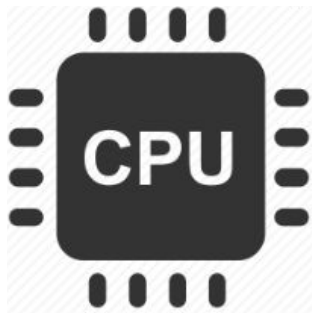


Перерыв!



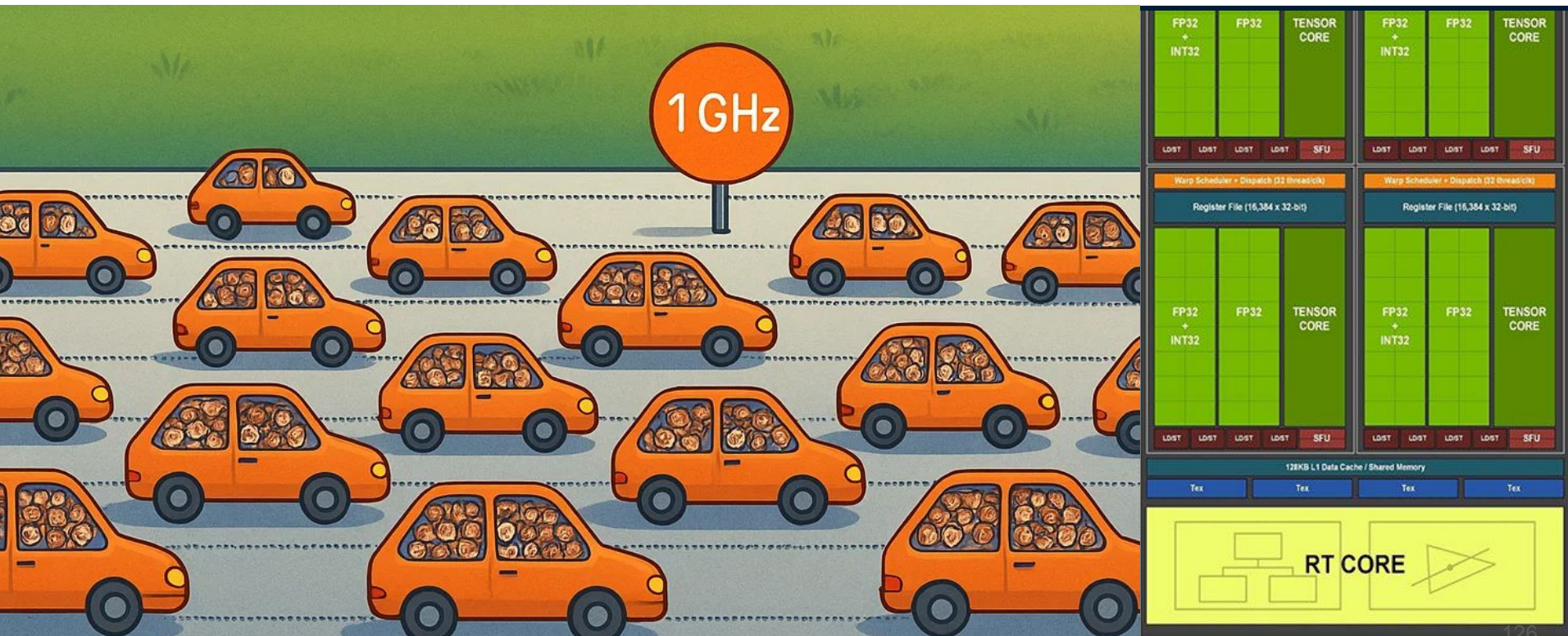
Глава 4: Модель вычислений массового параллелизма

Модель вычислений массового параллелизма



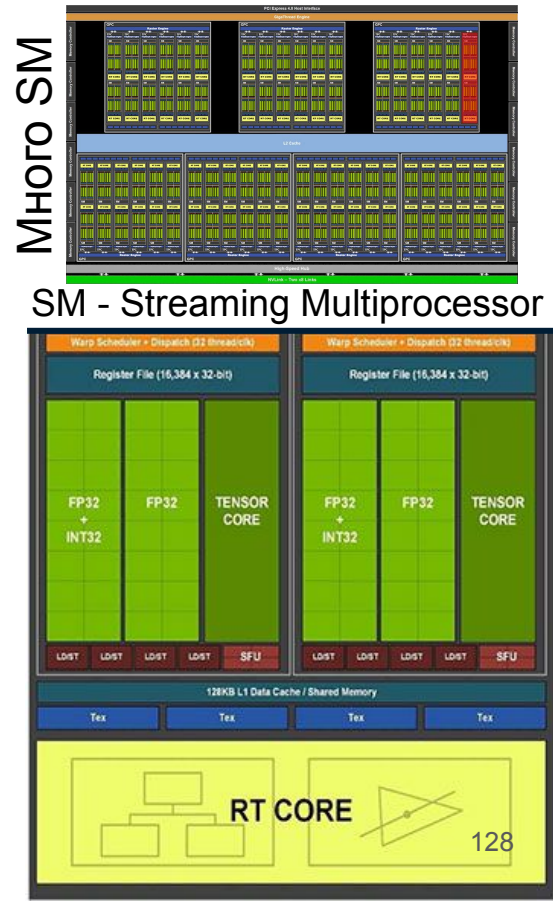
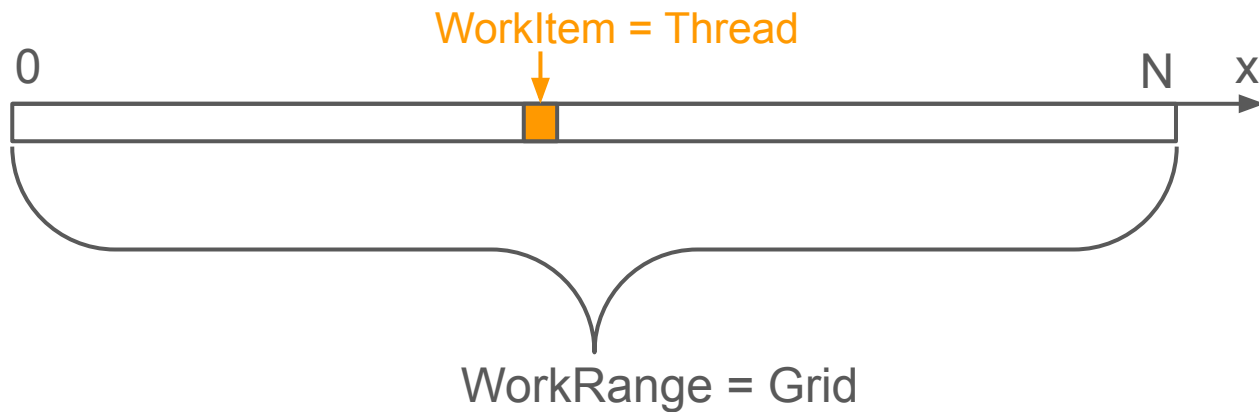
Модель вычислений массового параллелизма

SM - Streaming Multiprocessor

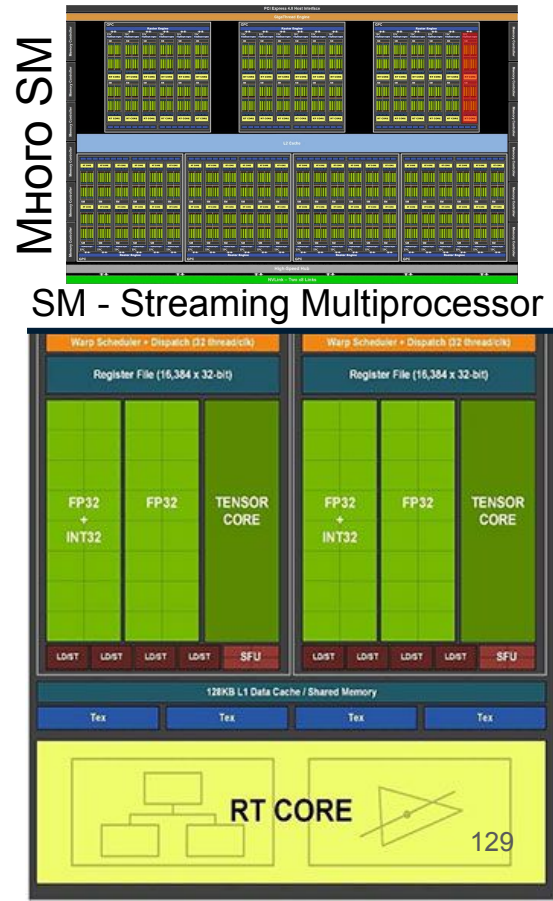
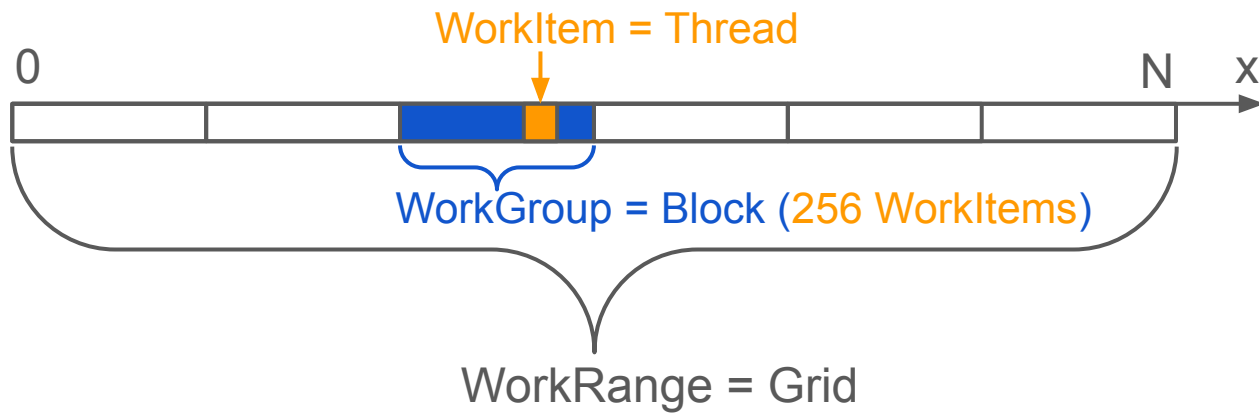


[illegible]

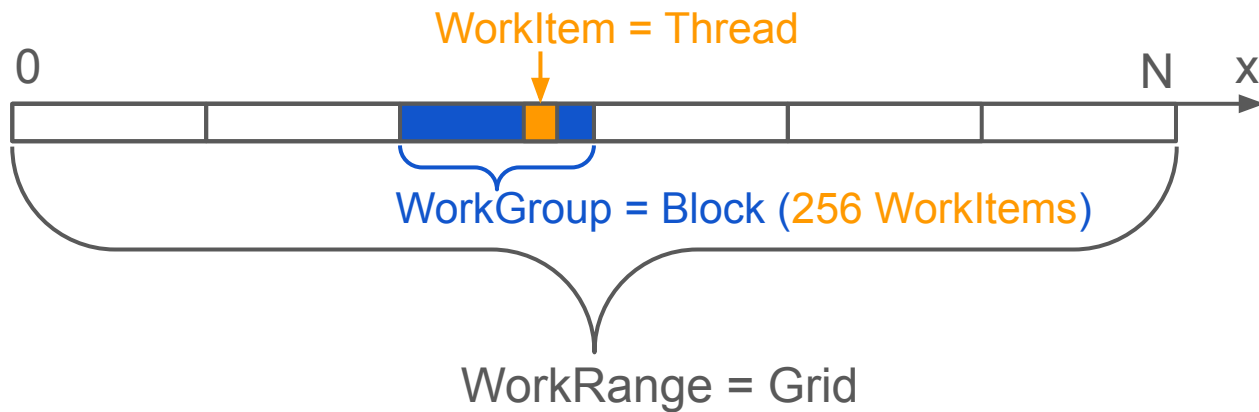
Модель вычислений массового параллелизма



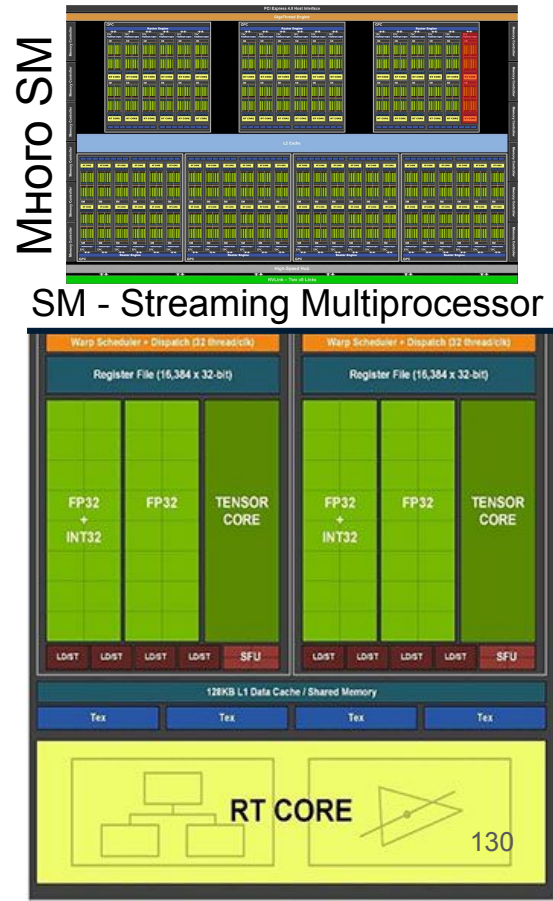
Модель вычислений массового параллелизма



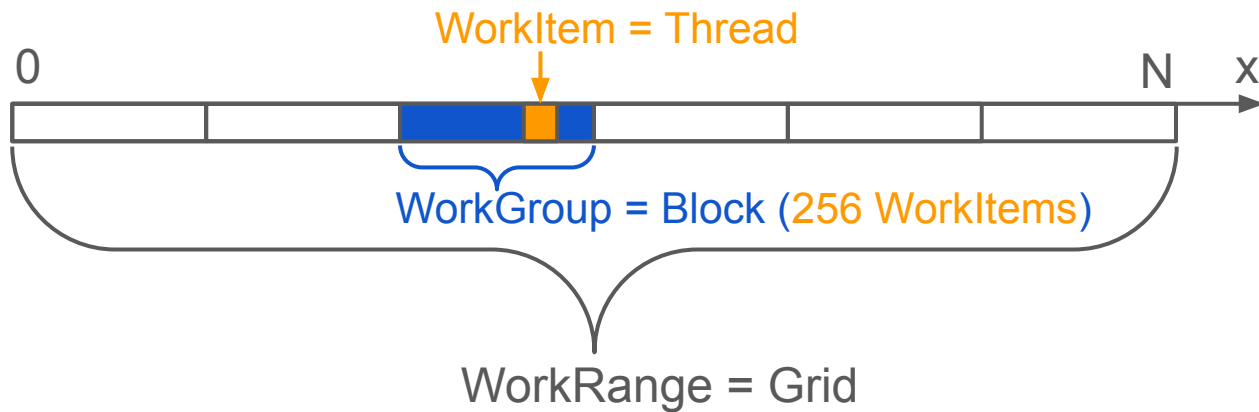
Модель вычислений массового параллелизма



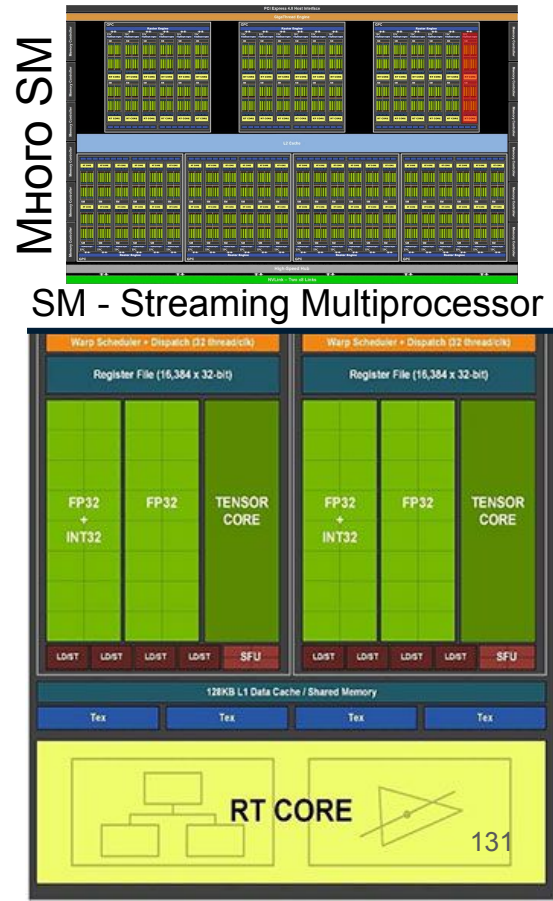
Но ведь в одном warp 32 потока!
(у **AMD**: 64 потока в wavefront)
Как так?



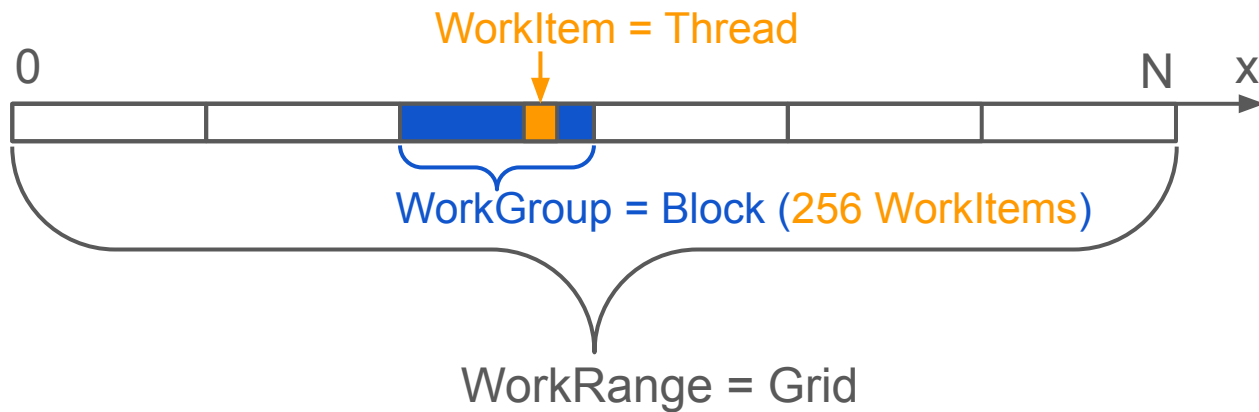
Модель вычислений массового параллелизма



WorkGroup = Block (256 WorkItems) = 8 x Warps (32 threads)

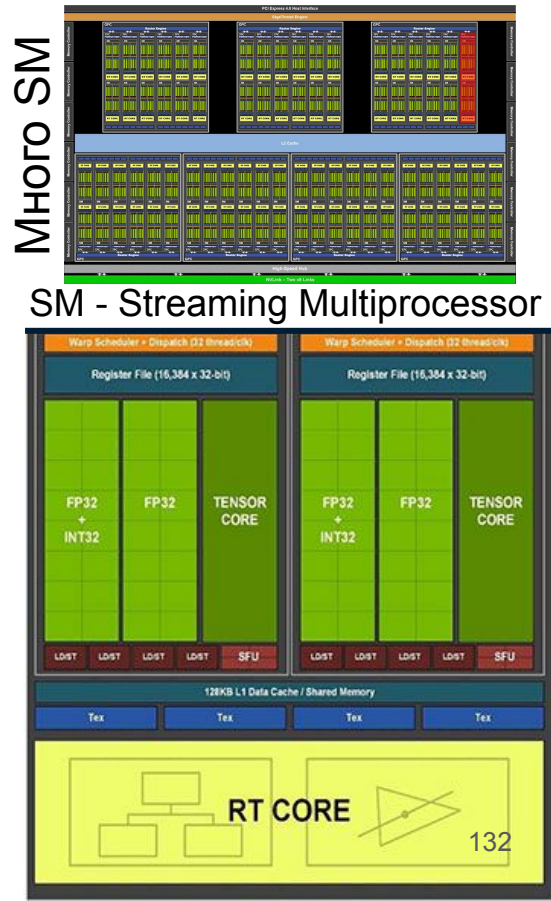


Модель вычислений массового параллелизма

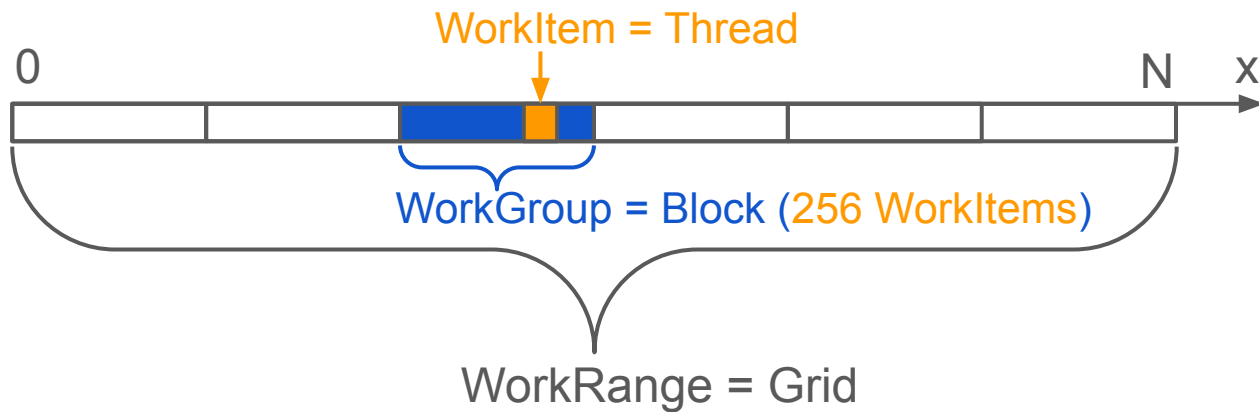


WorkGroup = Block (256 WorkItems) = 8 x Warps (32 threads)

Зачем нам контроль над **WorkGroups**?
Зачем на уровне API знать про **warps**?

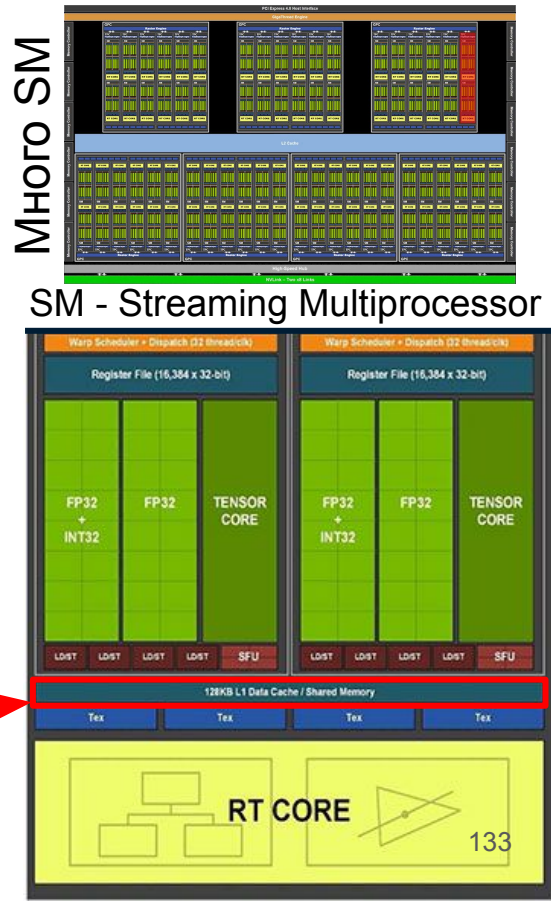


Модель вычислений массового параллелизма

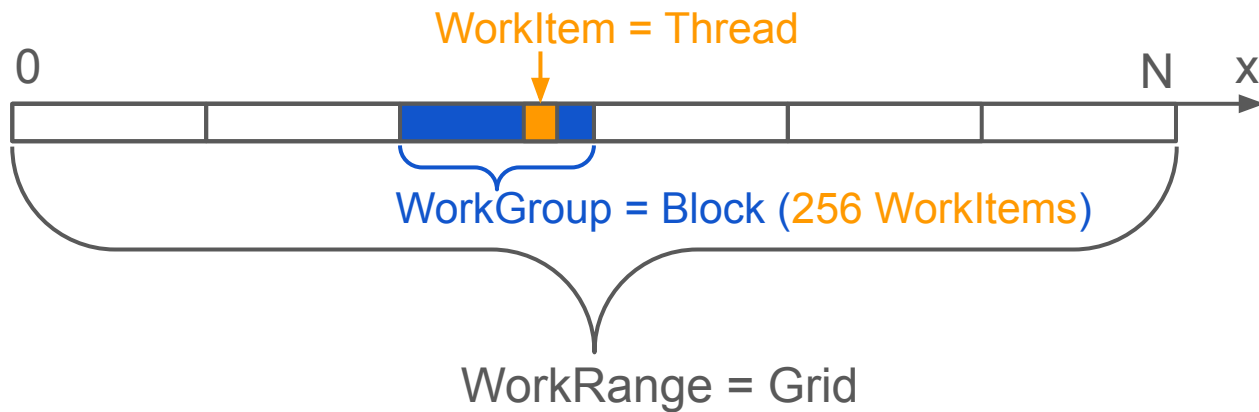


WorkGroup = Block (256 WorkItems) = 8 x Warps (32 threads)

- WorkItems коммуницируют через VRAM
- WorkItems в рамках одной WorkGroup общаются эффективнее - через **Shared/Local Memory (L1)**



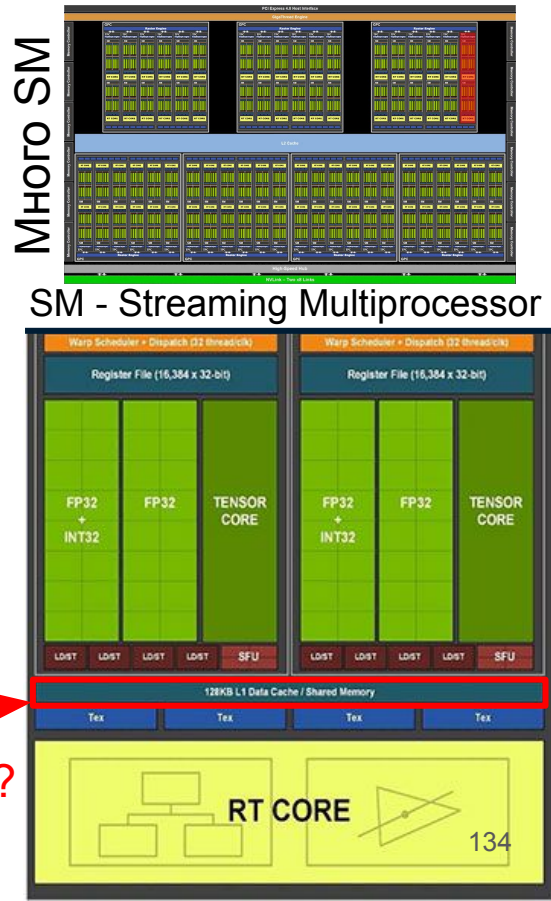
Модель вычислений массового параллелизма



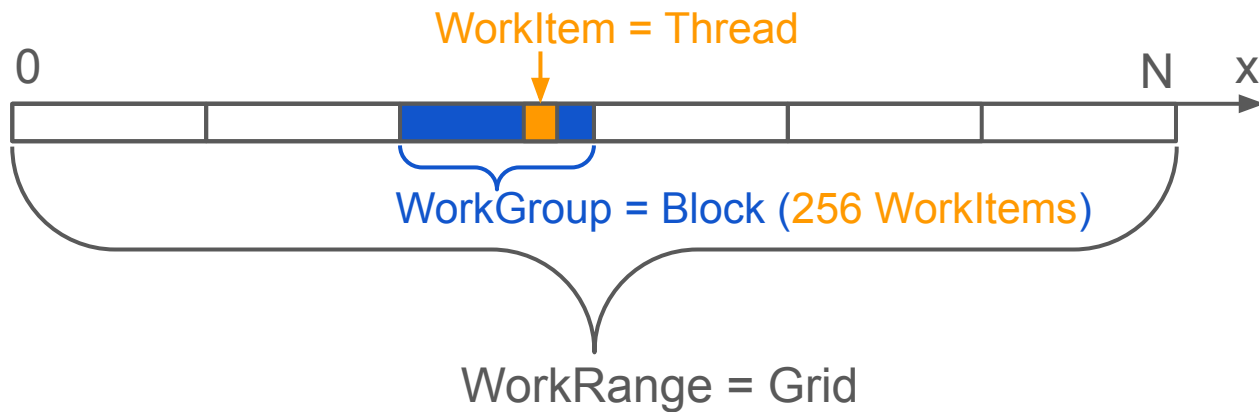
WorkGroup = Block (256 WorkItems) = 8 x Warps (32 threads)

- WorkItems коммуницируют через VRAM
- WorkItems в рамках одной WorkGroup общаются эффективнее - через **Shared/Local Memory (L1)**

Могут ли **warp**-ы одной **WorkGroup** исполняться на разных SM?



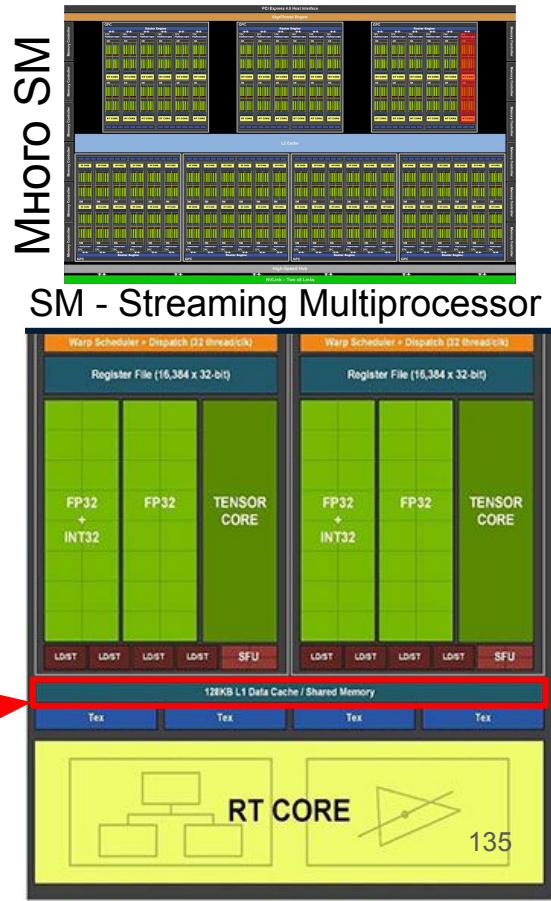
Модель вычислений массового параллелизма



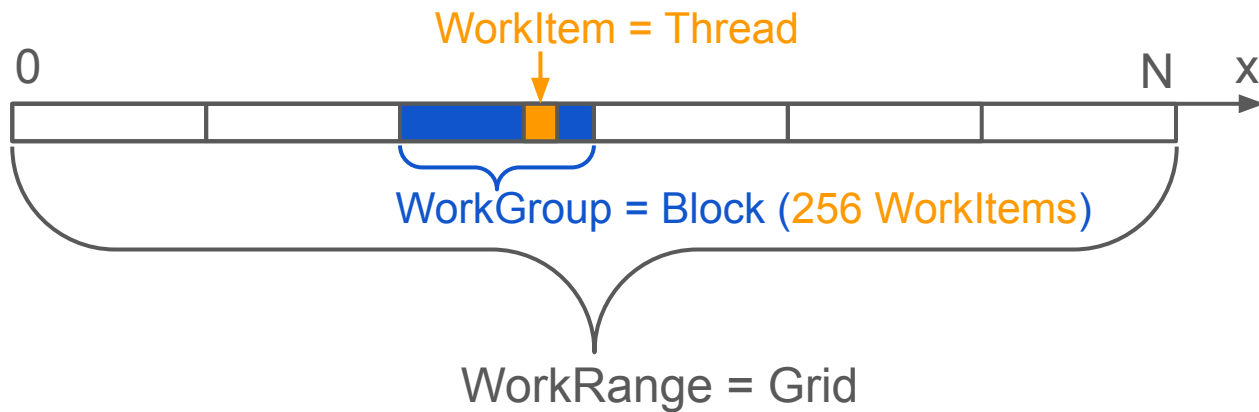
WorkGroup = Block (256 WorkItems) = 8 x Warps (32 threads)

- WorkItems коммуницируют через VRAM
- WorkItems в рамках одной WorkGroup общаются эффективнее - через **Shared/Local Memory (L1)**

А если один такой поток записал в **Local Memory** число - увидит ли другой поток этой **WorkGroup**?



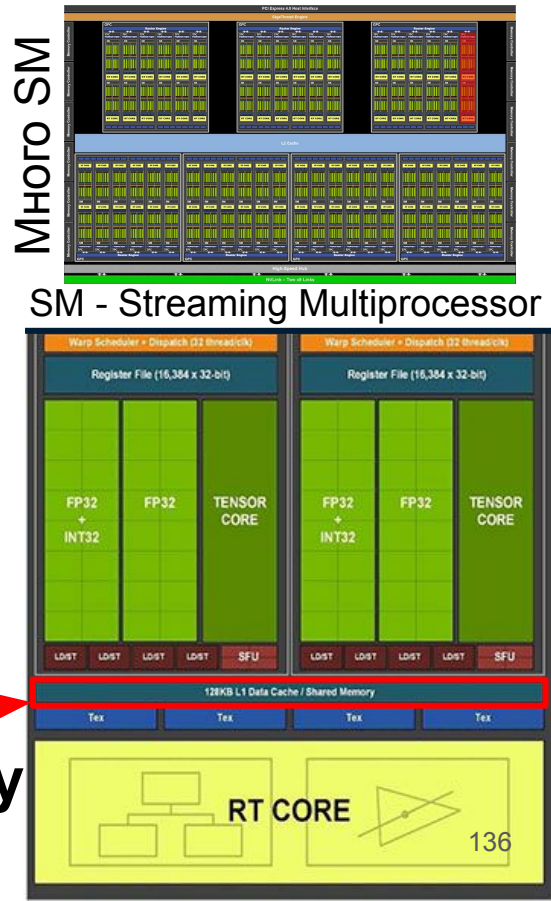
Модель вычислений массового параллелизма



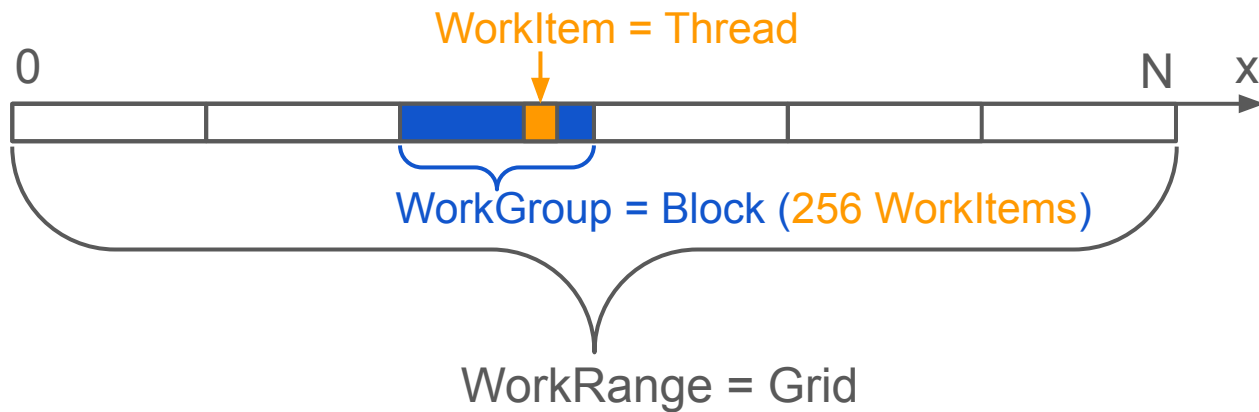
WorkGroup = Block (256 WorkItems) = 8 x Warps (32 threads)

- WorkItems коммуницируют через VRAM
- WorkItems в рамках одной WorkGroup общаются эффективнее - через **Shared/Local Memory (L1)**

А если один такой поток записал в **Local Memory** memory число - увидит ли другой поток этой **WorkGroup**? **barrier!**

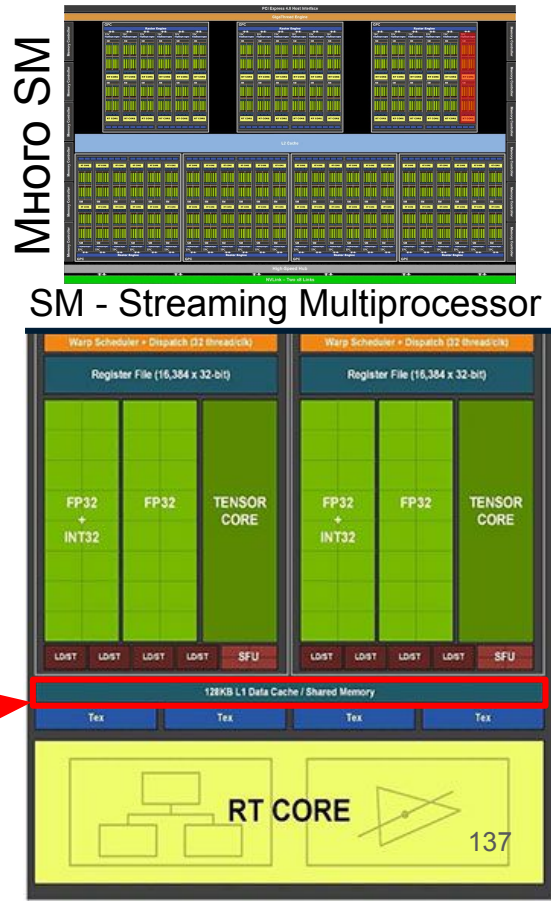


Модель вычислений массового параллелизма



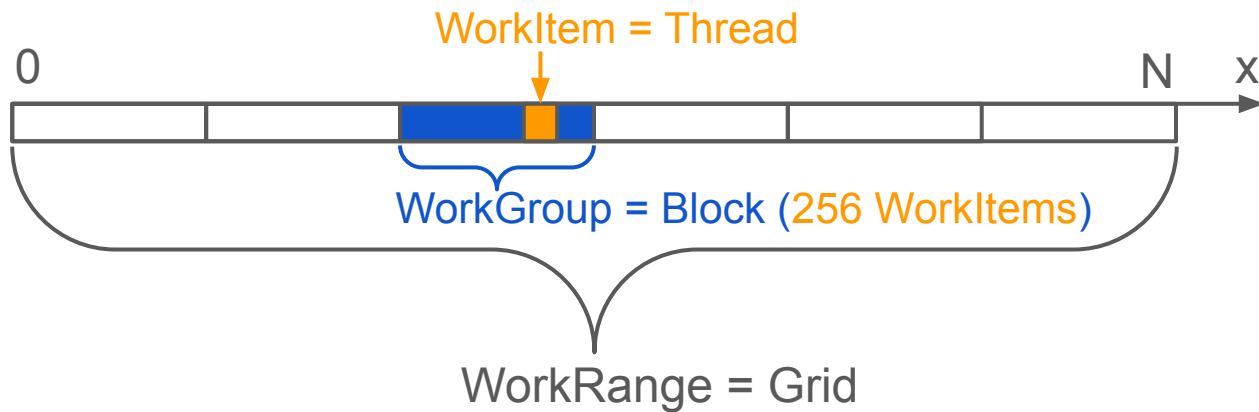
WorkGroup = Block (256 WorkItems) = 8 x Warps (32 threads)

- **WorkItems** коммуницируют через VRAM
- **WorkItems** в рамках одной **WorkGroup** общаются эффективнее - через **Shared/Local Memory (L1)**



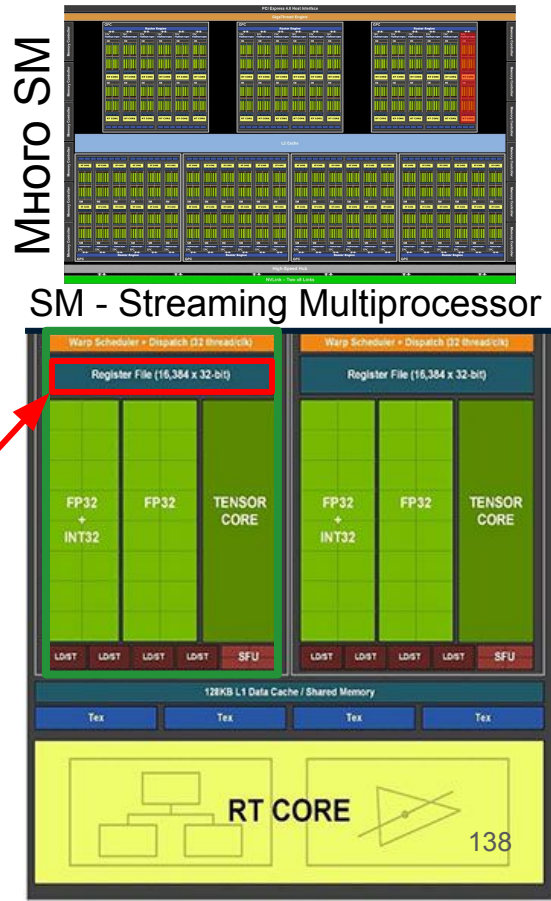
Могут ли потоки одного warp-а общаться еще эффективнее?

Модель вычислений массового параллелизма

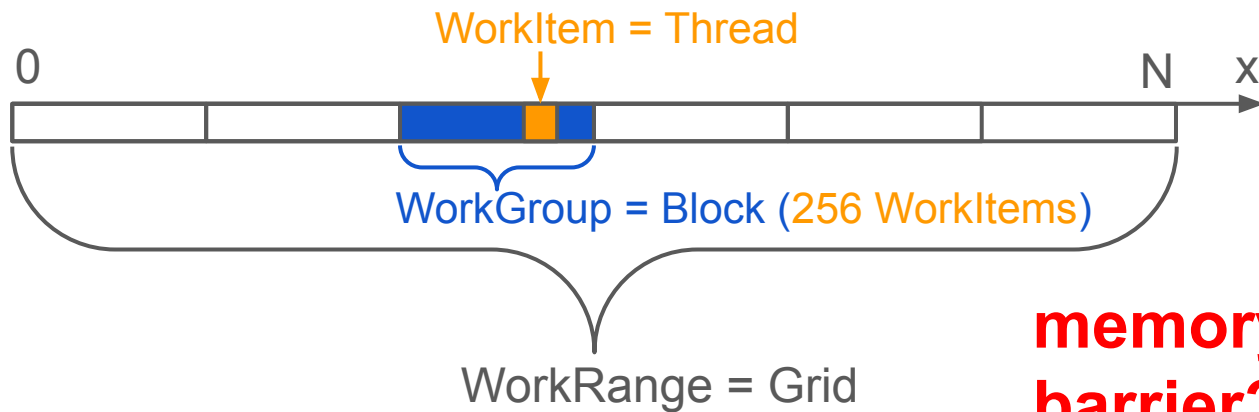


$\text{WorkGroup} = \text{Block} (256 \text{ WorkItems}) = 8 \times \text{Warps} (32 \text{ threads})$

- **WorkItems** коммуницируют через VRAM
- **WorkItems** в рамках одной **WorkGroup** общаются эффективнее - через **Shared/Local Memory (L1)**
- **WorkItems** в рамках одного warp-а могут подглядывать друг другу в **регистры** (*shuffle instr.*)



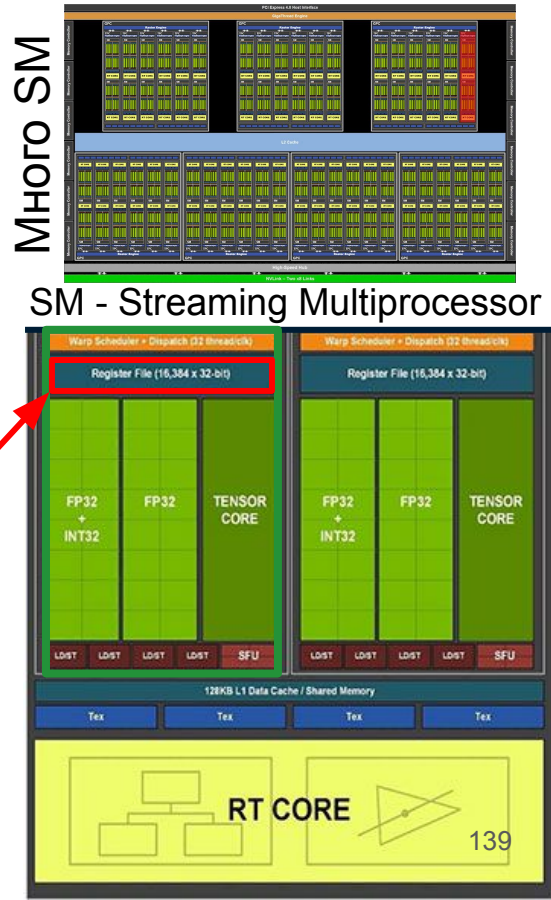
Модель вычислений массового параллелизма



WorkGroup = Block (256 WorkItems) = 8 x Warps (32 threads)

- WorkItems коммуницируют через VRAM
- WorkItems в рамках одной WorkGroup общаются эффективнее - через **Shared/Local Memory (L1)**
- WorkItems в рамках одного warp-а могут подглядывать друг другу в **регистры** (shuffle instr.)

memory barrier?

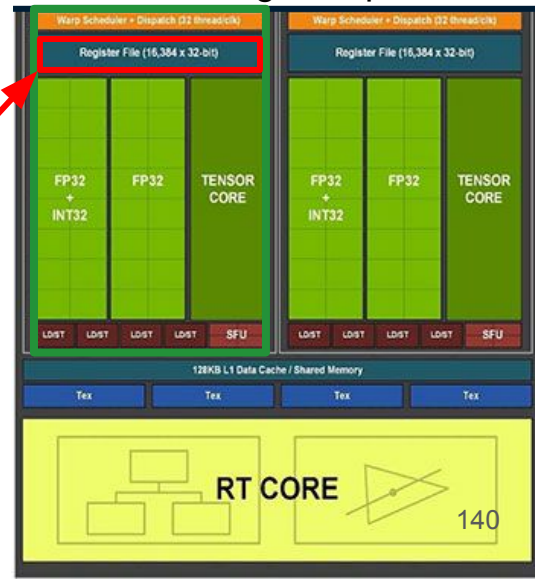


GLSL (Graphics Library Shading Language)

dFdx(...), dFdy(...) — return the partial derivative of an argument
with respect to x or y

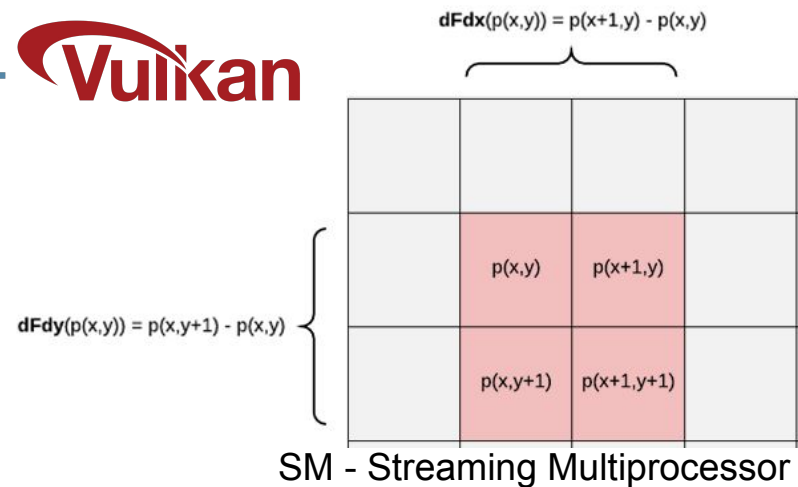
- **WorkItems** коммуницируют через VRAM
- **WorkItems** в рамках одной **WorkGroup** общаются эффективнее - через **Shared/Local Memory (L1)**
- **WorkItems** в рамках одного warp-а могут подглядывать друг другу в **регистры** (**shuffle instr.**)

SM - Streaming Multiprocessor

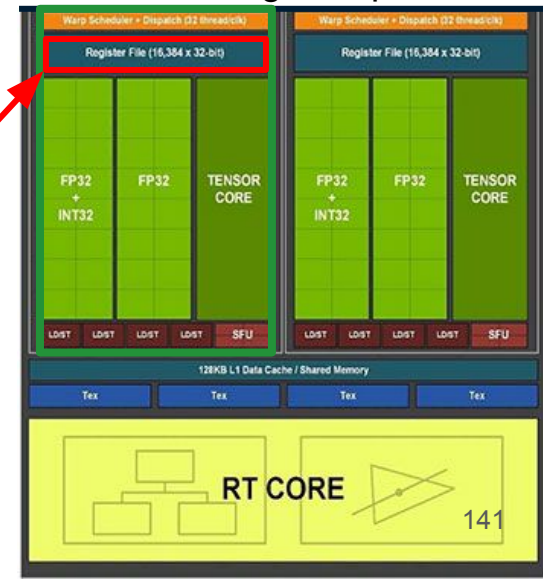


GLSL (Graphics Library Shading Language)

dFdx(...), dFdy(...) — return the partial derivative of an argument with respect to x or y

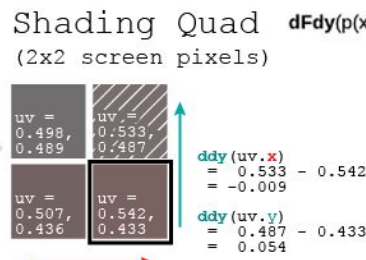
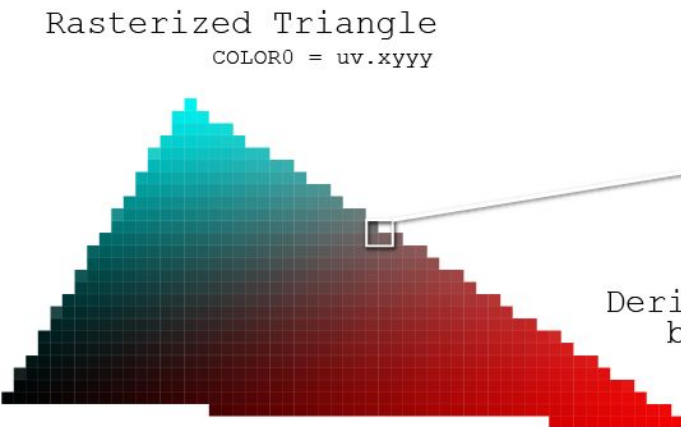


- **WorkItems** коммуницируют через VRAM
- **WorkItems** в рамках одной **WorkGroup** общаются эффективнее - через **Shared/Local Memory (L1)**
- **WorkItems** в рамках одного warp-а могут подглядывать друг другу в **регистры** (**shuffle instr.**)



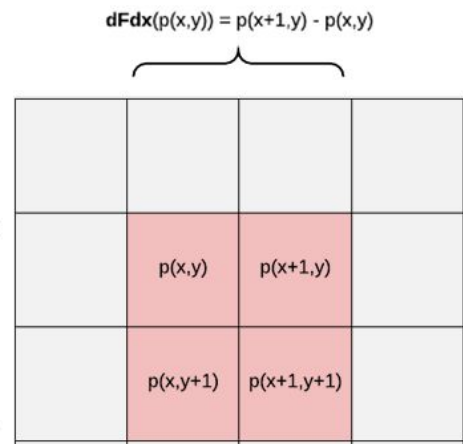
GLSL (Graphics Library Shading Language)

dFdx(...), dFdy(...) — return the partial derivative of an argument with respect to x or y

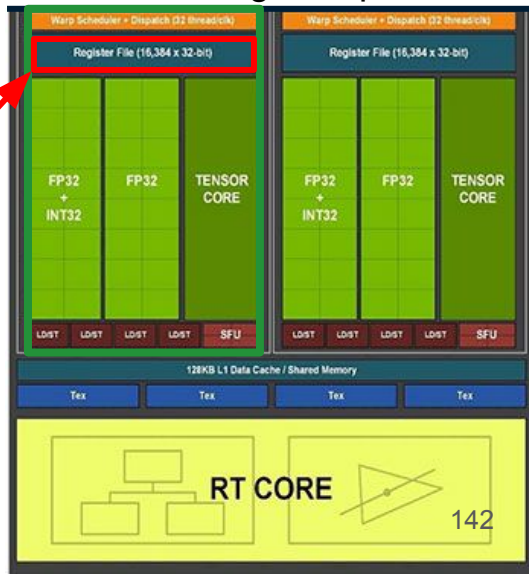


Derivatives for bottom-right fragment (high-precision)

ddx(uv.x) = 0.542 - 0.507 = 0.035
ddx(uv.y) = 0.433 - 0.436 = -0.003

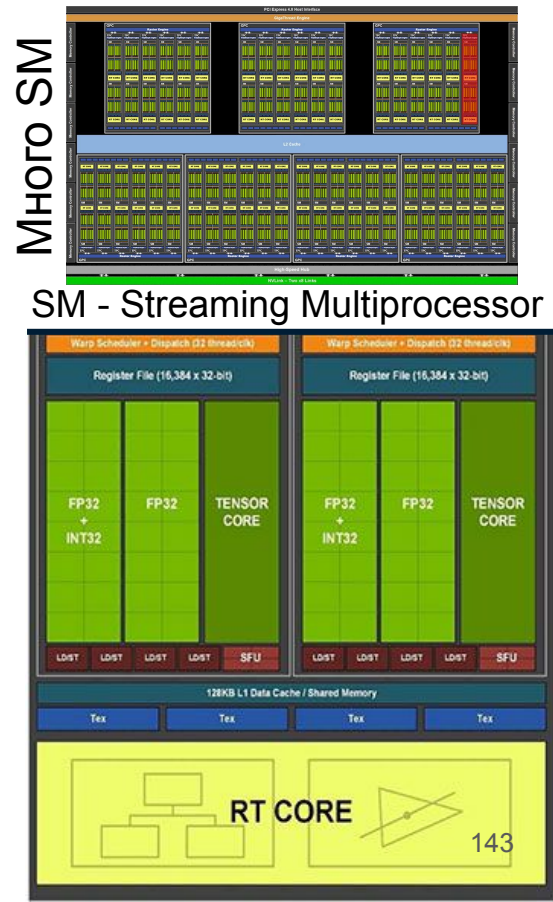
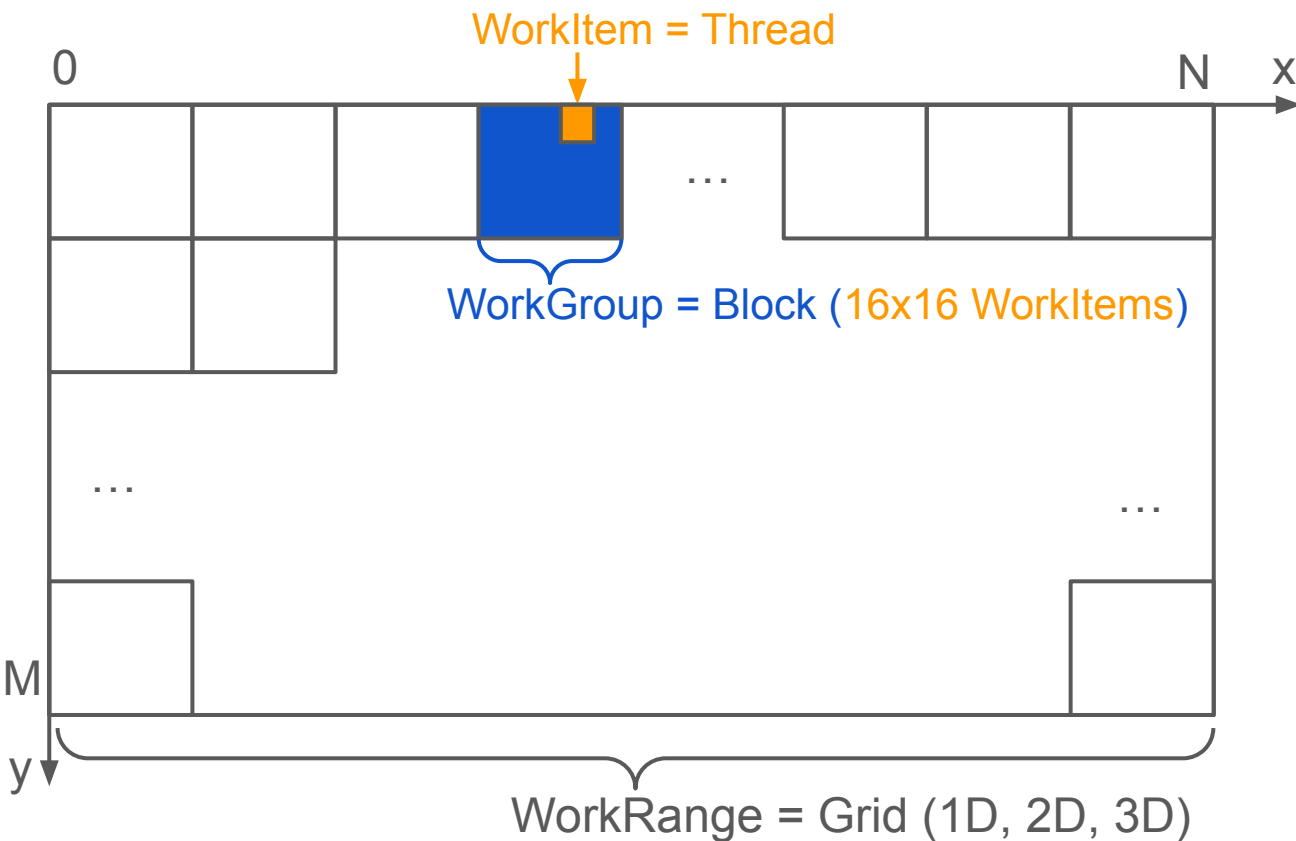


SM - Streaming Multiprocessor

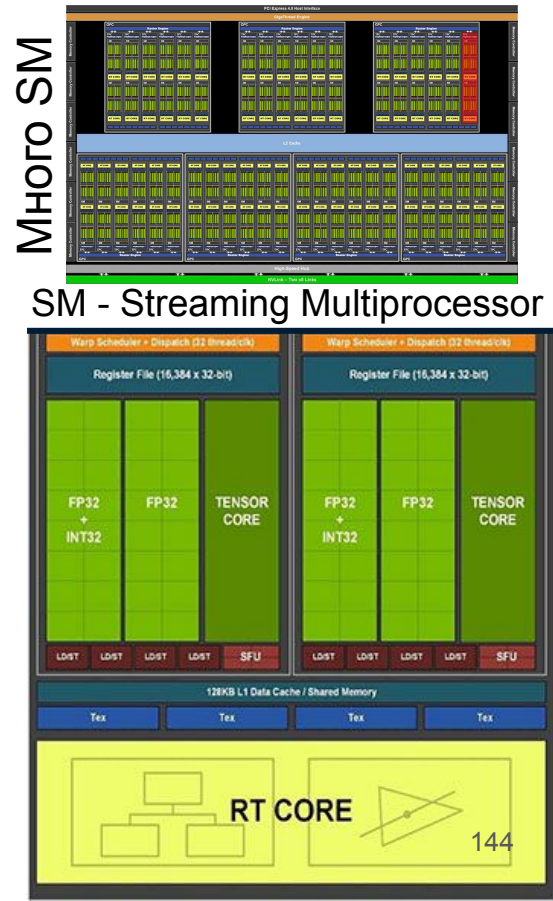
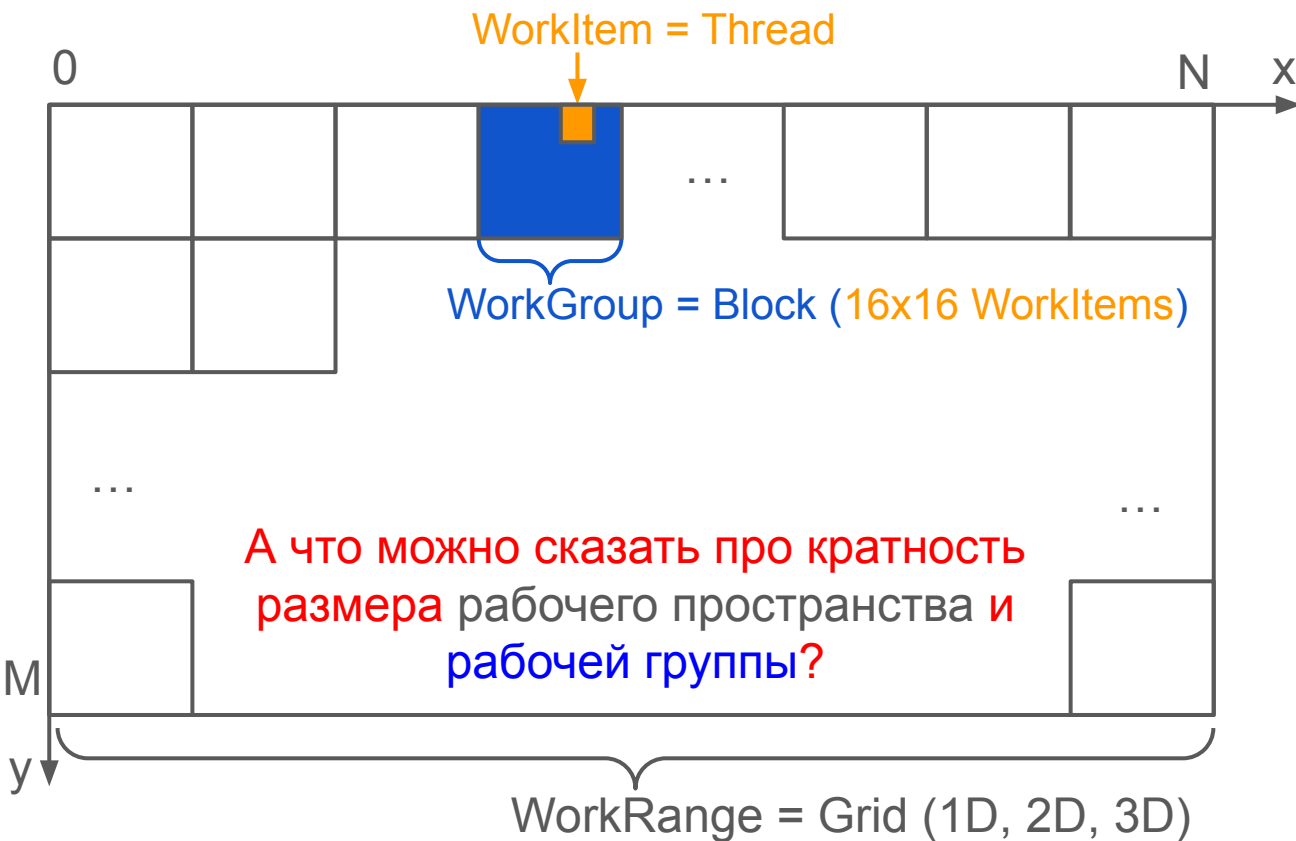


- **WorkItems** коммуницируют через VRAM
- **WorkItems** в рамках одной **WorkGroup** общаются эффективнее - через **Shared/Local Memory (L1)**
- **WorkItems** в рамках одного warp-а могут подглядывать друг другу в **регистры** (**shuffle instr.**)

Модель вычислений массового параллелизма



Модель вычислений массового параллелизма



Глава 5: Профилирование и оптимизация

МНОГО вычислений? (**ALU**)

МНОГО читать+писать? (**VRAM**)

$100 \cdot 10^{12}$ FLOPS



1000 GB/s



VRAM - GDDR6

Чего больше?
Как сравнить яблоки и
апельсины?

МНОГО вычислений? (**ALU**)

МНОГО читать+писать? (**VRAM**)

$100 \cdot 10^{12}$ FLOPS



1000 GB/s



VRAM - GDDR6

Чего больше?

МНОГО вычислений? (**ALU**)

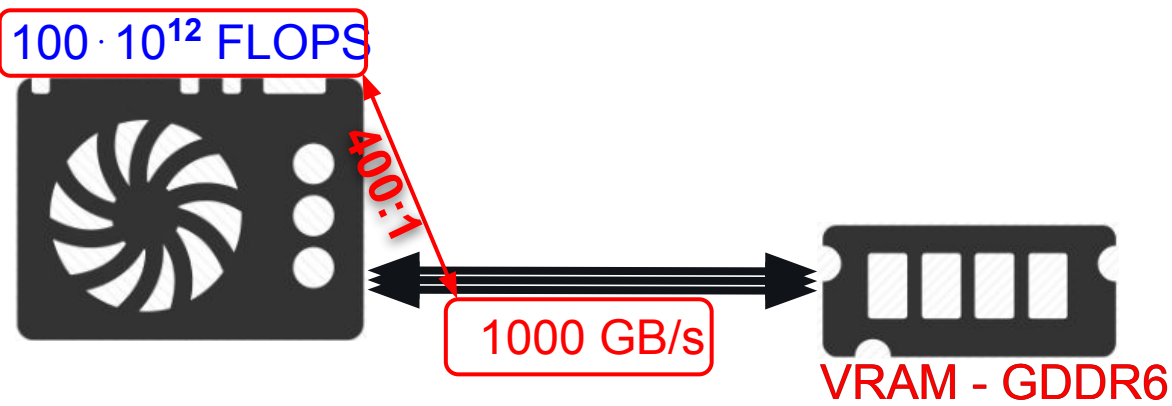
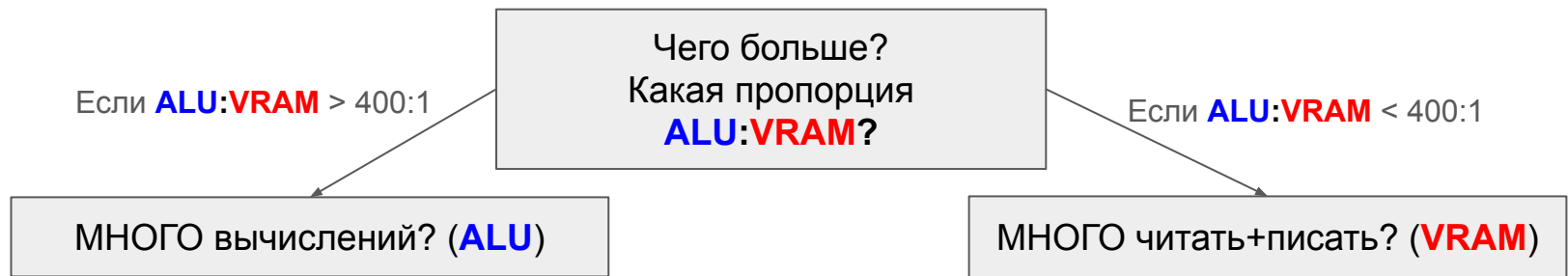
МНОГО читать+писать? (**VRAM**)

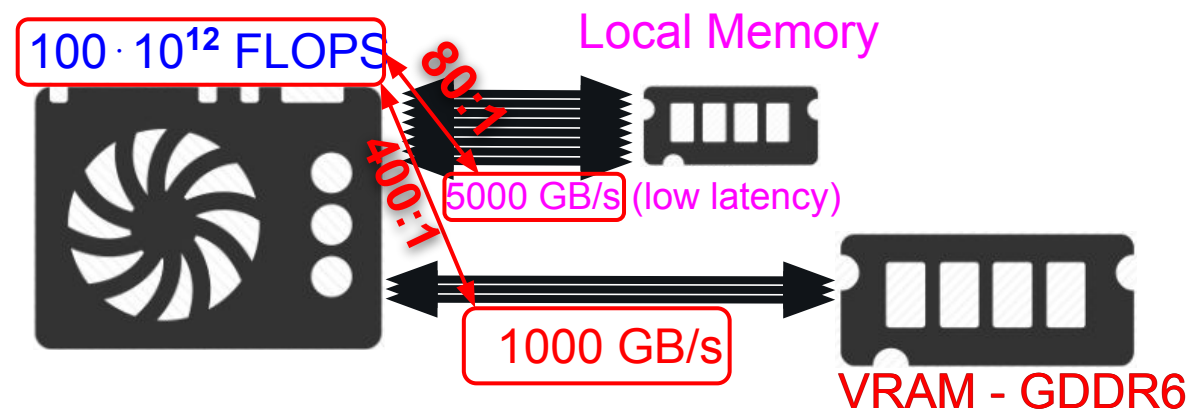
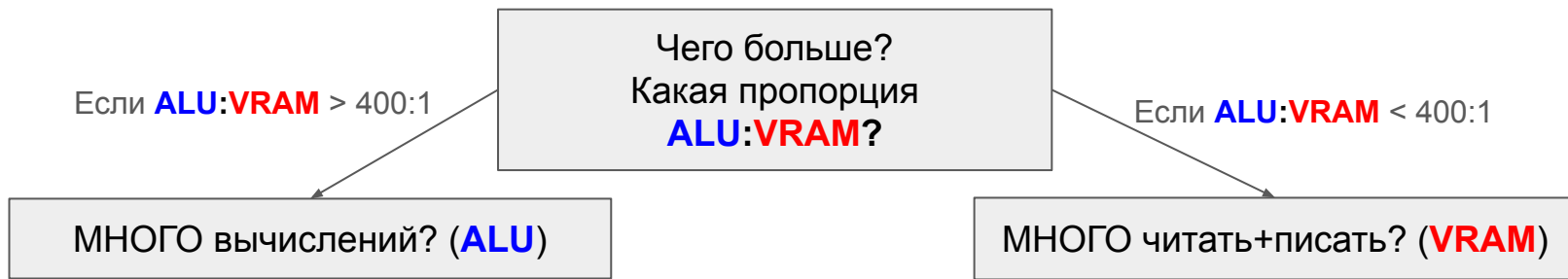
$100 \cdot 10^{12}$ FLOPS

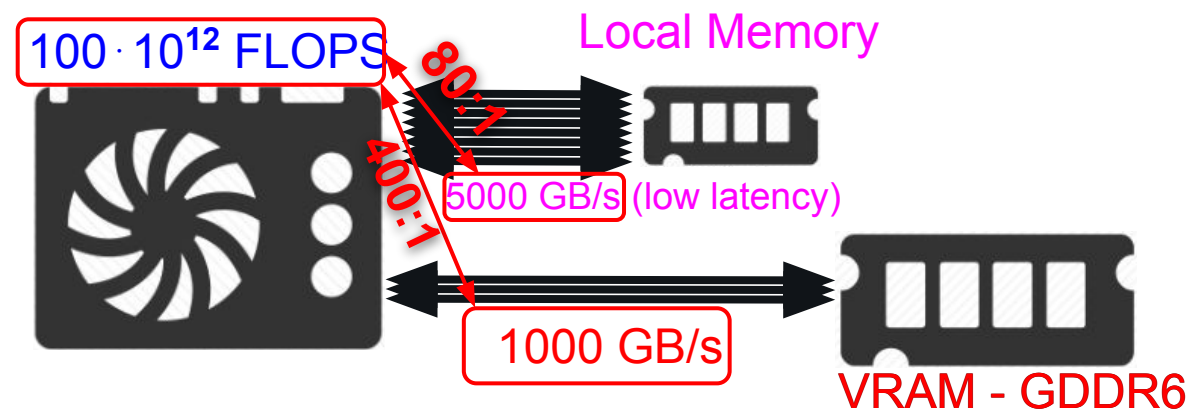
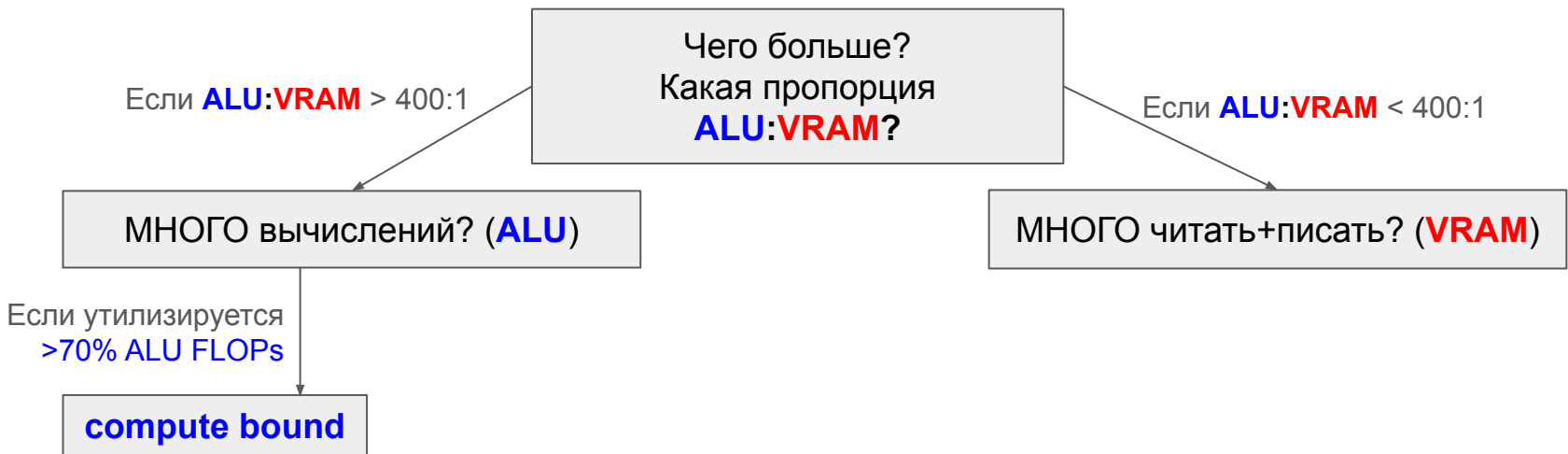
400:1

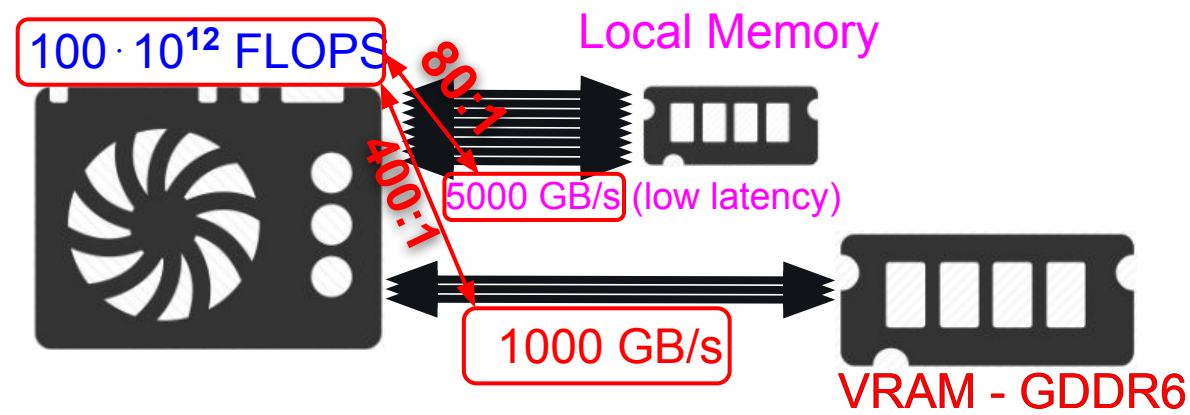
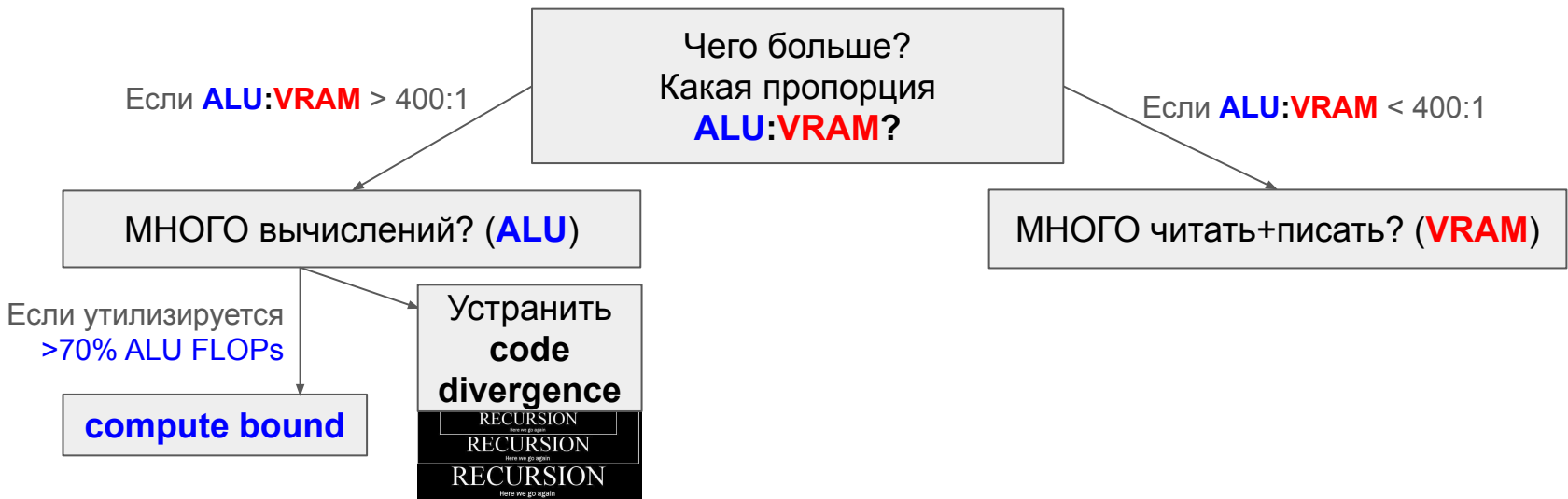
1000 GB/s

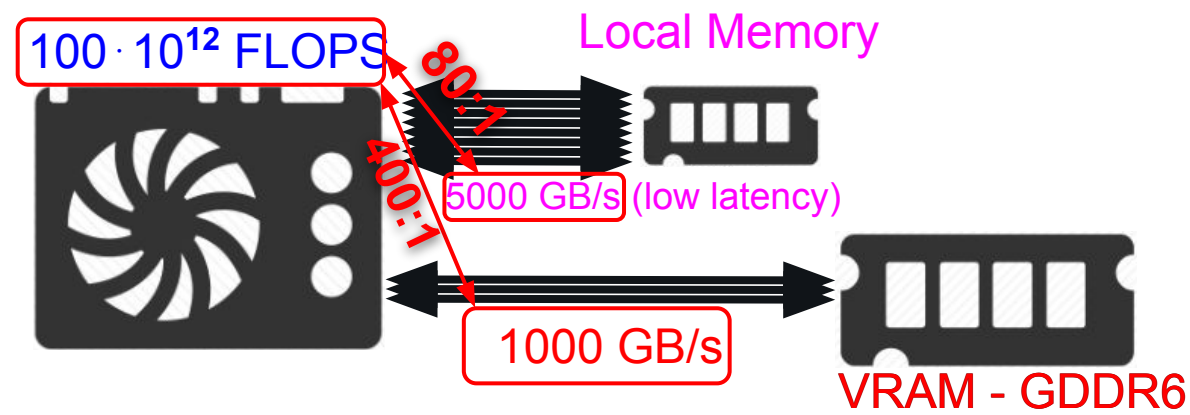
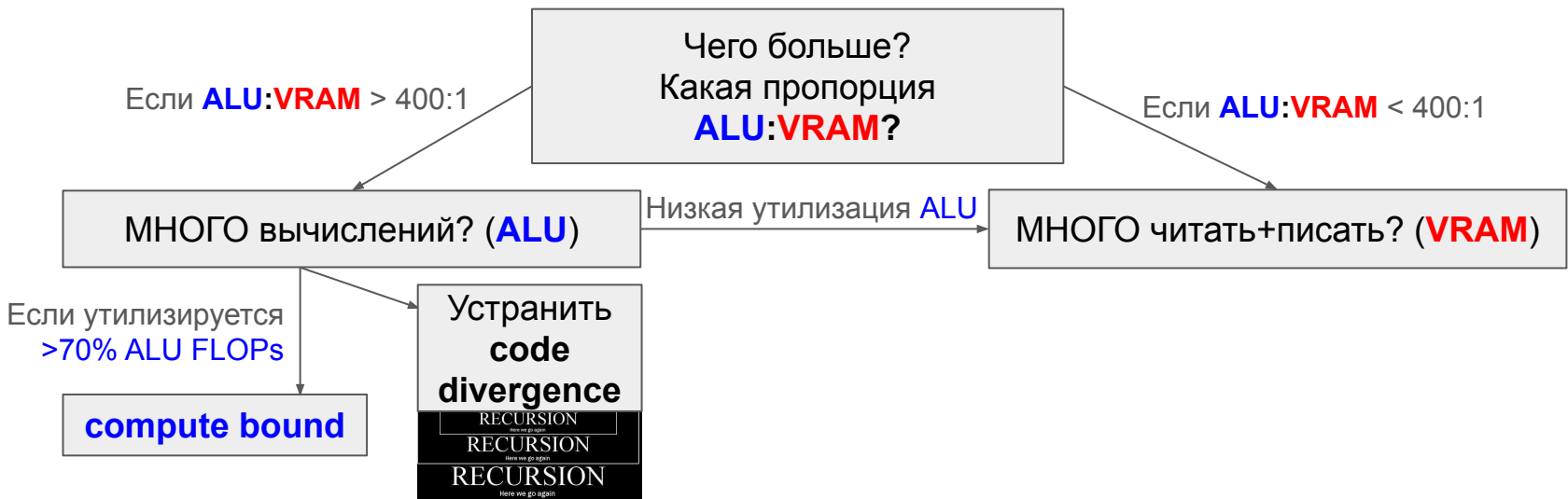
VRAM - GDDR6

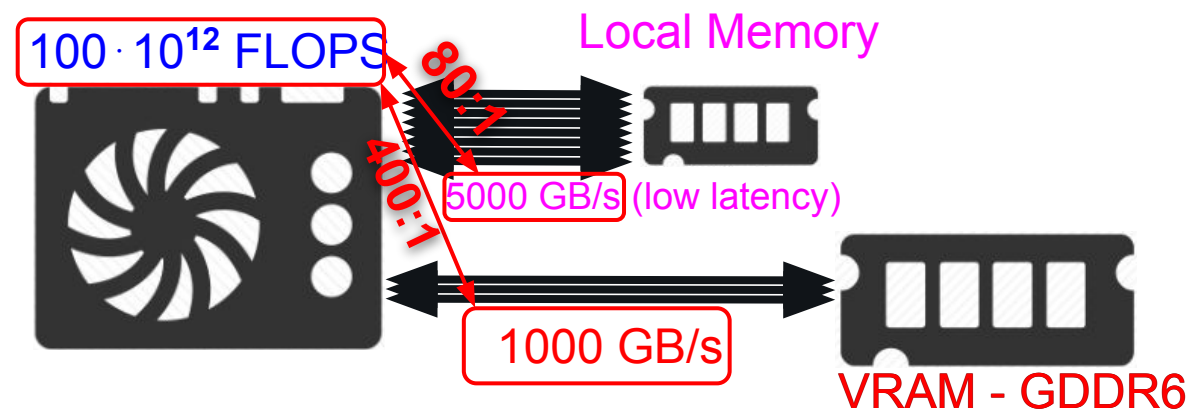


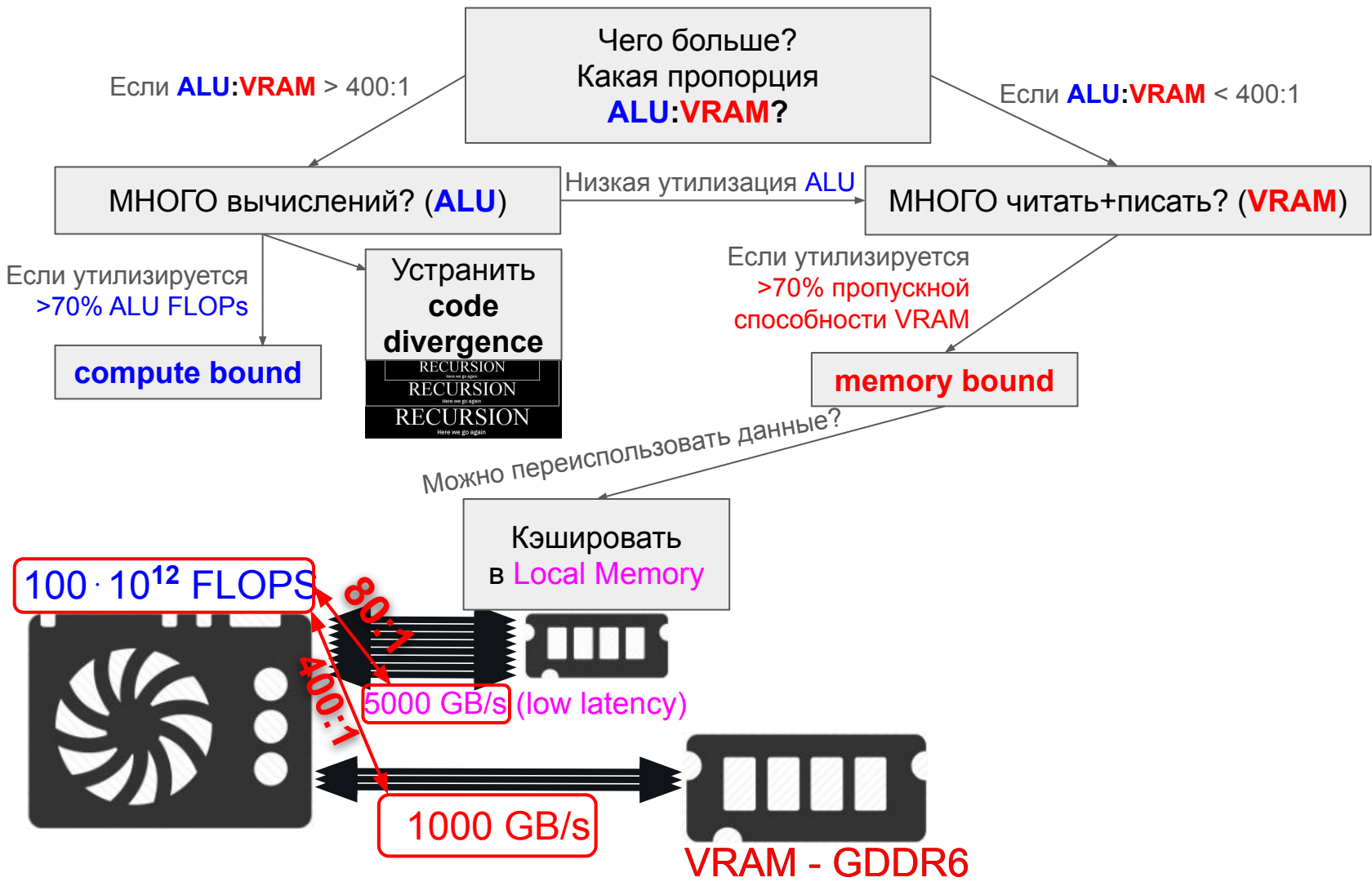


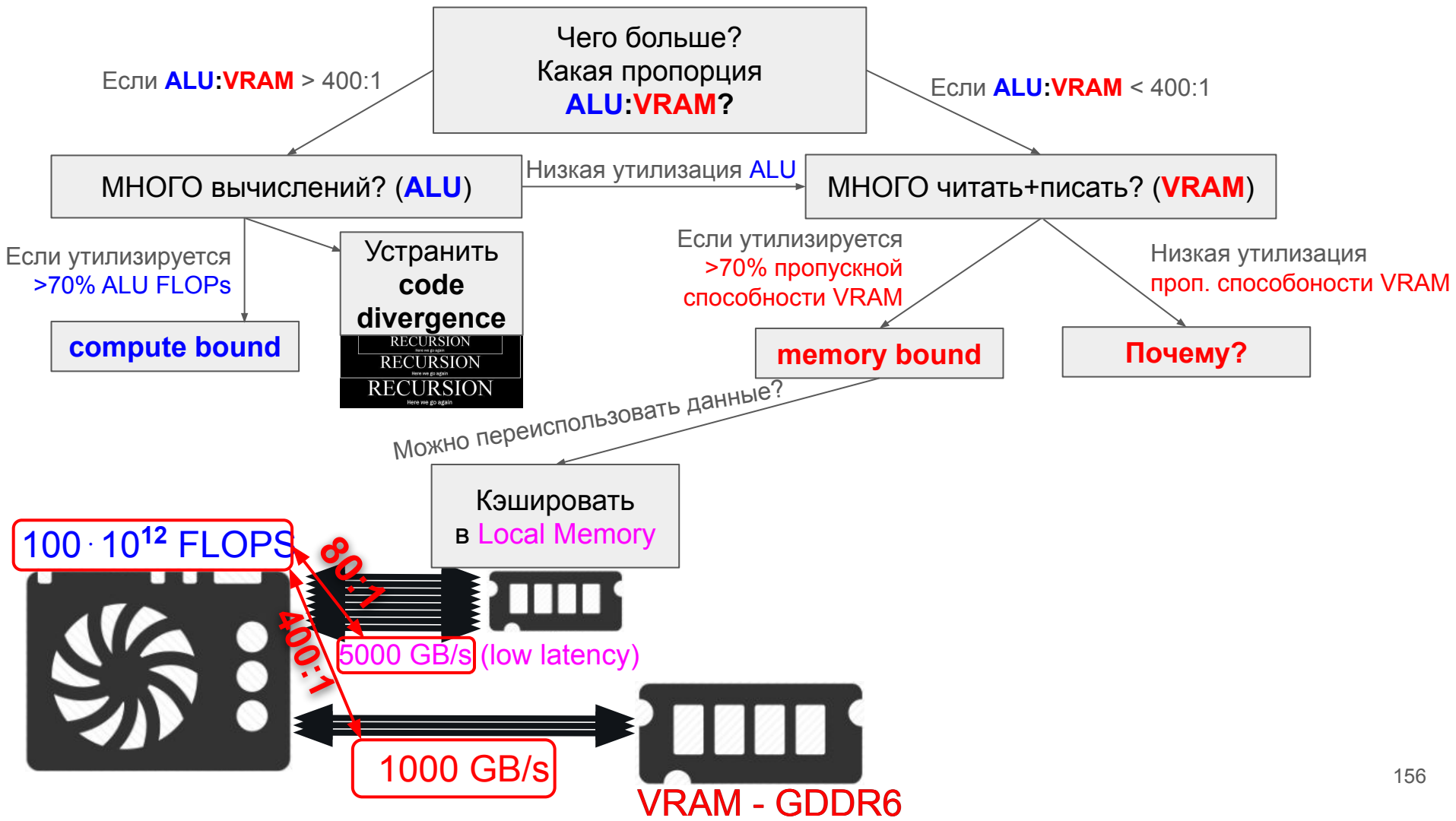


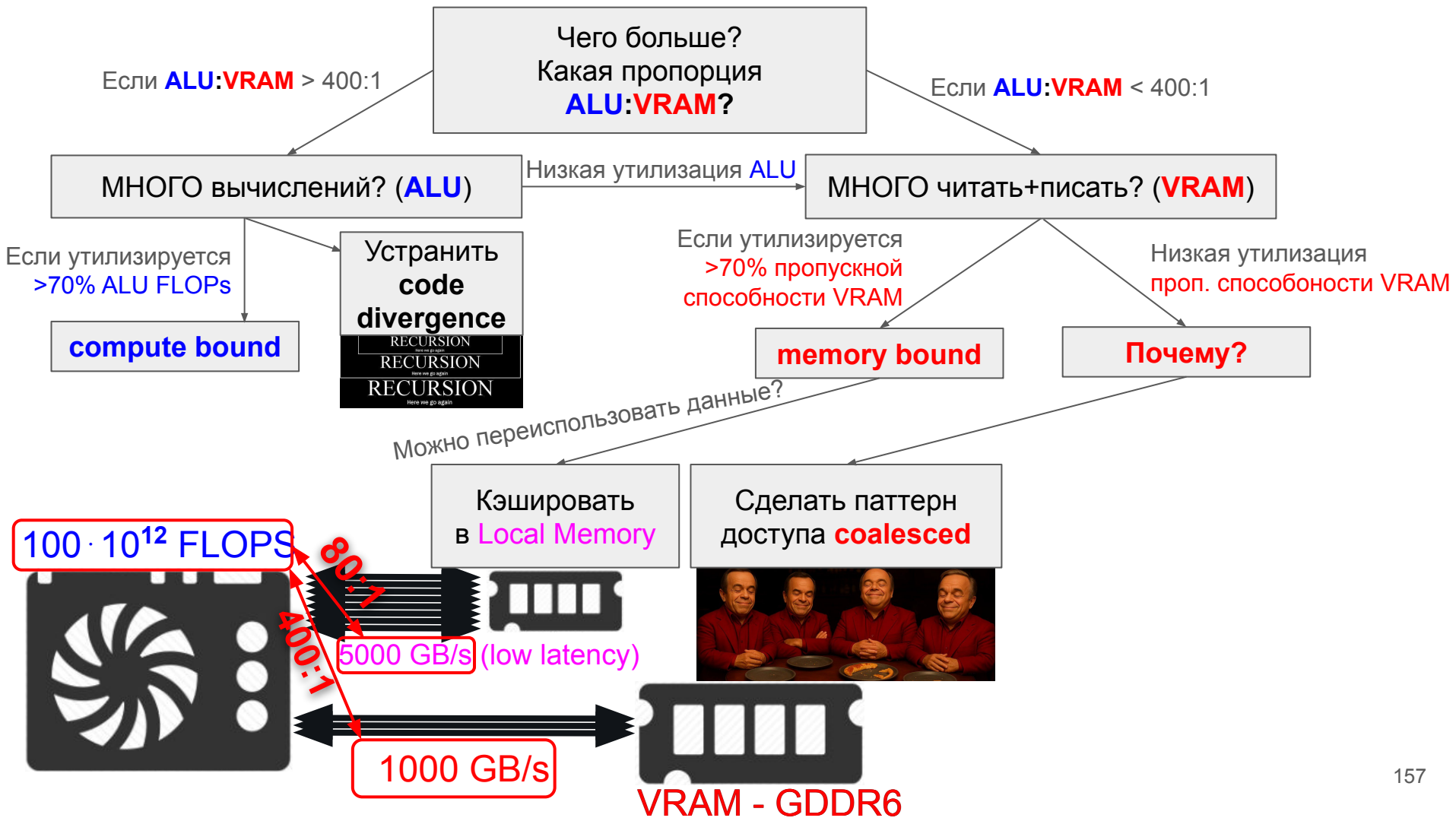


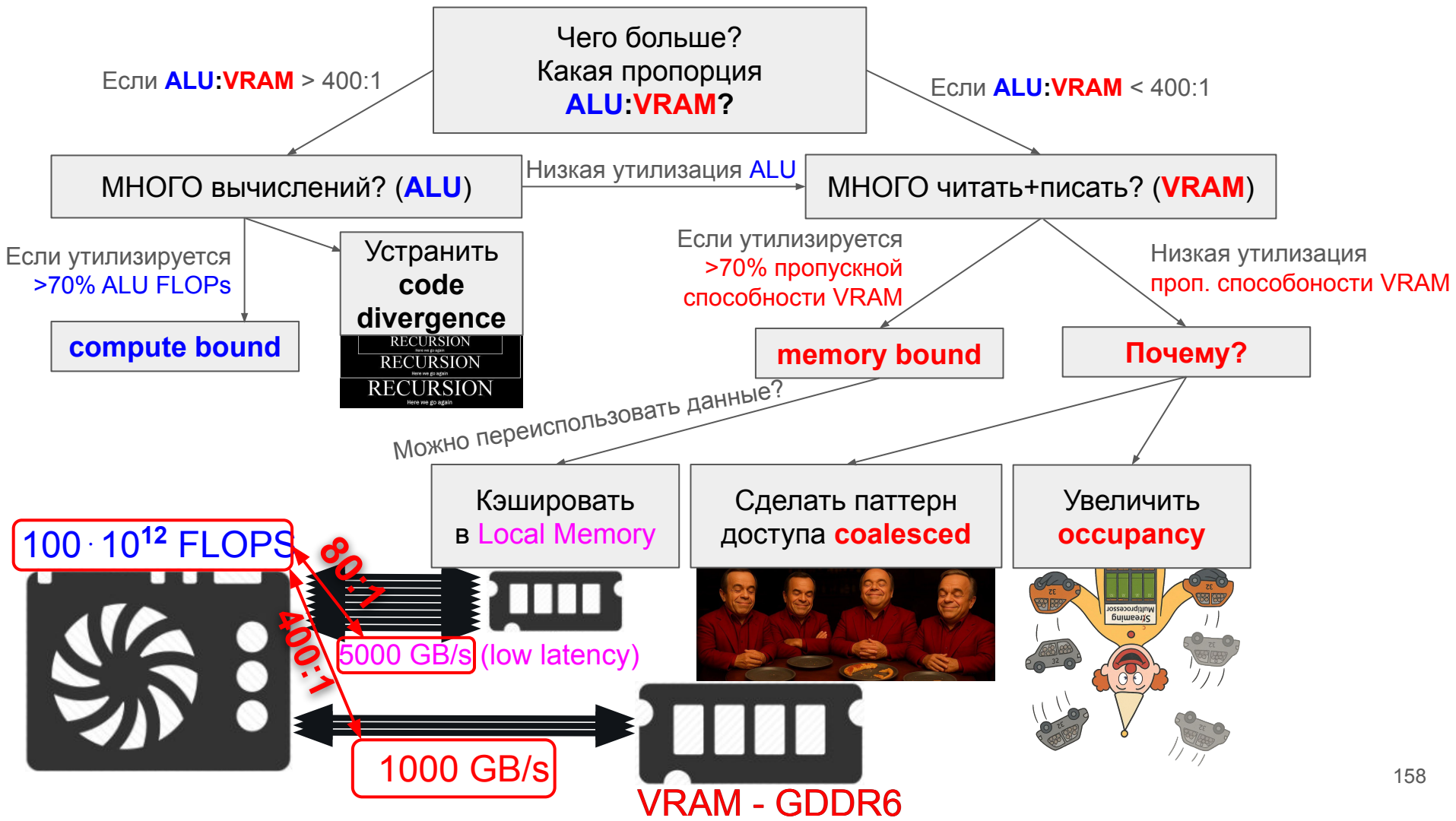


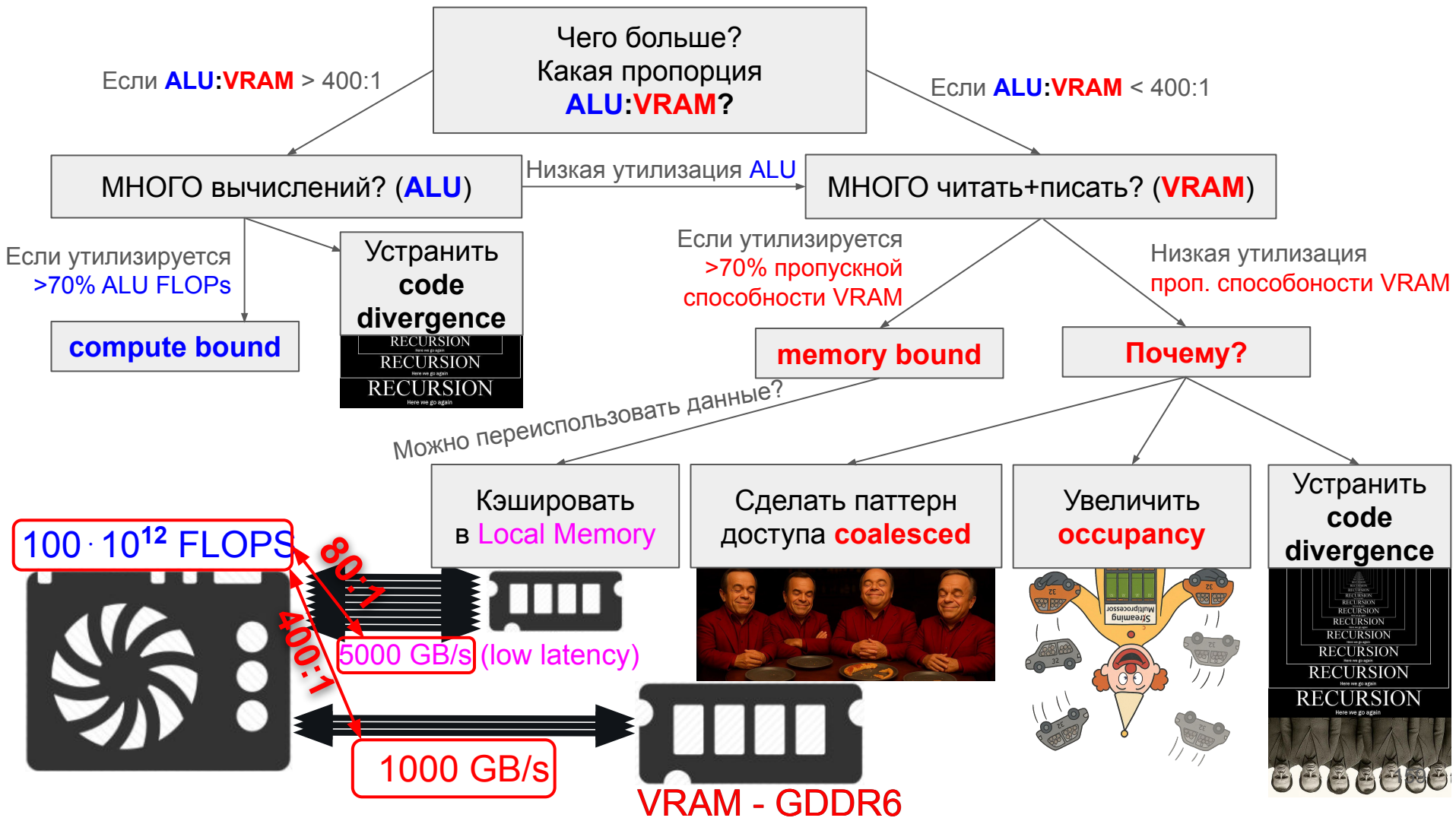


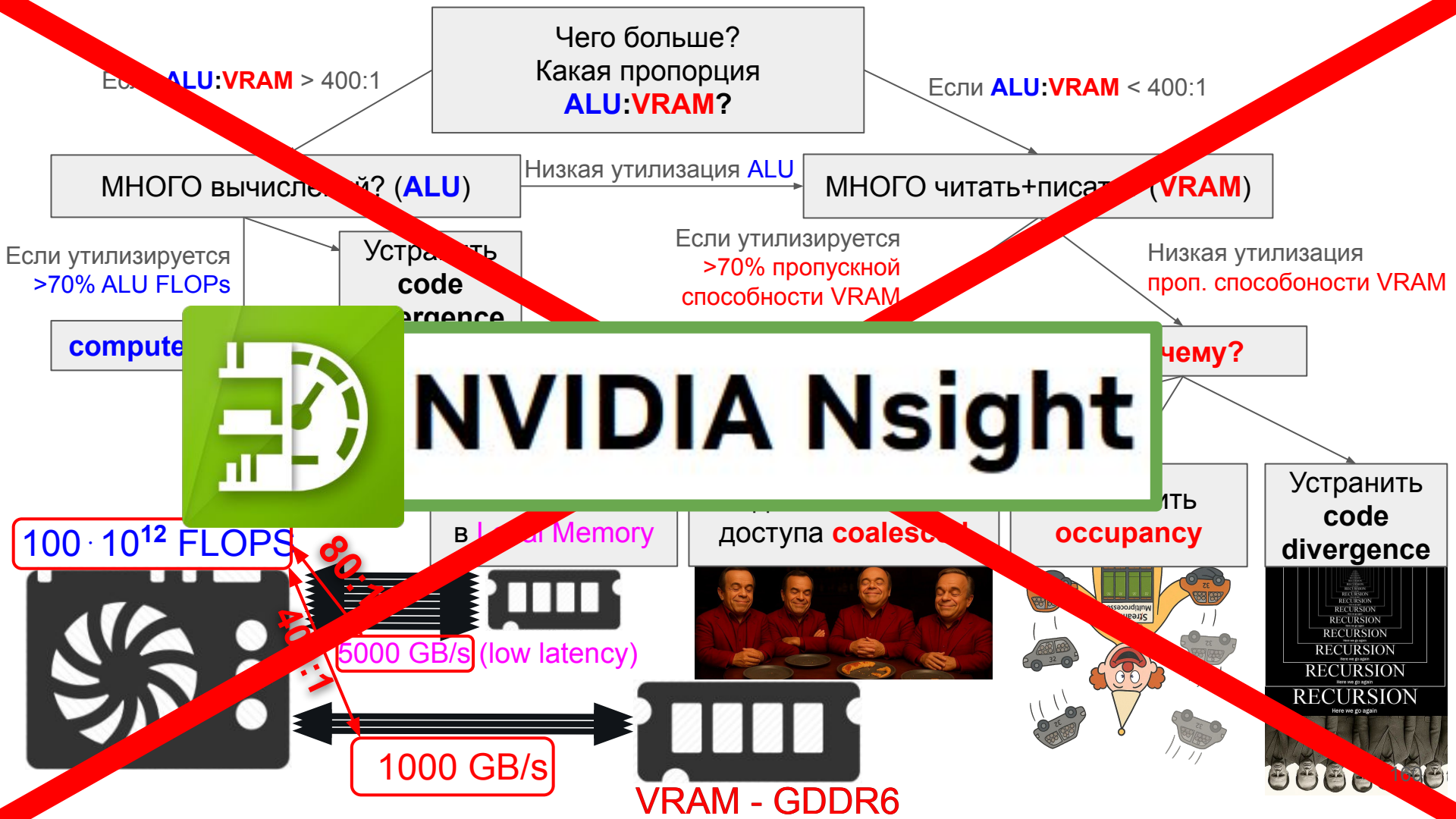






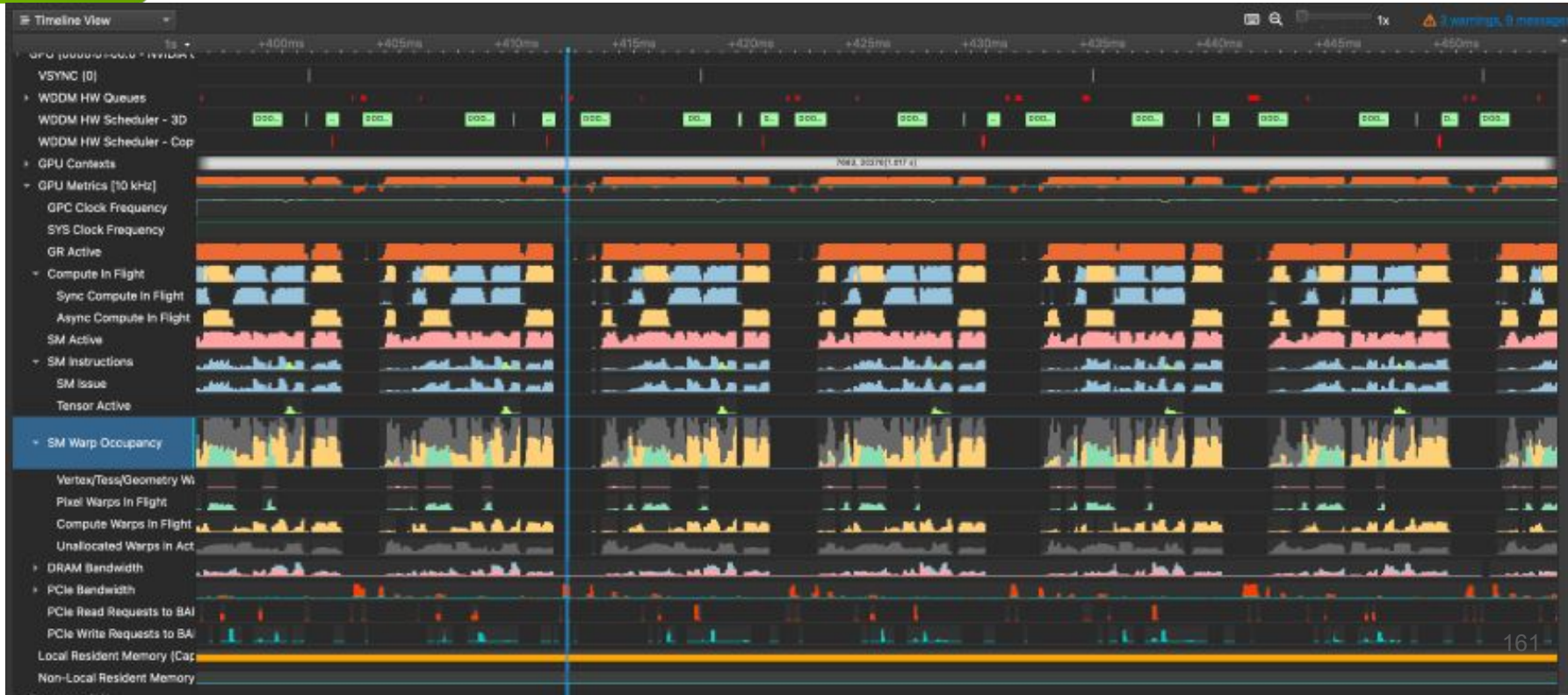








NVIDIA Nsight





Action Debug Profile Tools Window Help

[X] Disconnect [X] Terminate [X] Profile Kernel [X] Baselines [X] Metric Details

addConstDouble3.ncu-rep X addConstDouble3.ncu-rep X Untitled 1 X

Force [X] [Y] [Z]

Result	Time	GPU	SM Frequency	CC
double3 539 - addConstDouble3 (4096, 1, 1)x(256, 1, 1)	101.73 usecond	0 - NVIDIA TITAN V	1.17 cycle/nsecond	7.0

Source: uncoalescedGlobalAccesses.cu [X] [Y] [Z] Find...

Instructions Executed [X] [Y] [Z] [X] [Y] [Z]

Source

32 * global memory and generates an output array of double3 in global

33 */

34

35 #include <stdio.h>

36 #include <cuda_runtime_api.h>

37

38 #define BLOCK_SIZE 256

39

40 #define RUNTIME_API_CALL(apiFuncCall)

41 do {

42 cudaError_t _status = apiFuncCall;

43 if (_status != cudaSuccess) {

44 fprintf(stderr, "%s:%d: error: function %s failed with error

45 __FILE__, __LINE__, #apiFuncCall, cudaGetErrorStrr

46 exit(EXIT_FAILURE);

47 }

48 } while (0)

49

50 __global__ void addConstDouble3(int numElements, double3 *d_in, double3 *d_out)

51 {

52 int index = blockIdx.x * blockDim.x + threadIdx.x;

53 if (index < numElements)

54 {

55 double3 a = d_in[index];

56 a.x += k;

57 a.y += k;

58 a.z += k;

59 d_out[index] = a;

60 }

61 }

62 }

63 int main (int argc, char *argv[])

64 {

65 // Error code to check return values for CUDA calls

66 cudaError_t err = cudaSuccess;

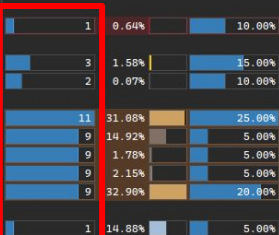
67 double constK = 10.0;

68

69 int kernelOption = 0;

70 if (argc > 1)

71 {

Live arp Stall Sampling
Registers (All Samples)Instructions
Executed

Report	Result	Time	GPU	SM Frequency	CC
addConstDouble	539 - addConstDouble (12288, 1, 1)x(256, 1, 1)	84.35 usecond	0 - NVIDIA TITAN V	1.16 cycle/nsecond	7.0

Source: uncoalescedGlobalAccesses.cu [X] [Y] [Z] Find...

Navigation: Instructions Executed [X] [Y] [Z] [X] [Y] [Z]

Source

32 * global memory and generates an output array of double3 in global

33 */

34

35 #include <stdio.h>

36 #include <cuda_runtime_api.h>

37

38 #define BLOCK_SIZE 256

39

40 #define RUNTIME_API_CALL(apiFuncCall)

41 do {

42 cudaError_t _status = apiFuncCall;

43 if (_status != cudaSuccess) {

44 fprintf(stderr, "%s:%d: error: function %s failed with error

45 __FILE__, __LINE__, #apiFuncCall, cudaGetErrorStrr

46 exit(EXIT_FAILURE);

47 }

48 } while (0)

49

50 __global__ void addConstDouble(int numElements, double *d_in, double *d_out)

51 {

52 int index = blockIdx.x * blockDim.x + threadIdx.x;

53 if (index < numElements)

54 {

55 d_out[index] = d_in[index] + k;

56 }

57 }

58

59 int main (int argc, char *argv[])

60 {

61 // Error code to check return values for CUDA calls

62 cudaError_t err = cudaSuccess;

63 double constK = 10.0;

64

65 int kernelOption = 0;

66 if (argc > 1)

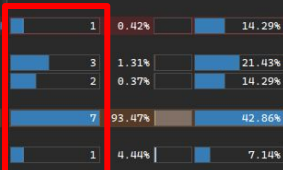
67 {

68 kernelOption = atoi(argv[1]);

69 }

70 }

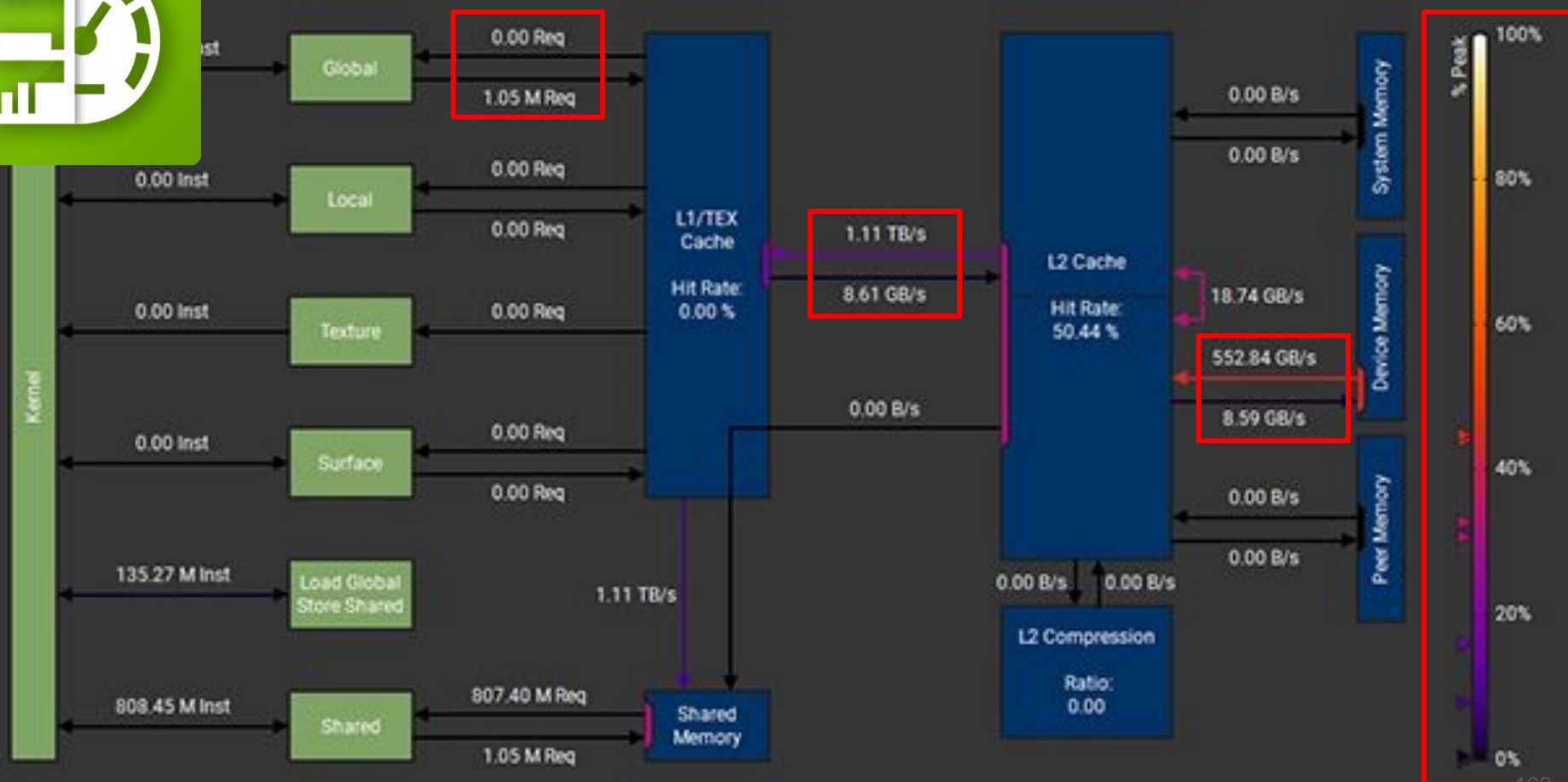
71 }

Live arp Stall Sampling
Registers (All Samples)Instructions
Executed



Memory Chart

Show As: **Throughput**





2 - 566 - mergeRanksAndIndicesKernel

Clear Baselines

Apply Rules

Occupancy Calculator

Source Comparison

Copy as Image

	Time	Cycles	GPU	SM Frequency	Process	Attributes
mergeRanksAndIndicesKernel (64, 1, 1)x(256, 1, 1)	4.19 us	4,326	0 - NVIDIA RTX A4500	1.03 cycle/ns	[762349] CuMergeSort	
mergeSortSharedKernel (4096, 1, 1)x(512, 1, 1)	555.10 us	5,79,537	0 - NVIDIA RTX A4500	1.04 cycle/ns	[762349] CuMergeSort	
generateSampleRanksKernel (64, 1, 1)x(256, 1, 1)	26.72 us	27,984	0 - NVIDIA RTX A4500	1.05 cycle/ns	[762349] CuMergeSort	

GPU Speed Of Light Throughput

GPU Throughput Chart

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	10.84 (-75.36%, z=-0.61)	Duration [us]	4.19 (-98.56%, z=-0.75)
Memory Throughput [%]	7.88 (-87.90%, z=-1.23)	Elapsed Cycles [cycle]	4,326 (-98.58%, z=-0.75)
L1/TEX Cache Throughput [%]	10.84 (-79.01%, z=-0.82)	SM Active Cycles [cycle]	2,192.52 (-99.26%, z=-0.74)
L2 Cache Throughput [%]	7.88 (-51.10%, z=-0.75)	SM Frequency [cycle/ns]	1.03 (-1.32%, z=-1.38)
DRAM Throughput [%]	5.53 (-83.26%, z=-1.08)	DRAM Frequency [cycle/ns]	7.55 (-0.53%, z=-1.39)

GPU Throughput



PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Baselines										
↑	↓	📄	📁	Difference Bars	Green/Red					
	Name	Launch	Report	Time	Cycles	Registers	GPU	SM Frequency	CC	Process
✓	mergeSort	560 - mergeSortSharedKernel (4096, ...	/home/prabhanshuc/Documents/me...	555.10 us	5,79,537	17	0 - NVIDIA RTX ...	1.04 cycle/ns	8.6	[762349] CuMergeSort
✓	generateSample	563 - generateSampleRanksKernel (6...	/home/prabhanshuc/Documents/me...	26.72 us	27,984	18	0 - NVIDIA RTX ...	1.05 cycle/ns	8.6	[762349] CuMergeSort
✓	mergeRanks	566 - mergeRanksAndIndicesKernel (...	/home/prabhanshuc/Documents/me...	4.19 us	4,326	16	0 - NVIDIA RTX ...	1.03 cycle/ns	8.6	[762349] CuMergeSort

Глава 6: Как выглядит код

OpenMP, OpenCL, CUDA, Vulkan (GLSL)

Пример: A + B

A

10	34	12	34	54	113	...	1
----	----	----	----	----	-----	-----	---

+

B

32	12	57	12	14	126	...	5
----	----	----	----	----	-----	-----	---

||

C

42	46	69	46	68	239	...	6
----	----	----	----	----	-----	-----	---

Пример: A + B

A

10	34	12	34	54	113	...	1
----	----	----	----	----	-----	-----	---

+

B

32	12	57	12	14	126	...	5
----	----	----	----	----	-----	-----	---

||

C

42	46	69	46	68	239	...	6
----	----	----	----	----	-----	-----	---

```
void solveCPU(int[] a, int[] b, int c[], int n) {  
    for (int i = 0; i < n; ++i) {  
        int sum = a[i] + b[i];  
        c[i] = sum;  
    }  
}
```

Пример: A + B

i=0

A	10	34	12	34	54	113	...	1
+								
B	32	12	57	12	14	126	...	5
								
C	42	46	69	46	68	239	...	6

```
void solveCPU(int[] a, int[] b, int c[], int n) {  
    for (int i = 0; i < n; ++i) {  
        int sum = a[i] + b[i];  
        c[i] = sum;  
    }  
}
```

Пример: A + B

	i=0	i=1					
A	10	34	12	34	54	113 ...	1
+	+	+					
B	32	12	57	12	14	126 ...	5
	↓	↓					
C	42	46	69	46	68	239 ...	6

```
void solveCPU(int[] a, int[] b, int c[], int n) {  
    for (int i = 0; i < n; ++i) {  
        int sum = a[i] + b[i];  
        c[i] = sum;  
    }  
}
```

Пример: A + B

	i=0	i=1	i=2				
A	10	34	12	34	54	113 ...	1
+	+	+	+				
B	32	12	57	12	14	126 ...	5
	↓	↓	↓				
C	42	46	69	46	68	239 ...	6

```
void solveCPU(int[] a, int[] b, int c[], int n) {  
    for (int i = 0; i < n; ++i) {  
        int sum = a[i] + b[i];  
        c[i] = sum;  
    }  
}
```

Пример: A + B

	i=0	i=1	i=2	i=3	i=4	i=5	i=N-1
A	10	34	12	34	54	113 ...	1
+	+	+	+	+	+	+	+
B	32	12	57	12	14	126 ...	5
	↓	↓	↓	↓	↓	↓	↓
C	42	46	69	46	68	239 ...	6

```
void solveCPU(int[] a, int[] b, int c[], int n) {  
    for (int i = 0; i < n; ++i) {  
        int sum = a[i] + b[i];  
        c[i] = sum;  
    }  
}
```

**Какая
асимптотика?**

Пример: A + B

	i=0	i=1	i=2	i=3	i=4	i=5		i=N-1
A	10	34	12	34	54	113	...	1
+	+	+	+	+	+	+		+
B	32	12	57	12	14	126	...	5
	↓	↓	↓	↓	↓	↓		↓
C	42	46	69	46	68	239	...	6

```
void solveCPU(int[] a, int[] b, int c[], int n) {  
    for (int i = 0; i < n; ++i) {  
        int sum = a[i] + b[i];  
        c[i] = sum;  
    }  
}
```

O(N)

Пример: A + B

A	10	34	12	34	54	113	...	1
+								
B	32	12	57	12	14	126	...	5
C	42	46	69	46	68	239	...	6



NVIDIA RTX 4090 - 16 тысяч ядер!

Пример: A + B

A	10	34	12	34	54	113	...	1
+								
B	32	12	57	12	14	126	...	5
C	42	46	69	46	68	239	...	6



```
void solveCPU(int[] a, int[] b, int c[], int n) {  
    for (int i = 0; i < n; ++i) {  
        int sum = a[i] + b[i];  
        c[i] = sum;  
    }  
}
```

O(N)

```
void solveGPU(int[] a, int[] b, int c[], int n) {  
    const int i = get_global_id(0);  
    c[i] = b[i] + a[i];  
}
```

Супер многопоточно!

NVIDIA RTX 4090 - 16 тысяч ядер!



Пример: A + B

A	10	34	12	34	54	113	...	1
+								
B	32	12	57	12	14	126	...	5
C	42	46	69	46	68	239	...	6



```
void solveCPU(int[] a, int[] b, int c[], int n) {  
    for (int i = 0; i < n; ++i) {  
        int sum = a[i] + b[i];  
        c[i] = sum;  
    }  
}
```

O(N)

```
void solveGPU(int[] a, int[] b, int c[], int n) {  
    const int i = get_global_id(0);  
    c[i] = b[i] + a[i];  
}
```

Какая асимптотика?

NVIDIA RTX 4090 - 16 тысяч ядер!



Пример: A + B

	i=0	i=1	i=2	i=3	i=4	i=5	i=N-1
A	10	34	12	34	54	113 ...	1
+	+	+	+	+	+	+	+
B	32	12	57	12	14	126 ...	5
	↓	↓	↓	↓	↓	↓	↓
C	42	46	69	46	68	239 ...	6

```
void solveCPU(int[] a, int[] b, int c[], int n) {  
    for (int i = 0; i < n; ++i) {  
        int sum = a[i] + b[i];  
        c[i] = sum;  
    }  
}
```

O(N)

```
void solveGPU(int[] a, int[] b, int c[], int n) {  
    const int i = get_global_id(0);  
    c[i] = b[i] + a[i];  
}
```

O(N / 16384)

NVIDIA RTX 4090 - 16 тысяч ядер!



OpenCL™

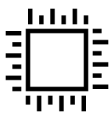
NVIDIA
CUDA®

Vulkan

Пример: A + B (N=100.000.000)

```
for (size_t i = 0; i < n; ++i) {  
    cs[i] = as[i] + bs[i];  
}
```

Пример: A + B (N=100.000.000)



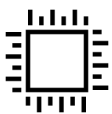
CPU - Intel 13700K

```
for (size_t i = 0; i < n; ++i) {  
    cs[i] = as[i] + bs[i];  
}
```

a + b median time: 0.068 sec (+-0.00481664)

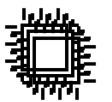
a + b median RAM bandwidth: 16.4351 GB/s

Пример: A + B (N=100.000.000)



CPU - Intel 13700K

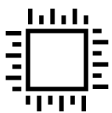
```
for (size_t i = 0; i < n; ++i) {  
    cs[i] = as[i] + bs[i];  
}  
  
a + b median time: 0.068 sec (+-0.00481664)  
a + b median RAM bandwidth: 16.4351 GB/s
```



CPU - Intel 13700K

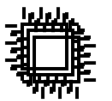
```
#pragma omp parallel for  
for (ptrdiff_t i = 0; i < n; ++i) {  
    cs[i] = as[i] + bs[i];  
}  
  
a + b median time: 0.047 sec (+-0.00153623)  
a + b median RAM bandwidth: 23.7784 GB/s
```

Пример: A + B (N=100.000.000)



CPU - Intel 13700K

```
for (size_t i = 0; i < n; ++i) {  
    cs[i] = as[i] + bs[i];  
}  
  
a + b median time: 0.068 sec (+-0.00481664)  
a + b median RAM bandwidth: 16.4351 GB/s
```



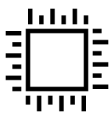
CPU - Intel 13700K

```
#pragma omp parallel for  
for (ptrdiff_t i = 0; i < n; ++i) {  
    cs[i] = as[i] + bs[i];  
}  
  
a + b median time: 0.047 sec (+-0.00153623)  
a + b median RAM bandwidth: 23.7784 GB/s
```

**Почему разница такая
незначительная?**

**Как проверить гипотезу?
Как спровоцировать
значительную разницу?**

Пример: **100*cos**(A + B) (N=100.000.000)

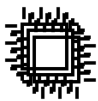


CPU - Intel 13700K

```
for (size_t i = 0; i < n; ++i) {  
    cs[i] = 100.0*cos( Left: as[i] + bs[i]);  
}
```

median time: 1.178 sec (+-0.0235009)

median RAM bandwidth: 0.948716 GB/s



CPU - Intel 13700K

```
#pragma omp parallel for
```

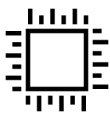
```
for (ptrdiff_t i = 0; i < n; ++i) {  
    cs[i] = 100*cos( Left: as[i] + bs[i]);  
}
```

median time: 0.109 sec (+-0.0121988)

median RAM bandwidth: 10.2531 GB/s

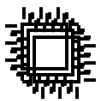
x10.9

Пример: A + B (N=100.000.000)



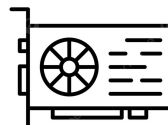
CPU - Intel 13700K

```
for (size_t i = 0; i < n; ++i) {  
    cs[i] = as[i] + bs[i];  
}  
  
a + b median time: 0.068 sec (+-0.00481664)  
a + b median RAM bandwidth: 16.4351 GB/s
```



CPU - Intel 13700K

```
#pragma omp parallel for  
for (ptrdiff_t i = 0; i < n; ++i) {  
    cs[i] = as[i] + bs[i];  
}  
  
a + b median time: 0.047 sec (+-0.00153623)  
a + b median RAM bandwidth: 23.7784 GB/s
```

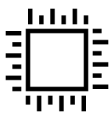


GPU - NVIDIA RTX 4090

```
__kernel void aplusb(__global const float* a,  
                    __global const float* b,  
                    __global float* c,  
                    unsigned int n)  
{  
  
    const unsigned int index = get_global_id( dimindx: 0 );  
    if (index >= n)  
        return;  
  
    c[index] = a[index] + b[index];  
}
```

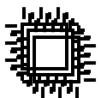
```
a + b median time: 0.002 sec (+-0.000632456)  
a + b median VRAM bandwidth: 558.794 GB/s
```

Пример: A + B (N=100.000.000)



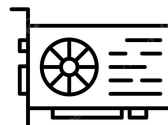
CPU - Intel 13700K

```
for (size_t i = 0; i < n; ++i) {  
    cs[i] = as[i] + bs[i];  
}  
  
a + b median time: 0.068 sec (+-0.00481664)  
a + b median RAM bandwidth: 16.4351 GB/s
```



CPU - Intel 13700K

```
#pragma omp parallel for  
for (ptrdiff_t i = 0; i < n; ++i) {  
    cs[i] = as[i] + bs[i];  
}  
  
a + b median time: 0.047 sec (+-0.00153623)  
a + b median RAM bandwidth: 23.7784 GB/s
```

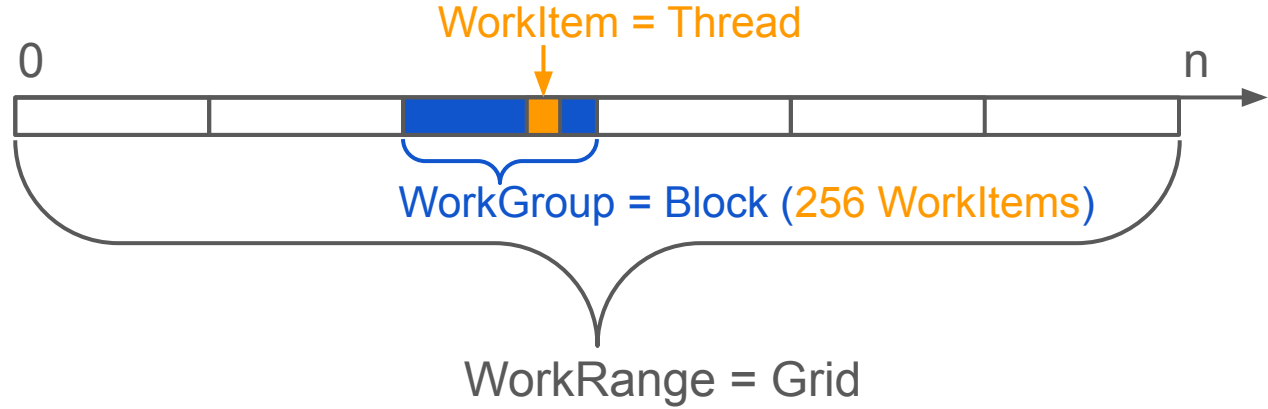


GPU - NVIDIA RTX 4090

```
__kernel void aplusb(__global const float* a,  
                    __global const float* b,  
                    __global float* c,  
                    unsigned int n)  
{  
  
    const unsigned int index = get_global_id( dimindx: 0 );  
    if (index >= n)  
        return;  
  
    c[index] = a[index] + b[index];  
}  
  
a + b median time: 0.002 sec (+-0.000632456)  
a + b median VRAM bandwidth: 558.794 GB/s
```

x23.5

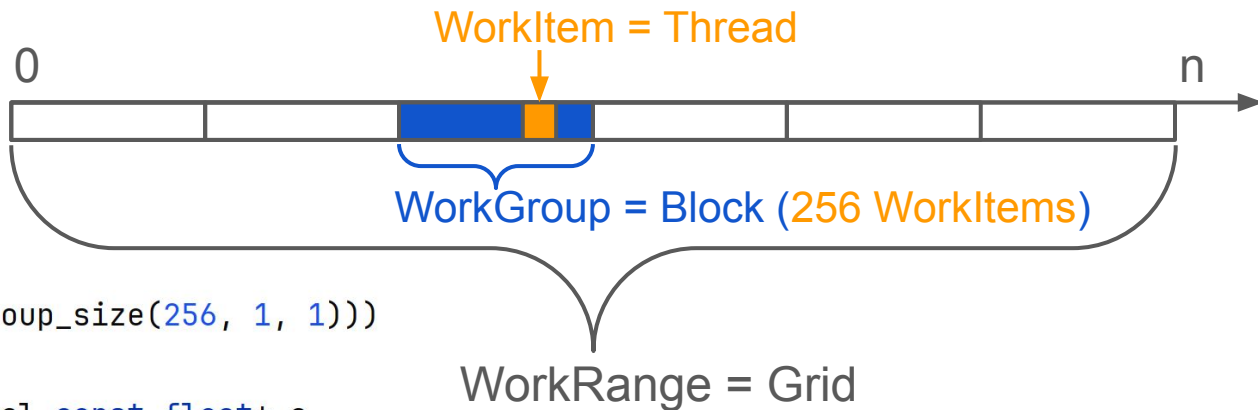
Пример: $A + B$ ($N=100.000.000$)



Пример: A + B (N=100.000.000)

```
1 #ifdef __CLION_IDE__  
2 #include <libgpu/opencl/cl/clion_defines.cl>  
3 #endif
```

```
4  
5 #line 5
```



```
6  
7  
8  
9  
10 __attribute__((reqd_work_group_size(256, 1, 1)))
```

```
11  
12 __kernel void aplusb(__global const float* a,  
13                     __global const float* b,  
14                     __global float* c,  
15                     unsigned int n)
```

```
16 {
```

```
17     const unsigned int index = get_global_id(dimindx: 0);
```

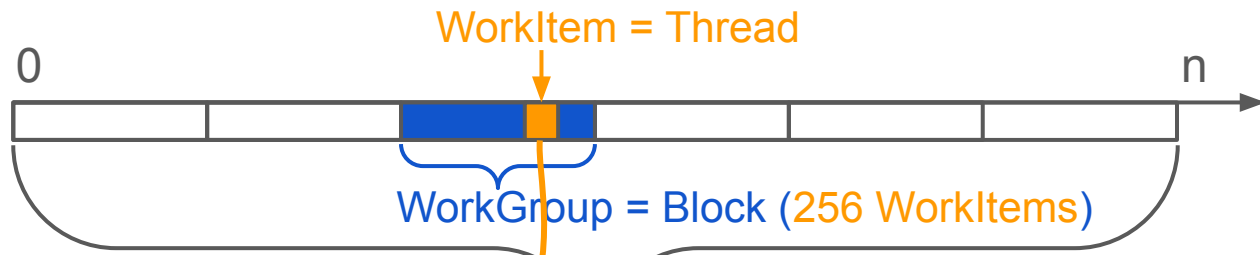
```
18     if (index >= n)
```

```
19         return;
```

```
20  
21     c[index] = a[index] + b[index];
```

```
22 }
```

Пример: A + B (N=100.000.000)



#line 5

```
__attribute__((reqd_work_group_size(256, 1, 1)))
```

```
__kernel void aplusb(__global const float* a,  
                    __global const float* b,  
                    __global float* c,  
                    unsigned int n)
```

```
{
```

```
    const unsigned int index = get_global_id(dimindx: 0);
```

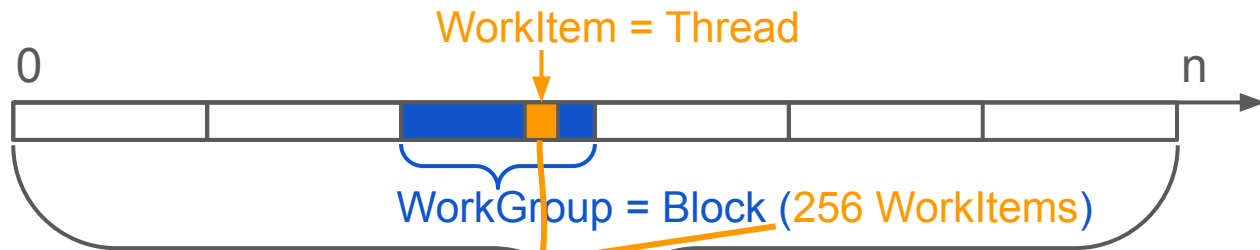
```
    if (index >= n)
```

```
        return;
```

```
    c[index] = a[index] + b[index];
```

```
}
```

Пример: A + B (N=100.000.000)



#line 5

```
__attribute__((reqd_work_group_size(256, 1, 1)))
```

```
__kernel void aplusb(__global const float* a,  
                    __global const float* b,  
                    __global float* c,  
                    unsigned int n)
```

```
{
```

```
    const unsigned int index = get_global_id( dimindx: 0);
```

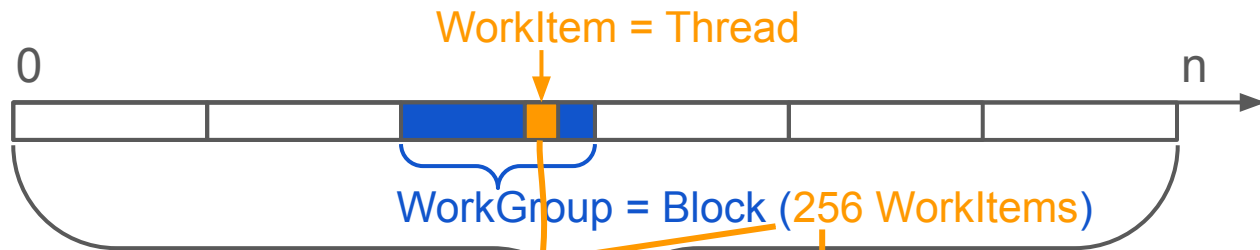
```
    if (index >= n)
```

```
        return;
```

```
    c[index] = a[index] + b[index];
```

```
}
```

Пример: A + B (N=100.000.000)



WorkRange = Grid

```
1 #ifndef __CLION_IDE__
2 #include <libgpu/opencl/cl/clion_defines.cl>
3 #endif
```

#line 5

```
10 __attribute__((reqd_work_group_size(256, 1, 1)))
```

```
12 __kernel void aplusb(__global const float* a,
13                     __global const float* b,
14                     __global float* c,
15                     unsigned int n)
```

```
16 {
```

```
17     const unsigned int index = get_global_id( dimindx: 0);
```

```
18     if (index >= n)
```

```
19         return;
```

```
21     c[index] = a[index] + b[index];
```

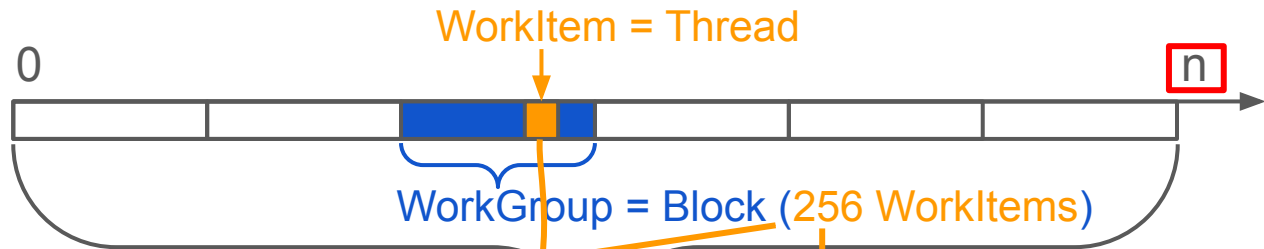
```
22 }
```

HOST-side (CPU, C++)

```
gpu::WorkSize workSize(GROUP_SIZE, n);
```

```
ocl_aplusb.exec(workSize, a_gpu, b_gpu, c_gpu, n);
```

Пример: A + B (N=100.000.000)



А где задается
размер рабочего
пространства?

```
1 #ifndef __CLION_IDE__
2 #include <libgpu/opencl/cl/clion_defines.cl>
3 #endif
```

#line 5

```
10 __attribute__((reqd_work_group_size(256, 1, 1)))
```

```
12 __kernel void aplusb(__global const float* a,
13                     __global const float* b,
14                     __global float* c,
15                     unsigned int n)
```

```
16 {
```

```
17     const unsigned int index = get_global_id( dimindx: 0);
```

```
18     if (index >= n)
```

```
19         return;
```

```
21     c[index] = a[index] + b[index];
```

```
22 }
```

WorkRange = Grid

HOST-side (CPU, C++)

```
gpu::WorkSize workSize(GROUP_SIZE, n);
```

```
ocl_aplusb.exec(workSize, a_gpu, b_gpu, c_gpu, n);
```

Пример: A + B (N=100.000.000)



```
1 #ifndef __CLION_IDE__
2 #include <libgpu/opencl/cl/clion_defines.cl>
3 #endif
```

#line 5

```
10 __attribute__((reqd_work_group_size(256, 1, 1)))
```

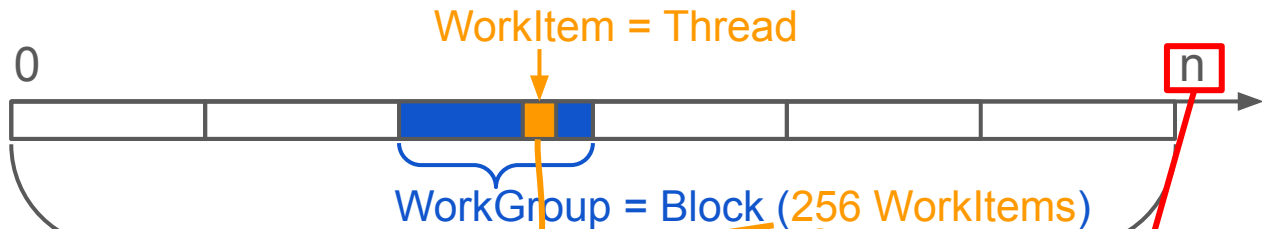
```
12 __kernel void aplusb(__global const float* a,
13                     __global const float* b,
14                     __global float* c,
15                     unsigned int n)
```

```
16 {
17     const unsigned int index = get_global_id( dimindx: 0);
```

```
18     if (index >= n)
19         return;
```

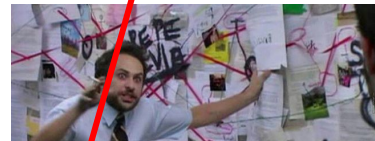
```
21     c[index] = a[index] + b[index];
```

```
22 }
```



WorkRange = Grid

А где задается
размер рабочего
пространства?



HOST-side (CPU, C++)

```
gpu::WorkSize workSize(GROUP_SIZE, n);
```

```
ocl_aplusb.exec(workSize, a_gpu, b_gpu, c_gpu, n);
```

Пример: A + B (N=100.000.000)



```
1 #ifndef __CLION_IDE__
2 #include <libgpu/opencl/cl/clion_defines.cl>
3 #endif
```

#line 5

```
10 __attribute__((reqd_work_group_size(256, 1, 1)))
```

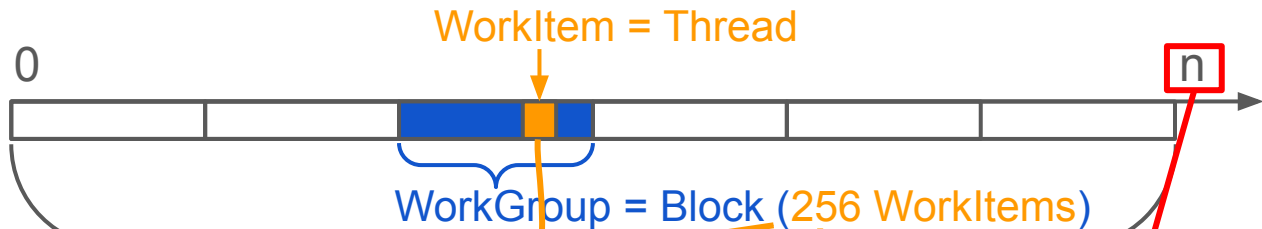
```
12 __kernel void aplusb(__global const float* a,
13                     __global const float* b,
14                     __global float* c,
15                     unsigned int n)
```

```
16 {
17     const unsigned int index = get_global_id( dimindx: 0);
```

```
18     if (index >= n)
19         return;
```

```
21     c[index] = a[index] + b[index];
```

```
22 }
```



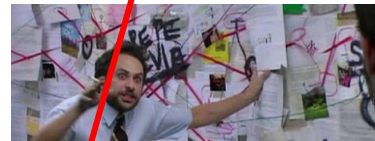
WorkRange = Grid

А где задается
размер рабочего
пространства?

HOST-side (CPU, C++)

```
gpu::WorkSize workSize(GROUP_SIZE, n);
```

```
ocl_aplusb.exec(workSize, a_gpu, b_gpu, c_gpu, n);
```



Пример: A + B (N=100.000.000)



```
1 #ifndef __CLION_IDE__
2 #include <libgpu/opencl/cl/clion_defines.cl>
3 #endif
```

#line 5

```
10 __attribute__((reqd_work_group_size(256, 1, 1)))
```

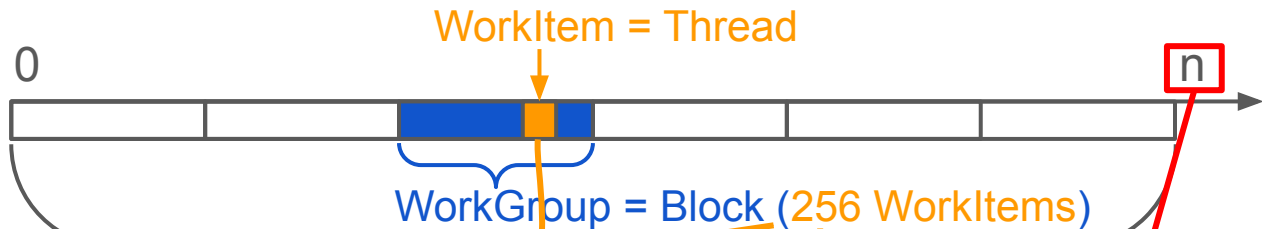
```
12 __kernel void aplusb(__global const float* a,
13                     __global const float* b,
14                     __global float* c,
15                     unsigned int n)
```

```
16 {
17     const unsigned int index = get_global_id( dimindx: 0);
```

```
18     if (index >= n)
19         return;
```

```
21     c[index] = a[index] + b[index];
```

```
22 }
```



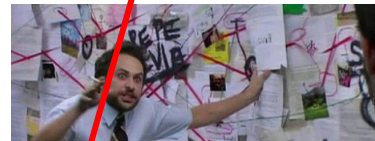
WorkRange = Grid

А где задается
размер рабочего
пространства?

HOST-side (CPU, C++)

```
gpu::WorkSize workSize(GROUP_SIZE, n);
```

```
ocl_aplusb.exec(workSize, a_gpu, b_gpu, c_gpu, n);
```



Пример: A + B (N=100.000.000)

```
1 #ifdef __CLION_IDE__
```

```
2 #include <libgpu/opencl/cl/clion_defines.cl>
```

```
3 #endif
```

```
4  
5 #line 5
```



```
6  
7  
8  
9  
10 __attribute__((reqd_work_group_size(256, 1, 1)))
```

```
11  
12 __kernel void aplusb(__global const float* a,  
13                     __global const float* b,  
14                     __global float* c,  
15                     unsigned int n)
```

```
16 {
```

```
17     const unsigned int index = get_global_id(dimindx: 0);
```

```
18     if (index >= n)
```

```
19         return;
```

```
20  
21     c[index] = a[index] + b[index];
```

```
22 }
```



```
blockIdx.x * blockDim.x + threadIdx.x;
```

```
1 __global__ void aplusb(const float* a,  
2                       const float* b,  
3                       float* c,  
4                       unsigned int n)
```

```
5 {
```

```
6     const unsigned int index = blockIdx.x * blockDim.x + threadIdx.x;
```

```
7     if (index >= n)
```

```
8         return;
```

```
9  
10     c[index] = a[index] + b[index];
```

```
11 }
```

Пример: A + B (N=100.000.000)



```
1  __global__ void aplusb(const float* a,  
2                          const float* b,  
3                          float* c,  
4                          unsigned int n)  
5  {  
6      const unsigned int index = blockIdx.x * blockDim.x + threadIdx.x;  
7      if (index >= n)          WorkGroup = Block (256 WorkItems)  
8          return;  
9  
10     c[index] = a[index] + b[index];  
11 }
```

HOST-side (CPU, C++)

Пример: $A + B$ ($N=100.000.000$)

```
12 void cuda_aplusb(const gpu::WorkSize &workSize,  
13                 const float* a, const float* b, float* c, unsigned int n,  
14                 cudaStream_t stream)  
15 {  
16     const unsigned int blockSize = 256;  
17     const unsigned int gridSize = (n + blockSize - 1) / blockSize;  
18     aplusb<<<gridSize, blockSize, 0, stream>>>(a, b, c, n);  
19     CUDA_CHECK_KERNEL(stream);  
20 }  
21
```



```
1 __global__ void aplusb(const float* a,  
2                       const float* b,  
3                       float* c,  
4                       unsigned int n)  
5 {  
6     const unsigned int index = blockIdx.x * blockDim.x + threadIdx.x;  
7     if (index >= n) WorkGroup = Block (256 WorkItems)  
8         return;  
9  
10    c[index] = a[index] + b[index];  
11 }
```

HOST-side (CPU, C++)

Пример: $A + B$ ($N=100.000.000$)

```
12 void cuda_aplusb(const gpu::WorkSize &workSize,  
13                 const float* a, const float* b, float* c, unsigned int n,  
14                 cudaStream_t stream)  
15 {  
16     const unsigned int blockSize = 256;  
17     const unsigned int gridSize = (n + blockSize - 1) / blockSize;  
18     aplusb<<<gridSize, blockSize, 0, stream>>>(a, b, c, n);  
19     CUDA_CHECK_KERNEL(stream);  
20 }  
21
```



NVIDIA
CUDA

А где задается
размер рабочего
пространства?

```
1 __global__ void aplusb(const float* a,  
2                       const float* b,  
3                       float* c,  
4                       unsigned int n)  
5 {  
6     const unsigned int index = blockIdx.x * blockDim.x + threadIdx.x;  
7     if (index >= n) WorkGroup = Block (256 WorkItems)  
8         return;  
9  
10    c[index] = a[index] + b[index];  
11 }
```

HOST-side (CPU, C++)

Пример: A + B (N=100.000.000)

```
12 void cuda_aplusb(const gpu::WorkSize &workSize,  
13                 const float* a, const float* b, float* c, unsigned int n,  
14                 cudaStream_t stream)  
15 {  
16     const unsigned int blockSize = 256;  
17     const unsigned int gridSize = (n + blockSize - 1) / blockSize;  
18     aplusb<<gridSize, blockSize, 0, stream>>>(a, b, c, n);  
19     CUDA_CHECK_KERNEL(stream);  
20 }  
21
```



А где задается
размер рабочего
пространства?

```
1 __global__ void aplusb(const float* a,  
2                       const float* b,  
3                       float* c,  
4                       unsigned int n)  
5 {  
6     const unsigned int index = blockIdx.x * blockDim.x + threadIdx.x;  
7     if (index >= n)           WorkGroup = Block (256 WorkItems)  
8         return;  
9  
10    c[index] = a[index] + b[index];  
11 }
```

Пример: A + B (N=100.000.000)

GLSL (Graphics Library Shading Language)



```
#version 450
#include <libgpu/vulkan/vk/common.vk>
```

```
layout (std430, binding = 0) readonly buffer AsIn { uint as[]; };
layout (std430, binding = 1) readonly buffer BsIn { uint bs[]; };
layout (std430, binding = 2) writeonly buffer CsOut { uint cs[]; };
```

```
layout (push_constant) uniform PushConstants {
    uint n;
} params;
```

```
layout (local_size_x = 256) in;
```

```
void main()
```

```
{
```

```
    const uint index = gl_GlobalInvocationID.x;
```

```
    if (index >= params.n)
```

```
        return;
```

```
    cs[index] = as[index] + bs[index];
```

```
}
```

```
__kernel void aplusb_matrix(__global const uint* a,  
                           __global const uint* b,  
                           __global      uint* c,  
                           unsigned int width,  
                           unsigned int height)
```

```
{
```

```
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < width; ++k) {
```

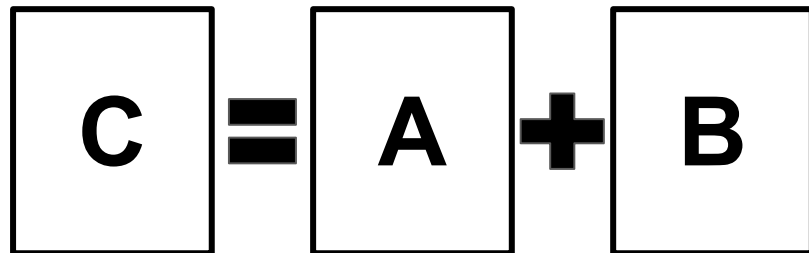
```
        unsigned int col = k;
```

```
        unsigned int row = index;
```

```
        c[row * width + col] = a[row * width + col]  
                               + b[row * width + col];
```

```
    }
```

```
}
```



```
__kernel void aplusb_matrix(__global const uint* a,  
                            __global const uint* b,  
                            __global      uint* c,  
                            unsigned int width,  
                            unsigned int height)
```

```
{
```

```
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < width; ++k) {
```

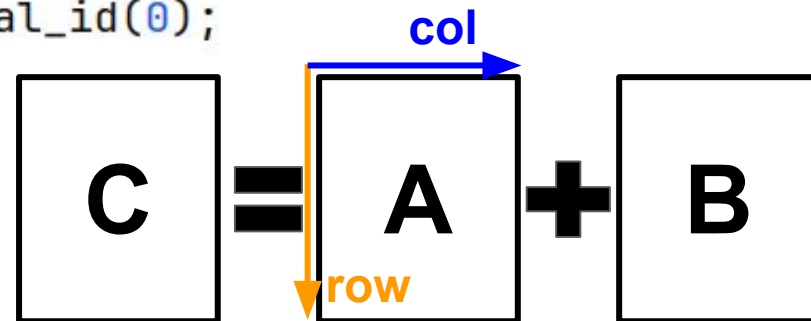
```
        unsigned int col = k;
```

```
        unsigned int row = index;
```

```
        c[row * width + col] = a[row * width + col]  
                                + b[row * width + col];
```

```
    }
```

```
}
```



```
__kernel void aplusb_matrix(__global const uint* a,
                           __global const uint* b,
                           __global      uint* c,
                           unsigned int width,
                           unsigned int height)
```

```
{
```

```
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < width; ++k) {
```

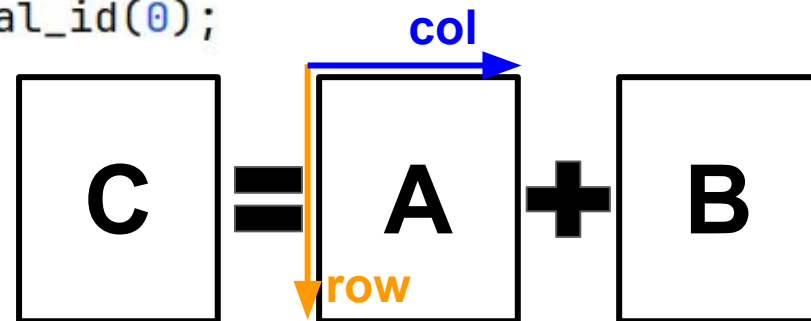
```
        unsigned int col = k;
```

```
        unsigned int row = index;
```

```
        c[row * width + col] = a[row * width + col]
                               + b[row * width + col];
```

```
    }
```

```
}
```



coalesced memory access?

```
__kernel void aplusb_matrix(__global const uint* a,
                           __global const uint* b,
                           __global      uint* c,
                           unsigned int width,
                           unsigned int height)
```

```
{
```

```
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < width; ++k) {
```

```
        unsigned int col = k;
```

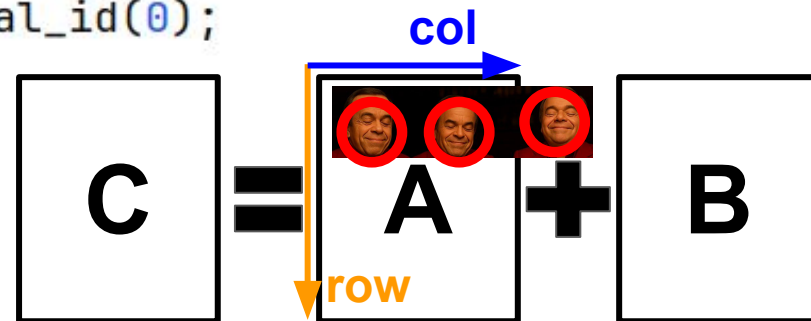
```
        unsigned int row = index;
```

```
        c[row * width + col] = a[row * width + col]
                               + b[row * width + col];
```

```
    }
```

```
}
```

Как выглядит *warp*?



coalesced memory access?

```
__kernel void aplusb_matrix(__global const uint* a,
                           __global const uint* b,
                           __global      uint* c,
                           unsigned int width,
                           unsigned int height)
```

```
{
```

```
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < width; ++k) {
```

```
        unsigned int col = k;
```

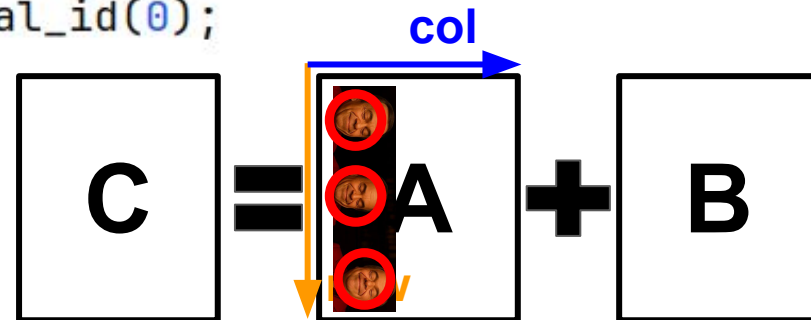
```
        unsigned int row = index;
```

```
        c[row * width + col] = a[row * width + col]
                               + b[row * width + col];
```

```
    }
```

```
}
```

Как выглядит *warp*?



coalesced memory access?

```
__kernel void aplusb_matrix(__global const uint* a,
                           __global const uint* b,
                           __global      uint* c,
                           unsigned int width,
                           unsigned int height)
```

HE coalesced memory access!!!

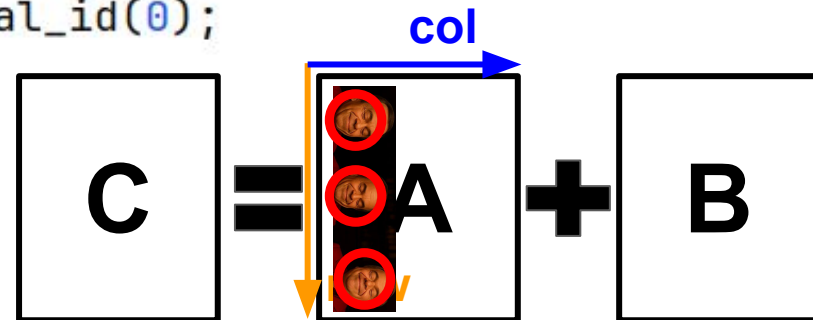
```
{
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < width; ++k) {
```

```
        unsigned int col = k;
```

```
        unsigned int row = index;
```

```
        c[row * width + col] = a[row * width + col]
                               + b[row * width + col];
```



```
__kernel void aplusb_matrix(__global const uint* a,  
                           __global const uint* b,  
                           __global      uint* c,  
                           unsigned int width,  
                           unsigned int height)
```

```
{
```

```
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < height; ++k) {
```

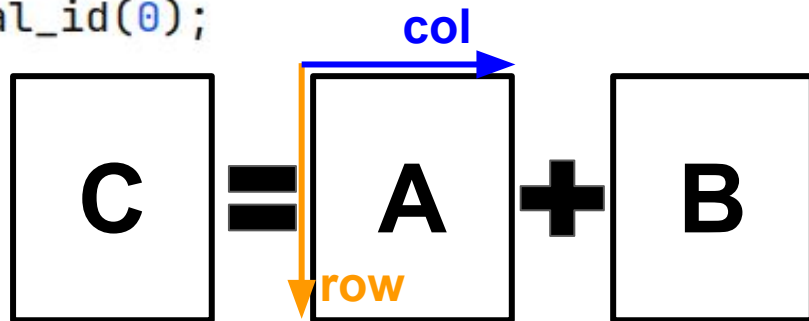
```
        unsigned int col = index;
```

```
        unsigned int row = k;
```

```
        c[row * width + col] = a[row * width + col]  
                               + b[row * width + col];
```

```
    }
```

```
}
```



```
__kernel void aplusb_matrix(__global const uint* a,  
                            __global const uint* b,  
                            __global      uint* c,  
                            unsigned int width,  
                            unsigned int height)
```

```
{
```

```
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < height; ++k) {
```

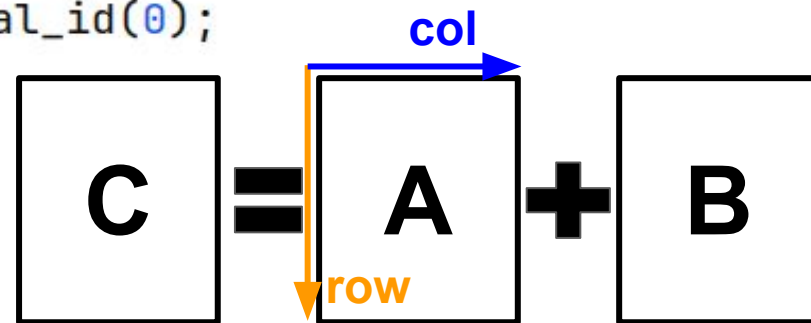
```
        unsigned int col = index;
```

```
        unsigned int row = k;
```

```
        c[row * width + col] = a[row * width + col]  
                                + b[row * width + col];
```

```
    }
```

```
}
```



coalesced memory access?

```
__kernel void aplusb_matrix(__global const uint* a,
                           __global const uint* b,
                           __global      uint* c,
                           unsigned int width,
                           unsigned int height)
```

```
{
```

```
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < height; ++k) {
```

```
        unsigned int col = index;
```

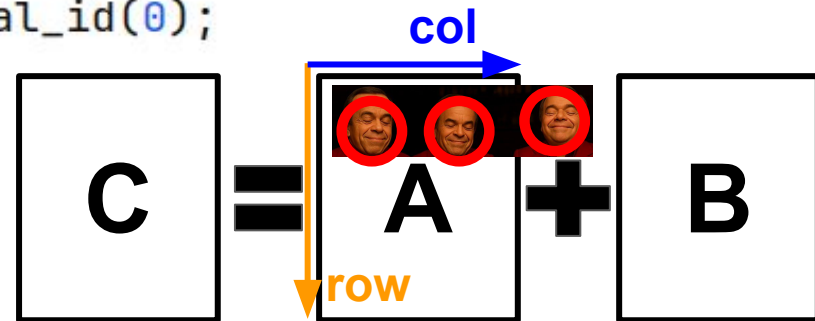
```
        unsigned int row = k;
```

```
        c[row * width + col] = a[row * width + col]
                               + b[row * width + col];
```

```
    }
```

```
}
```

Как выглядит *warp*?



coalesced memory access?

```
__kernel void aplusb_matrix(__global const uint* a,
                           __global const uint* b,
                           __global      uint* c,
                           unsigned int width,
                           unsigned int height)
```

```
{
```

```
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < height; ++k) {
```

```
        unsigned int col = index;
```

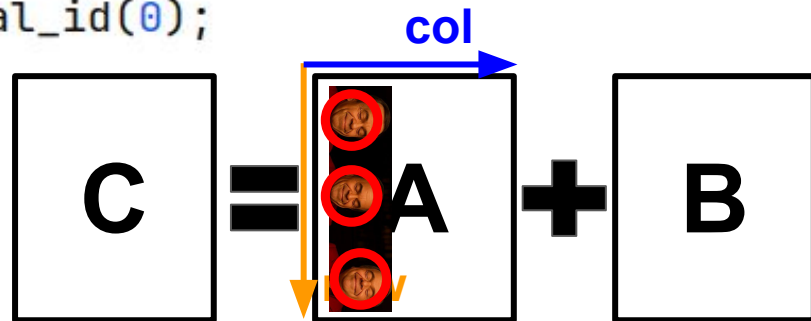
```
        unsigned int row = k;
```

```
        c[row * width + col] = a[row * width + col]
                               + b[row * width + col];
```

```
    }
```

```
}
```

Как выглядит *warp*?



coalesced memory access?

```
__kernel void aplusb_matrix(__global const uint* a,
                           __global const uint* b,
                           __global      uint* c,
                           unsigned int width,
                           unsigned int height)
```

```
{
```

coalesced memory access!!!

```
    const unsigned int index = get_global_id(0);
```

```
    for (int k = 0; k < height; ++k) {
```

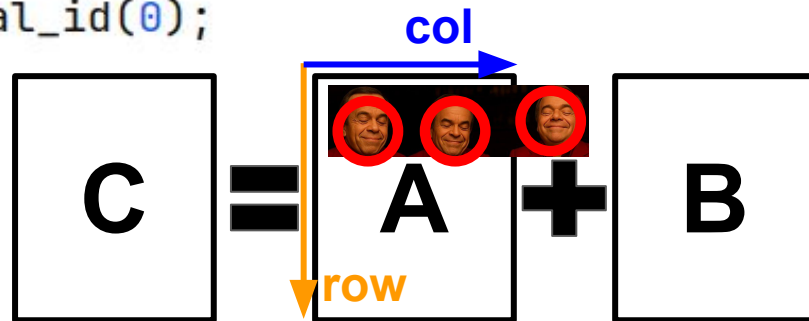
```
        unsigned int col = index;
```

```
        unsigned int row = k;
```

```
        c[row * width + col] = a[row * width + col]
                               + b[row * width + col];
```

```
    }
```

```
}
```





 **Vulkan**



nVIDIA
CUDA

 **OpenCL™**

Вопросы?



@UnicornGlade



@PolarNick239



polarnick239@gmail.com



Николай Полярный

Activate Ubuntu
Go to Settings to activate Ubuntu.